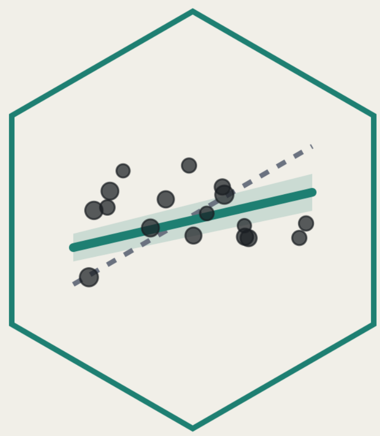


Statistical Practicals

MED5652

Introduction to
Epidemiology & Statistics

Fred Ho

2026-08-12

Table of contents

Introduction

What this book is

This is a companion ebook for the statistical practicals of the course MED5652 Introduction to Epidemiology and Statistics for UofG MPH students. The principles and theories of statistics are taught in the seminars. This ebook, and the practical sessions in it, are about applying that knowledge in epidemiology and statistics to real data using R.

The first two preparatory sessions help you ease into the analysis environment, since R is primarily a programming language, and we don't expect you to have prior analysis or programming experience. Then each weekly practical session follows a seminar. Every practical picks up from the theories taught in the seminars, but it never reproduces the seminar's content, so don't expect this book to substitute for going to class. Think of the seminar as where an idea is introduced, and the practical as where you actually do it, in R, on real data.

Where this book sits against the module's outcomes

MED5652 as a whole has four intended learning outcomes. By the end of the module, you will be able to:

1. Critically assess the role of epidemiology and statistics within public health.
2. Critically appraise key concepts in epidemiological design.
3. Critically appraise and apply key statistical methods.
4. Critically appraise and evaluate the concept of causality within public health literature.

This book is built primarily to get you through ILO 3. Along the way, it also touches ILO 1, since statistics only matters here because of the public health decisions it informs, and ILO 4, to explore causality using statistical methods.

Structure of the book

Two parts. First, two preparatory practicals, worked through before seminars on statistics:

IES week	What it covers
4	R and R Studio
5	Data and data handling

IES Week 6 is reading week, so there's no practical for it. Then five weekly practicals, one per statistics week, each following that week's seminar:

IES week	What it covers
7	Describing and visualising data
8	Estimation, precision, and study size
9	Hypothesis tests and measures of association
10	Correlation and simple linear regression
11	Multiple linear and logistic regression

That list looks like five separate topics, but it isn't. Each week is a stage of one thing. IES Week 7 describes a variable. IES Week 8 attaches a confidence interval to that description, so we can say how precise it is. IES Week 9 compares two groups, with a test and with the risk ratios and odds ratios that you've learnt in the epidemiology seminars. IES Week 10 models a relationship between two variables directly, with a regression line. IES Week 11 puts several variables into that model at once, so we can adjust one association for others, and extends the same idea to a binary outcome with logistic regression. By the end, the separate pieces from IES Weeks 7 to 9 are sitting inside one regression model.

Structure of a chapter

Every chapter in this book follows the same structure, so once you've worked through one, you know what to expect from the rest. It opens with a short list of learning objectives, then a setup chunk that reads in the data. After that comes the content itself, worked through with code and output, followed by a full worked example that puts several pieces together at once. Numbered exercises come next, each one tied to a specific ILO. Each of the five weekly chapters then ends with a writing exercise: a short results-section paragraph, not a calculation. These aren't decoration. The statistics assessment is a structured question and answer, combining analysis with written interpretation, and the writing exercises are deliberate practices for that.

Two chapters fold the worked example into the chapter's own shape rather than setting it apart as its own section. IES Week 5 builds toward one continuously, ending in your own first script. IES Week 11 opens with an exercise instead, deriving the outcome every model in that chapter needs, and its crude-versus-adjusted and sensitivity-analysis sections do the work a worked example would elsewhere.

The preparatory chapters work a little differently: each include multiple-choice questions instead, aimed at the places R's syntax most often trips people up, and IES Week 5 has a small writing exercise as well. Solutions to the numbered exercises, the self-checks, and the writing exercise usually sit in a collapsed section at the very end of each chapter. IES Week 11 is the one exception: its opening exercise is answered inline, right where it's set, so you can attempt it before reading on rather than jumping to the end. Chapters also carry the 'Try it yourself' box, which is a prompt to change one thing in code you've just run and see what happens. Those don't have written answers.

The two datasets

Almost everything in this book runs on one of two datasets. `nhanes_mph` is a curated extract of real health survey data. IES Week 5 reads it in for the first time and gets you comfortable with it as data; IES Week 7 introduces it properly and is where it becomes the dataset the rest of the book runs on. `strep_tb`, from the `medicaldata` package, is the 1948 MRC streptomycin trial, a historic randomised controlled trial, used from IES Week 8 onward wherever a binary outcome and a genuine trial design are useful. Both get introduced properly, in context, the first time that matters. Learning two datasets well, rather than a new one every week, means you spend your attention on the method, not on re-orienting yourself to unfamiliar data.

Not the be-all and end-all

This book gets you to a working, applied floor in statistics for an MPH graduate. It is not a complete statistics education, and it doesn't try to be. Several real techniques are left out on purpose. Our data carry a death indicator and a follow-up time, for instance, and IES Week 11 uses them to build a fixed-window mortality outcome, but proper time-to-event analysis, Kaplan-Meier curves, Cox regression, is never covered. Formally deciding whether a variable is a confounder or a mediator, choosing between competing regression models, testing for interaction between predictors, and handling missing data properly rather than dropping incomplete records are all left out too. Where something is skipped, we say so and name where it's picked up: an Advanced Statistics module, later in the programme. Treat every one of those signposts as a real boundary, not a gap in this book you're expected to fill in yourself. IES Week 11 closes with the full list, once you've met enough of the material for it to mean something.

How to actually use it

Work through the chapters in order. Each one assumes you've already met whatever came before it, and later chapters build on earlier code without re-explaining it. Attempt the exercises yourself before you open the solutions; the point of a collapsed solution is that you look at it after you've tried, not instead of trying. Work along in a script of your own as you go, the same way IES Week 5 shows you how to, typing the code rather than copying it. And keep this book open for the rest of the module, not just read once and set aside. It stays the reference you come back to whenever you need a reminder of how something works.

1 R and R Studio (IES Week 4)

This is a preparatory chapter to learn about the basics of R, and the RStudio interface. It will cover what a script is, how to create things in R, and how to read the handful of error messages we'll all meet in the next ten weeks. Nothing here is about statistics yet, but these are an important foundation for applying statistical analysis to data.

1.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Install R and RStudio on your own device, and verify the installation by running code and loading a package
2. Work in RStudio using a Project and a saved script, and explain why we script rather than work in the console
3. Create objects by assignment, and recognise R's basic data types and structures (numeric, character, logical, factor; vector, data frame)
4. Call a function using named arguments, explain what a default value is, and find its help page
5. Write and read a short piped sequence that includes a logical condition and extracts a column from a data frame
6. Diagnose a failed line of code from its error message
7. Build a project folder, following sensible file and folder naming conventions

1.2 Setup

There's no data to read yet, so this chapter's setup chunk is almost empty. The one thing we set is the console's print width, so that printed output stays narrow enough to fit this book's page. That's a setting for producing the book, not part of any analysis, and from IES Week 7 onward we keep it out of sight rather than opening every chapter with it. You don't need it in your own scripts.

```
options(width = 76)
```

1.3 Installing R and RStudio

R is the programming language itself, and is the software program that actually runs your code. RStudio is a separate, free program that sits on top of R and makes it far nicer to work in: a proper editor, a console, and the panes we look at next. Install R first, then RStudio, in that order. You can use R without RStudio (even though it's harder for a beginner) but not the other way round.

Installing R. Go to https://cran.r-project.org and choose the download link for your operating system, "Download R for Windows" or "Download R for macOS." For most people, you can run the installer and accept every default option.

Installing RStudio. Go to https://posit.co/download/rstudio-desktop/ and download the free RStudio Desktop installer for your operating system. Run it the same way, accepting the defaults throughout.

Verification. Installed isn't the same as working, so check before moving on. Open RStudio, type `2 + 2` into the console, the panel where typed commands run, covered properly in a moment, and press Enter. You should see `[1] 4`. Then confirm a package installs and loads too: run `install.packages("dplyr")` once, then `library(dplyr)`. Both get a proper explanation later in this chapter, under Packages; for now, just check that neither throws an error. If anything here fails, get it fixed before going on, since nothing later in this chapter works without it.

1.4 Finding your way around RStudio

1.4.1 The four panes

RStudio's window is split into four panes. Where each one sits is consistent no matter what you're doing, so it's worth learning the layout by position, not just by name.

- **Top-left: the source pane.** This only shows when you have an active script, so you most likely won't see this in RStudio out of the box. This is where you write and save scripts, the `.R` files that hold everything worth keeping. Code typed here doesn't run on its own; you have to tell RStudio to run it, which is the next thing we cover.
- **Bottom-left: the console.** This is where code actually executes, one line at a time, and where its output appears. You can type directly into the console, but anything typed there is gone once you close RStudio, which is exactly why the source pane exists.
- **Top-right: the environment pane.** This lists every object that currently exists in your R session, its type, and a preview of its contents. Every time you create something, it appears here. When something you expected to see isn't listed, that's usually the first clue something went wrong.

- **Bottom-right: a tabbed pane.** This holds the file browser, the plot viewer, the list of installed packages, and the help viewer, one tab each.

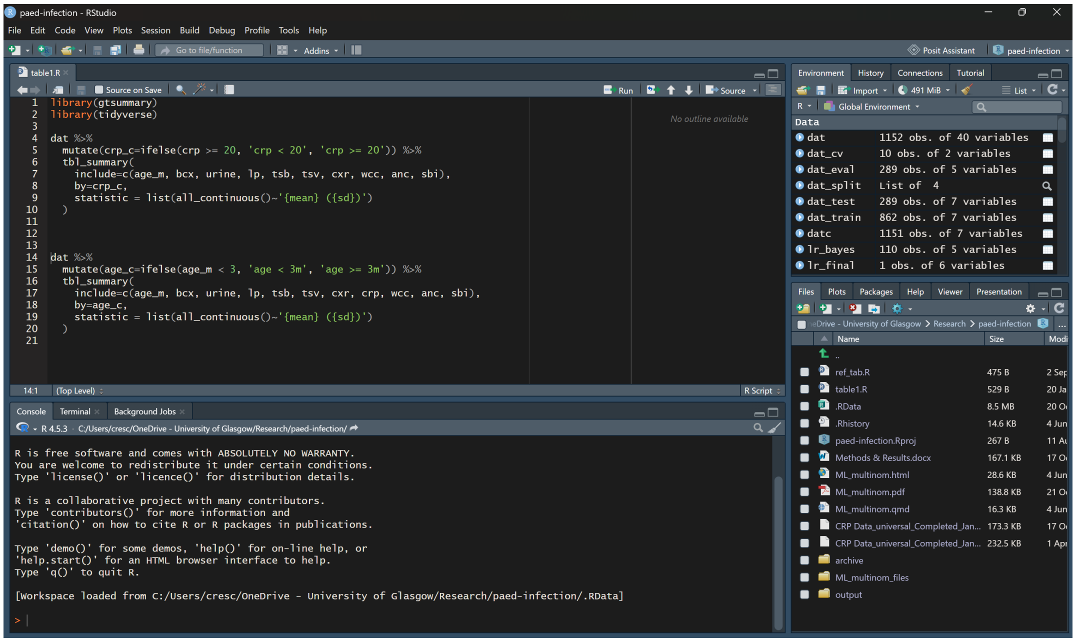


Figure 1.1: RStudio's four panes: source (top-left), console (bottom-left), environment (top-right), and a tabbed pane for files, plots, packages, and help (bottom-right). This particular window is shown in a dark colour theme rather than RStudio's own light default; it's the layout and position of the four panes that matters here, not the colours.

1.4.2 Building a project folder

Working with R means you have to manage at least a couple of files, most likely some data files and an R script file. It is usually a good idea to put them into a 'project folder'. Pick somewhere sensible on your machine, for example inside Documents. Create a new folder there and give it a short, sensible name, for example `ies_mph`, following the naming habit we'll use for every file and folder in this course from here on: no spaces, no `&`, `#`, or `%`, consistent case, and a date written as `YYYY-MM-DD` if a name needs one, so files sort in the right order. Inside that folder, create three subfolders: `data`, for anything you read in; `scripts`, for the `.R` files you write; and `outputs`, for anything you save out, a plot or a table. Keeping these three separate means you always know where to look for something later, in this course and beyond it. This folder structure isn't compulsory, just a suggestion. We'd still recommend sticking with it throughout the course, just to build the habit. Once you're more familiar with R and RStudio, feel free to work with whatever folder structure works for you.

1.4.3 Creating an RStudio Project

An RStudio Project turns that folder into something RStudio understands directly, which is what makes relative paths work rather than having to type a full path every time.

1. In RStudio's menu, go to **File → New Project…**
2. Choose **Existing Directory**, since the folder already exists.
3. Browse to the project folder you just built, and click **Create Project**.
4. RStudio reopens with that folder as the **working directory**. You can confirm this by checking the Files pane, bottom-right, which now shows the folder's contents, and the title bar, which now shows the project's name.

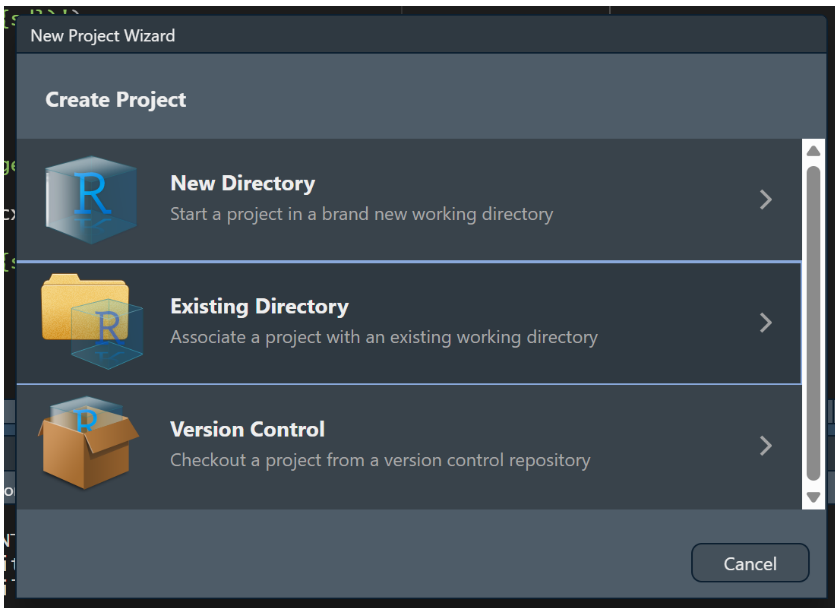


Figure 1.2: The New Project wizard, with Existing Directory highlighted.

A Project's working directory is the folder R treats as "here" when it looks for a file. Once we start reading data in IES Week 5, this is what lets a path like `data/mph_nhanes_20241107.csv` work on any machine, no matter where the project folder actually sits on that machine's disk.

1.4.4 Console versus script

You can type a line directly into the console and press Enter, and it runs immediately. That's fine for a quick check, but it isn't saved anywhere. Close RStudio, and it's gone.

For anything worth keeping, we write it in a script in the source pane instead, then run it from there:

- **Ctrl+Enter** (Windows) or **Cmd+Return** (Mac) runs the single line the cursor is on, or a selected block of lines, and sends it to the console.
- Highlighting several lines and pressing the same shortcut runs all of them in order.
- The **Source** button, top-right of the source pane, runs the entire script from top to bottom.

The reason this course insists on scripts rather than the console is simple: you will need to do this again, and you will not remember exactly what you typed. A saved script is a record you, or anyone else, can rerun from scratch and get the same result. That's the whole idea behind reproducible work, and it's why every exercise in this book asks you to write and save a script rather than work directly in the console.

1.5 Objects and assignment

R creates an object by assigning a value to a name, using `<-`:

```r
age <- 45
age
```

```
[1] 45
```

`age` now holds the value 45. It appears in the Environment pane, top right, and stays there for the rest of the session, or until you overwrite it by assigning something else to the same name.

`=` looks similar, but does a different job. Inside the parentheses of a function call, `=` doesn't create an object at all. It names which argument of the function a value belongs to:

```r
round(x = 3.14159, digits = 2)
```

```
[1] 3.14
```

Here, `x = 3.14159` doesn't create an object called `x`. It tells `round()` that 3.14159 is the value for its `x` argument. Outside a function call, `=` can actually be used for assignment too, the same way `<-` can. This course always writes `<-` for assignment and keeps `=` for naming arguments inside a function call, so the two jobs stay visually distinct rather than looking identical on the page.

1.6 Vectors and a first look at data frames

`c()` combines several values into one object, called a **vector**:

```
ages <- c(45, 62, 38, 71)
ages
```

```
[1] 45 62 38 71
```

A vector holds one type of value, several at once. A **data frame** holds several vectors side by side, as columns, which is the structure this course actually works with from the next chapter onward. Building one from two vectors previews what's coming:

```
smoker <- c("No", "Yes", "No", "Yes")
toy_data <- data.frame(ages, smoker)
toy_data
```

```
  ages smoker
1   45     No
2   62    Yes
3   38     No
4   71    Yes
```

Note that both vectors have to have the same length of data (4 values each), otherwise R will complain when you create a data frame.

1.7 Data types

R keeps track of what kind of value an object holds. `class()` reports it:

```
class(45)
```

```
[1] "numeric"
```

```
class("Yes")
```

```
[1] "character"
```

```r
class(TRUE)
```

```
[1] "logical"
```

```r
class(factor(c("Yes", "No")))
```

```
[1] "factor"
```

Four types matter throughout this course:

- **numeric**: a number, like 45
- **character**: text, like `"Yes"`, always in quotes
- **logical**: `TRUE` or `FALSE`
- **factor**: a set of categories. It looks like text but is stored with a fixed, known list of possible categories behind it. IES Week 7 turns six columns of `nhanes_mph` into factors, exactly this way, against the dictionary file.

1.8 Functions and arguments

A function is a machine to process something. Most functions take some input, process it, and output it. In the example below, `round()` is a function to round numeric values. Note that a function is always followed by a pair of parentheses `()`.

A function takes different **arguments**, basically the inputs the machine runs on. A function's arguments can be matched two ways: by **position**, in the order the function expects them, or by **name**, in any order at all.

```r
round(3.14159, 2)
```

```
[1] 3.14
```

```r
round(3.14159, digits = 2)
```

```
[1] 3.14
```

```r
round(digits = 2, x = 3.14159)
```

```
[1] 3.14
```

All three lines return the same result. The first matches purely by position: `round()`'s first argument is `x`, its second is `digits`. The second and third name `digits` explicitly, so its position in the call no longer matters, which is why the third line can put `digits` before `x` without changing the answer.

The examples above simply run the function, so the output is shown in the console. If you're going to reuse the output later, you'll have to save it, the same way as the assignment above:

```
x <- round(3.14159, 2)
x * 2
```

```
[1] 6.28
```

Some arguments have a **default**: a value the function uses automatically if you don't supply one.

```
readings <- c(10, 12, NA, 15)
mean(readings)
```

```
[1] NA
```

```
mean(readings, na.rm = TRUE)
```

```
[1] 12.33333
```

`readings` has one missing value, `NA`, out of four. `mean()`'s `na.rm` argument defaults to `FALSE`, so by default a single missing value makes the whole result `NA`, exactly as the first line shows. Setting `na.rm = TRUE` tells `mean()` to drop that one missing value first, then average the three that remain. We'll meet `na.rm` constantly from here on, and it always means exactly this: drop the missing values before calculating, and be clear about how many were dropped.

1.9 Comparison and logical operators

These build a condition that's either `TRUE` or `FALSE` for each value in a vector, which is what every `filter()` from IES Week 7 onward runs on:

```
ages > 50
```

```
[1] FALSE  TRUE FALSE  TRUE
```

```r
ages == 45
```

```
[1]  TRUE FALSE FALSE FALSE
```

```r
ages != 45
```

```
[1] FALSE  TRUE  TRUE  TRUE
```

`!` negates a condition, `&` combines two conditions and requires both to be true, and `%in%` tests whether each value appears anywhere in a set:

```r
!(ages > 50)
```

```
[1]  TRUE FALSE  TRUE FALSE
```

```r
ages > 50 & ages < 70
```

```
[1] FALSE  TRUE FALSE FALSE
```

```r
ages %in% c(45, 71)
```

```
[1]  TRUE FALSE FALSE  TRUE
```

1.10 Packages

Everything used so far, `c()`, `class()`, `round()`, `mean()`, comes built into R itself. Most of what this course does lives in add-on packages instead: `dplyr` for manipulating data, `ggplot2` for plotting, and others still to come. Getting a package ready to use is two separate actions. **Installing** downloads it onto this computer, once. **Loading**, with `library()`, makes its functions available in the current R session, and has to be done again every time you start a fresh session, even though you never need to install the same package twice.

Installing is typically a one-off, run in the console rather than saved in a script: `install.packages("dplyr")`. Loading is what actually goes in a script, since every script that uses `dplyr` needs it loaded first:

```r
library(dplyr)
```

Loading a package sometimes prints a short message, often about functions it's replacing from another package. It's suppressed here since it doesn't matter yet, but don't be alarmed if you see one later in the course. It's routine, not an error.

1.11 `$` versus `select()`

`$` pulls a single column out of a data frame, as a plain vector:

```r
toy_data$smoker
```

```
[1] "No"  "Yes" "No"  "Yes"
```

`select()`, from `dplyr`, keeps a column too, but keeps it inside a data frame:

```r
select(toy_data, smoker)
```

```
  smoker
1     No
2    Yes
3     No
4    Yes
```

The values are the same either way. The difference is the container. `toy_data$smoker` is four bare values with nothing else attached. `select(toy_data, smoker)` is still a data frame, one column wide, four rows long. This matters practically from IES Week 8 onward: some functions, like `t.test()`, want a bare vector and expect `$`. Most of `dplyr`, starting in IES Week 7, works on data frames throughout and expects `select()`.

1.12 Quoting rules and backticks

Text values need quotes. Numbers, `TRUE`/`FALSE`, and the names of objects or columns don't:

```r
class(45)
```

```
[1] "numeric"
```

```
class("45")
```

```
[1] "character"
```

`45` is a number. `"45"` is text that happens to look like a number, and `class()` treats them completely differently. The same logic applies to object and column names: writing `ages` refers to the vector we created earlier, but writing `"ages"` is just the four-letter word.

A column name containing a space needs a different kind of quoting mark entirely: backticks, not quotation marks.

```
toy <- data.frame(`patient id` = c(1, 2, 3), check.names = FALSE)
toy$`patient id`
```

```
[1] 1 2 3
```

Backticks aren't easy to work with, and generally it's not recommended to have variable names containing a space. `data.frame()` normally cleans up column names automatically, turning a space into a dot. `check.names = FALSE` switches that off, purely so this example can show a genuine spaced name. When we read a CSV file with `read_csv()` in IES Week 5, column names come in exactly as written, spaces and all, with no automatic cleanup. Its dictionary file has a column literally called `Variable names`, and backticks are exactly how we'll refer to it.

1.13 The pipe

`|>` takes what's on its left and feeds it in as the first input to what's on its right. Read it aloud as "and then":

```
c(16, 25, 36) |> sqrt() |> sum()
```

```
[1] 15
```

Take these three numbers, and then take the square root of each, and then add up what's left. Written without the pipe, the same calculation reads inside-out: `sum(sqrt(c(16, 25, 36)))`. The pipe exists so a sequence of steps can be read in the order they actually happen, which is how every multi-step `dplyr` sequence in this book, starting in IES Week 7, is written.

There is a keyboard shortcut for the pipe: **Ctrl+Shift+M** (Windows) or **Cmd+Shift+M** (Mac). You'll likely write a lot of pipes, so it's worth remembering.

It's also worth knowing that `%>%` is the older pipe, and is only available once you've loaded the `dplyr` package. Practically, both pipes can be treated the same.

1.14 Reading errors

You will encounter a lot of errors when you work with R, and that's normal. Even a very experienced R user will encounter errors. The skill worth building now is reading the message rather than panicking at it. R, by way of the `rlang` package installed alongside the tidyverse, formats errors as a short heading followed by a line starting with `!` stating what went wrong. Five kinds of failure account for nearly everything you'll see in this course.

Object not found. R can't find something by that name, usually a typo or a variable that was never created:

```
mean(ags2)
```

```
Error:
! object 'ags2' not found
```

```
mean(ages)
```

```
[1] 54
```

Could not find function. Same idea, but for a function name. R is case-sensitive throughout, so `Mean()` is not the same thing as `mean()`:

```
Mean(ages)
```

```
Error in `Mean()`:
! could not find function "Mean"
```

```
mean(ages)
```

```
[1] 54
```

Unexpected symbol. R's interpreter (the machine that reads your code and executes it) found something it couldn't make sense of, usually a missing operator or a missing comma between two things that were meant to be one call:

```
c(1, 2) c(3, 4)
```

```
Error in parse(text = input): <text>:1:9: unexpected symbol
1: c(1, 2) c
            ^
```

```r
c(1, 2, 3, 4)
```

```
[1] 1 2 3 4
```

Unexpected end of input. A bracket was opened and never closed, so R keeps waiting for the rest of the expression that never arrives. In the console this shows up as a `+` prompt instead of the usual `>`, waiting for you to finish the line; here, where the code has to run to completion on its own, it shows as an error instead:

```r
sum(c(1, 2, 3)
```

```
Error in parse(text = input): <text>:2:0: unexpected end of input
1: sum(c(1, 2, 3)
   ^
```

```r
sum(c(1, 2, 3))
```

```
[1] 6
```

File does not exist. R can't find a file at the path given, usually a typo in the file name or a path built from the wrong folder:

```r
read.csv("nonexistent_file.csv")
```

```
Warning in file(file, "rt"): cannot open file 'nonexistent_file.csv': No
such file or directory
```

```
Error in `file()`:
! cannot open the connection
```

The warning line above states the actual reason, a missing file, before the error itself. We'll meet this one properly in IES Week 5, once there's a real file to read. `read.csv()` is base R's own version of the function; from there onward we use `read_csv()` from the `readr` package instead, which behaves a little differently but fails the same way when a path is wrong.

1.15 Getting help

Every function used in this course has a help page. Typing `?mean` in the console and pressing Enter opens `mean()`'s help page in the Help pane, bottom-right, showing exactly what arguments it takes and what it returns. It's usually faster than searching online, and it's always correct for the version of R actually installed. RStudio also keeps a set of cheat sheets under **Help → Cheatsheets**, one per major package. Beyond that, this book itself stays the main reference for the rest of the course. IES Week 5 explains how to find your way around it.

1.16 Worked example: four clinic visits

Let's put several of today's pieces together on one small, made-up example: four clinic visits, invented purely to practise on. From IES Week 7 onward, everything in this book runs on real-world data (`nhanes_mph` or `strep_tb`) instead; today's numbers exist only to give the syntax something to run on before either dataset has been introduced.

```
visit_id <- c(101, 102, 103, 104)
patient_age <- c(45, 62, 38, 71)
smoker <- c("No", "Yes", "No", "Yes")
weight_kg <- c(81.456, 50.204, 68.933, 73.118)

visits <- data.frame(visit_id, patient_age, smoker, weight_kg)
visits
```

```
  visit_id patient_age smoker weight_kg
1      101          45     No    81.456
2      102          62    Yes    50.204
3      103          38     No    68.933
4      104          71    Yes    73.118
```

We can check what type each piece is:

```
class(visits)
```

```
[1] "data.frame"
```

```
class(visits$patient_age)
```

```
[1] "numeric"
```

```r
class(visits$smoker)
```

```
[1] "character"
```

Pull out one column as a bare vector, and build a logical condition on it:

```r
visits$patient_age
```

```
[1] 45 62 38 71
```

```r
visits$patient_age > 50 & visits$smoker == "Yes"
```

```
[1] FALSE  TRUE FALSE  TRUE
```

Two of the four visits are for a smoker over 50. Compare `$` against `select()` on the same data frame:

```r
visits$smoker
```

```
[1] "No"  "Yes" "No"  "Yes"
```

```r
select(visits, smoker)
```

```
  smoker
1     No
2    Yes
3     No
4    Yes
```

Same values, different container: a bare vector from `$`, a one-column data frame from `select()`. Every idea in this worked example, a vector, a data frame, `$`, a comparison, loading a package, the pipe, comes back constantly from IES Week 7 onward. Only the data changes.

1.17 Exercises

Exercise 1 (ILO 3). Create a vector called `height_cm` holding the values of 177, 155, 168, 181, using `c()` and `<-`. Check its class with `class()`, and confirm `height_cm` appears in the Environment pane after you run your code.

Exercise 2 (ILOs 4, 5). Using `visits$weight_kg` from the worked example, round the values to one decimal place, naming the `digits` argument explicitly rather than relying on position. Then build a logical condition that identifies which visits are for a smoker weighing over 70 kg, combining a comparison operator with `&`.

Then open `round()`'s help page by typing `?round` in the console. Find the **Usage** section at the top, and write down what value `digits` takes when you don't supply one. Check your answer by running `round()` on the same values with the `digits` argument left out entirely.

Exercise 3 (ILOs 4, 5). Update the data frame `visits` to include the new `height_cm` variable. Calculate the BMI (weight in kg divided by height in metres, squared) of all the patients by using the `$` sign.

Exercise 4 (ILO 5). Write a piped sequence, using `|>`, that takes `visits$weight_kg`, rounds it to zero decimal places, and sums the result. Then use `select()` to keep just `visit_id` and `smoker` from `visits`, and write one sentence stating how the result differs from `visits$smoker`.

1.18 Self-check

A short multiple-choice check, not a writing exercise. There's no statistical result yet to interpret in a sentence. That starts in IES Week 7. Instead, these five questions target the places students most often get tripped up in R's syntax. Answers are in the Solutions callout below.

1. In `mean_x <- mean(x, na.rm = TRUE)`, what's the difference between `<-` and `=`?

a) They mean exactly the same thing
b) `<-` creates or updates an object; `=` here only names an argument inside the function call
c) `<-` only works inside a function call, `=` only works outside one
d) `=` creates an object; `<-` only names an argument

2. `visits$smoker` and `select(visits, smoker)` return the same four values. What actually differs between them?

a) Nothing, they always return identical output
b) `$` only works on vectors, never on data frames
c) `$` returns a bare vector; `select()` returns a data frame with one column

d) `select()` is base R; `$` is from `dplyr`

3. In `round(x = 3.14159, digits = 2)`, what happens if you write `round(digits = 2, x = 3.14159)` instead?

a) Nothing changes: named arguments can be given in any order
b) R throws an error, because the arguments are in the wrong order
c) R silently ignores `digits` and uses its default instead
d) It only works if `x` is written first, always

4. Why does R separate `install.packages()` from `library()`?

a) They do the same thing; `install.packages()` is just the older name for it
b) `install.packages()` downloads a package once; `library()` loads it into the current session, and has to be run again every time R restarts
c) `library()` downloads the package; `install.packages()` only loads it
d) Both have to be run every single time you use a function from a package

5. Running `Mean(ages)` produces `Error in Mean(ages) : could not find function "Mean"`. What's actually wrong?

a) The object `ages` doesn't exist
b) R is case-sensitive and there's no function called `Mean`; the function is `mean()`
c) A package needs to be installed first
d) There's a missing comma somewhere in the call

Solutions

Exercise 1

```r
height_cm <- c(177, 155, 168, 181)
height_cm
```

```
[1] 177 155 168 181
```

```r
class(height_cm)
```

```
[1] "numeric"
```

Exercise 2

```r
round(visits$weight_kg, digits = 1)
```

```
[1] 81.5 50.2 68.9 73.1
```

```
visits$smoker == "Yes" & visits$weight_kg > 70
```

```
[1] FALSE FALSE FALSE  TRUE
```

`?round`'s Usage section reads `round(x, digits = 0)`, so `digits` defaults to 0: leave it out and R rounds to the nearest whole number. Running it both ways confirms that:

```
round(visits$weight_kg)
```

```
[1] 81 50 69 73
```

```
round(visits$weight_kg, digits = 0)
```

```
[1] 81 50 69 73
```

The two lines give identical results, which is what a default argument means in practice. The help page is the fastest way to find out what a function does when you don't tell it, and it's always right for the version of R you actually have installed.

Exercise 3

```
visits <- data.frame(visits, height_cm)
bmi <- visits$weight_kg / (visits$height_cm / 100)^2
bmi
```

```
[1] 26.00019 20.89657 24.42354 22.31861
```

Exercise 4

```
visits$weight_kg |> round(digits = 0) |> sum()
```

```
[1] 273
```

```
select(visits, visit_id, smoker)
```

```
  visit_id smoker
1      101     No
2      102    Yes
3      103     No
4      104    Yes
```

`select(visits, visit_id, smoker)` returns a data frame with two columns and four rows. `visits$smoker` returns just the smoker column, on its own, as a bare vector with no `visit_id` alongside it and no data frame structure around it.

Self-check answers

1. **b.** `<-` creates or updates an object. `=` inside a function call only names which argument a value belongs to; it never creates an object there.
2. **c.** Same values, different container: `$` strips the data frame away and leaves a bare vector, `select()` keeps the data frame structure.
3. **a.** Named arguments are matched by name, not position, so their order in the call makes no difference to the result.
4. **b.** Installing is a one-off download. Loading has to happen at the start of every R session, since a fresh session starts with no packages loaded, even ones already installed.
5. **b.** R is case-sensitive throughout. `Mean` and `mean` are different names as far as R is concerned, and only one of them is a real function.

2 Data and data handling (IES Week 5)

This session follows IES Week 4, where we learned the language itself: objects, functions, packages, and how to read an error message. Today we bring in real data for the first time. We'll take a spreadsheet apart to see what makes it usable, import it into R, look it over, spot what's missing, and produce one small piece of real analysis, start to finish. By the end you will have written and saved your own R script, and re-run it from a fresh session to prove it still works. That is the way you'll work for the rest of the course. IES Week 6 is reading week, so there's no practical for it, and IES Week 7 picks up from here and goes straight into using `nhanes_mph` properly.

2.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Identify features of a spreadsheet that will prevent it being analysed, and restructure it
2. Import a CSV into R using a correctly specified path, and diagnose and resolve the most common import failures
3. Inspect an imported dataset's dimensions, variable names, types, and missing values
4. Write, save, and re-run an R script that imports data, summarises it, and produces a plot
5. Locate a downloaded file on your own operating system, construct or copy a valid path to it, and explain what a CSV file physically contains

2.2 Setup

There's nothing to read in yet, so, as in IES Week 4, this chunk only sets the console's print width.

```
options(width = 76)
```

2.3 Tidy data, in a spreadsheet

2.3.1 What makes data usable

A dataset R can work with follows three simple rules: one variable per column, one observation per row, one value per cell. Data laid out this way is called **tidy**. It sounds obvious written down, but very few spreadsheets people actually hand you follow it, because a spreadsheet is usually built to be *read by a person*, not imported by software. People merge cells to make a heading look nice. They use colour instead of writing a word down. They leave blank rows for visual spacing. None of that survives import, and some of it actively breaks it.

2.3.2 What breaks an import

- **Merged cells.** A heading merged across several columns looks fine to a person, but when R reads the underlying columns back out, most of them come back empty, because the merge only stored the value in one of the cells it covers.
- **Colour used to carry information**, instead of a word or a number. A whole row filled in, say, yellow to mean "smoker", with no column recording it at all, and the only place that tells you what the colour means is a caption sitting several rows away, disconnected from the table itself. R reads cell values, not cell fills, so the colour disappears completely on import, and the caption comes in as an isolated row of text with nothing left to connect it back to the patients it was describing.
- **Units mixed into the value, and inconsistent with each other.** Most of a Weight or Height column might be plain numbers, `72`, `1.70`, and so on, but one row breaks the pattern: `195 lbs` and `6 ft` instead of kilograms and metres, or a height like `178` sitting uncommented among a column of metres, really centimetres in disguise. R reads a whole column as text the moment a single cell in it isn't a bare number, so it only takes one row like this to turn an otherwise clean numeric column into text throughout.
- **Blank rows used for spacing.** A blank row inserted purely to separate two sections becomes a row of missing values, `NA`, in every column, once imported.
- **Notes written in a stray cell**, off to the side of the actual table, with no column of their own. This can be a note about one specific patient, or, as with the colour above, a caption meant to explain something elsewhere in the sheet that never made it into a proper column.
- **More than one table on a single sheet.** R expects a sheet to be one table. A second, unrelated table lower down the same sheet gets read as more rows of the first table, and the result makes no sense.

Spreadsheet exercise (ILO 1). `data/05-prep2-messy-clinic.xlsx` is a small, made-up clinic visit log, built specifically to have every one of these problems at once. Open it in Excel, or whatever spreadsheet software you have.

Screenshot placeholder (`images/05-prep2-messy-spreadsheet.png`): `05-prep2-messy-clinic.xlsx` open in a spreadsheet application, showing the merged title row, the merged sub-header over the Weight and Height columns, the yellow-highlighted rows with no `Smoker` column anywhere in sight, and the second table below the gap.

Go through the sheet and list every feature above that you can find in it. Then build a clean version: one header row naming every column, one row per patient, weight and height as plain numbers in consistent units with the units moved into the column name instead of the value, a proper smoker column holding an actual word rather than a colour and a caption, no stray notes, and the second table (the follow-up schedule) either removed or saved as a completely separate file. Save your clean version as a `.csv` file. The Solutions section describes the fixes, but building the file yourself, in spreadsheet software, is the actual point of this exercise. There's nothing to submit for it.

2.4 Finding a file, and telling R where it is

Before we can read anything into R, we need to know exactly where the file we want actually lives. This is the file system skill everything from here on assumes.

Where downloads go. When you download a file from a browser, most browsers save it into a folder called `Downloads` by default, without asking. On Windows that's usually `C:\Users\yourname\Downloads`; on a Mac it's usually `/Users/yourname/Downloads`. If you can't find a file you just downloaded, that folder is the first place to check, and it's worth confirming where your own browser is set to save to, since some let you change it.

Reading a path. A file path is a set of directions from somewhere fixed down to one specific file, one folder at a time, separated by slashes. `data/mph_nhanes_20241107.csv` reads as "the folder called `data`, then inside it the file called `mph_nhanes_20241107.csv`." A longer path just has more folders to walk through before reaching the file.

Copying a path. You'll rarely type a full path out by hand; instead, you copy it straight from your file browser.

- **Windows:** hold Shift and right-click the file, then choose **Copy as path**.
- **macOS:** right-click the file, then hold the **Option** key, which changes the menu, and choose **Copy "filename" as Pathname**.

Backslashes versus forward slashes. A path copied this way on Windows uses backslashes, `\`, for example `C:\Users\yourname\Documents\ies_mph\data\file.csv`. R uses forward slashes, `/`, instead, and treats a lone backslash as the start of a special character rather than a folder separator, so a pasted Windows path usually needs every `\` replaced with `/` before R will accept it.

What a CSV actually is. Open a `.csv` file in a plain text editor, Notepad on Windows or TextEdit on a Mac, rather than in spreadsheet software, and you'll see plain text: one line per row, values separated by commas. There's no hidden formatting behind it, which is exactly why it's such a safe format to hand between different pieces of software, R included.

2.5 Getting data in

2.5.1 `read_csv()` and `here()`

`readr`'s `read_csv()` reads a `.csv` file into R as a data frame. It needs a path telling it exactly where that file is, using the path skills from earlier in this chapter. Building that path with `here()`, from the `here` package, means the same code works on any machine, regardless of where the project folder actually sits on that machine's disk: `here()` starts from the project folder and adds the pieces you give it, one folder at a time.

This chapter needs three packages: `readr` for reading files, `here` for building paths, and `ggplot2` for the plot at the end. As IES Week 4 explained, installing a package and loading it are two different actions, so unless you've installed these already, run this once in the console, not inside this document:

```
install.packages(c("readr", "here", "ggplot2"))
```

After that, `library()` is all you need, in this session and every session after it:

```
library(readr)
library(here)
```

`library(here)` prints one line when it loads, which this book hides because it's too wide to fit the page. In your own console it says `here() starts at ...` followed by the full path to your project folder. That's worth reading the first time you see it, because it's `here()` telling you exactly which folder it will build every path from. If that path isn't the project folder you built in IES Week 4, nothing below will find its files, and this is where you'd find that out.

2.5.2 Four things that go wrong

Almost every import problem you'll hit is one of four things. Each one below is deliberately broken, then fixed.

A wrong filename. Close, but not exact:

```r
read_csv(here("data", "mph_nhanes_20241107_data.csv"))
```

```
Error:
! '/projects/ies_stats_ebook/data/mph_nhanes_20241107_data.csv' does
  not exist.
```

The real file is called `mph_nhanes_20241107.csv`, no `_data` in the name. R's message states plainly that the file doesn't exist at the path it tried, which is exactly what IES Week 4's "file does not exist" error type looks like with a real file behind it.

```r
read_csv(here("data", "mph_nhanes_20241107.csv"))
```

A path pasted straight from Windows Explorer. Shift+right-click → *Copy as path*, from earlier in this chapter, gives you a full path using backslashes, for example `C:\Users\yourname\Documents\new_folder\mph_nhanes_20241107.csv`. Paste that directly into R as a string, without changing anything:

```r
read_csv("C:\Users\yourname\Documents\new_folder\mph_nhanes_20241107.csv")
```

```
Error: '\U' used without hex digits in character string (<input>:1:14)
```

This isn't even a "file not found" error. It never gets that far. `\U` is one of a handful of backslash sequences R's parser treats specially inside a string, and `\Users` isn't a valid one, so R refuses to parse the string at all. Every Windows path copied this way starts with `C:\Users\...`, so this isn't a rare edge case, it's the default shape of a pasted Windows path. The fix from earlier in this chapter still applies: replace every backslash with a forward slash before using a path in R. Better still, once the path is inside the project folder, `here()` avoids the problem entirely, since you never type the backslash-filled part of the path at all.

The right file, wrong location. Forgetting which subfolder the file lives in:

```r
read_csv(here("mph_nhanes_20241107.csv"))
```

```
Error:
! '/projects/ies_stats_ebook/mph_nhanes_20241107.csv' does not
  exist.
```

The filename is exactly right this time. What's missing is `"data"`, the subfolder it actually lives in. R's error message looks identical to the wrong-filename case above, which is worth noticing: the message tells you *that* nothing was found at that path, not *why* your path was wrong. You have to check that yourself.

```r
read_csv(here("data", "mph_nhanes_20241107.csv"))
```

The wrong file type. `data/05-prep2-messy-clinic.xlsx`, from the exercise above, is an Excel file, not a CSV. Reading it with `read_csv()` doesn't throw an error at all.

To see what actually came in, we need three functions. `dim()` reports how many rows and columns a dataset has, and `names()` lists its column names. Both get a proper introduction in the next section, where we use them on real data; here they're just the quickest way to look. `substr()`, from base R, takes a piece of text and keeps only part of it, given a starting and an ending position: `substr(x, 1, 50)` keeps the first 50 characters and throws the rest away. We need it because the column name that comes back is far too long to print in full.

There's one extra argument in the call below, `show_col_types = FALSE`, and it's worth saying what it switches off rather than letting it slide past. `read_csv()` normally prints a short report of what it found and what type it guessed for each column. We meet that report properly in a moment, on the real data file, where it's useful. Here it would just be several lines of nonsense about a file that hasn't imported, so we turn it off to keep the broken result below the only thing on screen. That's the only reason it's there.

```r
messy_attempt <- read_csv(here("data", "05-prep2-messy-clinic.xlsx"),
  show_col_types = FALSE)
```

```
Multiple files in zip: reading '[Content_Types].xml'
```

```r
dim(messy_attempt)
```

```
[1] 1 1
```

```r
substr(names(messy_attempt), 1, 50)
```

```
[1] "<?xml version=\"1.0\" encoding=\"UTF-8\" standalone=\"y"
```

No error, but nothing usable either: one column, zero rows, and a column name that's a fragment of raw XML. An `.xlsx` file is actually a small zip archive of several XML files bundled together, and `read_csv()` has gone looking inside that archive and started reading one of those

XML files as if it were plain text. This is the quietest kind of import failure there is: nothing crashes, nothing complains, and the result still looks like a data frame at a glance. Always check what actually came in, which is exactly what the next section is for.

Put right, all four attempts above collapse into the one call we actually want, and this is the real import we use for the rest of this book. The file holds `nhanes_mph`, an extract of a large American health survey in which adults were interviewed and then given a physical examination, so it carries measured values like cholesterol alongside things people reported about themselves. IES Week 7 introduces it properly, including one important simplification we make about it throughout. For today we only need it to be a real file with real data in it:

```
nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))
```

2.5.3 The block of output you'll see

Run that line yourself and `read_csv()` prints eight lines the moment it finishes. This book hides them, because one of them is too wide to fit the page, so here they are as they'll appear in your console:

```
Rows: 41765 Columns: 27
  Column specification
Delimiter: ","
chr  (3): smoking, phq.c, lcod
dbl (24): SEQN, age, gender, eth, edu, alc_wk, vpa_w, ...

 Use `spec()` to retrieve the full column specification
  for this data.
 Specify the column types or set `show_col_types = FALSE`
  to quiet this message.
```

It looks like something went wrong. Nothing did. This is a **message**, not a warning and not an error, and it appears every single time `read_csv()` reads a file successfully. It's worth reading once, properly, because it's the first thing you know about a dataset you've never opened.

`Rows: 41765 Columns: 27` is the size of what came in. That's the same fact `dim()` gives us below, arriving before we ask for it. If a file you expected to have a few hundred rows reports three, you've learned that here rather than ten minutes later.

`Delimiter: ","` is `read_csv()` confirming it split each line at commas, which is what the "comma" in "comma separated values" means, and what we saw for ourselves earlier in this chapter when we opened a CSV in a text editor.

Then the column specification: `chr (3)` lists three columns read as character, that is text, and `dbl (24)` lists twenty-four read as numbers. `dbl` is short for "double", which is R's ordinary way of storing a number that can have decimal places. This is `read_csv()` telling us what it *guessed*, one column at a time, and that guess is the thing worth checking. Notice `smoking` and `phq.c` sitting in the character list, and `gender` and `eth` sitting in the number list, and hold onto that: it comes back in the next section but one.

The two lines beginning with an   are `read_csv()` offering you a way to see the full specification, and a way to make the whole message go away. That second one is `show_col_types = FALSE`, the argument we used on the broken `.xlsx` import above. Now you know what it silences. Use it when you already know what's in a file and don't want the report again, not to make untidy output go away, because "untidy output" and "the only warning you were going to get" often look identical.

2.5.4 A first look

The message already told us the size and the column types. Four functions let us check that against what we actually have.

```r
dim(nhanes)
```

```
[1] 41765    27
```

```r
names(nhanes)
```

```
 [1] "SEQN"     "age"      "gender"   "eth"      "edu"      "smoking"
 [7] "alc_wk"   "vpa_w"    "mpa_w"    "wal_r"    "vpa_r"    "mpa_r"
[13] "mpa_t"    "vpa_t"    "met_w"    "met_r"    "met_t"    "sed"
[19] "chol"     "phq"      "phq.c"    "blm.cvd"  "dth"      "lcod"
[25] "ucod_dm"  "ucod_ht"  "fu_month"
```

`dim()` gives rows, then columns. `names()` lists every column name, in order, useful for checking a name before you type it.

```r
head(nhanes)
```

```
# A tibble: 6 x 27
   SEQN   age gender   eth   edu smoking  alc_wk vpa_w mpa_w wal_r vpa_r
  <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>     <dbl> <dbl> <dbl> <dbl> <dbl>
1    12    37      1     3     3 Never     0.460    NA    NA    NA    NA
```

```
2      20     23        2      1      1 Previous  3         NA     NA     NA     NA
3      34     38        2      4      2 Current   1         NA     NA     NA     NA
4      66     37        1      1      3 Never    NA         NA     NA     NA     NA
5      69     27        1      4      3 Never     2         NA     NA     NA     NA
6      81     30        1      3      3 Never     1         NA     NA     NA     NA
# i 16 more variables: mpa_r <dbl>, mpa_t <dbl>, vpa_t <dbl>, met_w <dbl>,
#   met_r <dbl>, met_t <dbl>, sed <dbl>, chol <dbl>, phq <dbl>,
#   phq.c <chr>, blm.cvd <dbl>, dth <dbl>, lcod <chr>, ucod_dm <dbl>,
#   ucod_ht <dbl>, fu_month <dbl>
```

`head()` prints the first six rows, so you can eyeball what real values actually look like.

```
summary(nhanes$age)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
  18.00   31.00   46.00   47.52   63.00   85.00 
```

`summary()`, run on one column, gives its minimum, maximum, quartiles, and mean in one line, plus a count of missing values if there are any. Run on the whole data frame instead of one column, it repeats this for every column at once, which is a fast way to eyeball the whole dataset in one go.

Look at `gender` and `smoking` in the output above. `gender` is a plain number, 1 or 2, standing in for a category. `smoking` is already text, "Never", "Previous", or "Current". Neither is a proper R factor yet, a category R actually understands as one. IES Week 7 turns both, and four more columns like them, into real factors, checked against the dictionary file. For now we're only looking, not changing anything.

2.5.5 When R guesses the wrong type

`read_csv()` decides each column's type by looking at the values in it. Most of the time that guess is right. It goes wrong in exactly the situation the messy spreadsheet exercise built on purpose: a column that's plain numbers for every patient except one, who was recorded as `"195 lbs"` instead of a number in kilograms. R has no way to know that `"lbs"` should be split off, converted, and the rest treated as a number. It just sees text and stores the whole column as text, one stray row is all it takes. This is why the clean version of that spreadsheet needed every patient converted to the same unit, with that unit moved into the column *name* (`weight_kg`) and out of every individual *value*: it's the only way `read_csv()` ends up with a column it can actually calculate on.

2.5.6 Spotting missing data

`is.na()` checks each value in a vector and returns `TRUE` where it's missing, `FALSE` where it isn't. `sum()` on that result counts the `TRUE`s, since R treats `TRUE` as 1 and `FALSE` as 0 when you add them up.

```
sum(is.na(nhanes$vpa_w))
```

```
[1] 7354
```

That's how many participants have no value recorded for `vpa_w`, vigorous physical activity at work, out of 41,765 participants in total. `nrow()`, from base R, returns just the row count on its own, the same number `dim()` gives as its first value above, useful whenever we want the total on its own rather than alongside the column count. It's a real gap, not a rounding artefact, and it matters: whatever we go on to calculate from `vpa_w` only ever describes the participants who actually have a value, and we should say so out loud whenever we report it, rather than letting it pass silently.

2.5.7 Using the dictionary file

`nhanes_mph` arrives as two separate files. We've already read the data itself. The second file names every column and states what it means, and it's every bit as important as the data. Reading it prints the same column specification message we just unpacked, saying two columns of text this time instead of twenty-seven columns of mixed types. This book hides it from here on, since it says the same kind of thing every time and we've now read one properly. You'll still see it in your own console, and it's still worth a glance:

```
dictionary <- read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
dictionary
```

```
# A tibble: 27 x 2
   `Variable names` Description
   <chr>            <chr>
 1 SEQN             Participant identifier
 2 age              Age in years
 3 gender           Gender (1=Male; 2=Female)
 4 eth              Ethnicity (1=Mexican; 2=Other Hispanic; 3=White; 4=Blac~
 5 edu              Educaton (1=Less than high school; 2=high school gradua~
 6 smoking          Smoking status (Never/Previous/Current)
 7 alc_wk           Alcohol drinking per week
```

```
 8 vpa_w              Vigourous physical activity (hour/day) at work
 9 mpa_w              Moderate physical activity (hour/day) at work
10 wal_r              Walking (hour/day) outside work
# i 17 more rows
```

The dictionary's first column is called `Variable names`, with a space in it, so referring to it needs backticks, exactly as IES Week 4 covered. `%in%`, from the same chapter, tests whether each value is a member of a set. Put inside square brackets after a data frame's column, with the row's index implied, this keeps only the rows where the condition is true:

```
dictionary[dictionary$`Variable names` %in% c("age", "chol", "smoking"), ]
```

```
# A tibble: 3 x 2
  `Variable names` Description
  <chr>            <chr>
1 age              Age in years
2 smoking          Smoking status (Never/Previous/Current)
3 chol             Total cholesterol (mmol/L)
```

That's the habit worth building now: when a column's meaning isn't obvious from its name, the answer lives in this file, not in a guess. IES Week 7 uses this same file to build proper factors out of six of these columns; for today, just knowing how to look one up is enough.

2.6 A first analysis, and your first script

2.6.1 One summary statistic

```
mean(nhanes$chol, na.rm = TRUE)
```

```
[1] 4.973316
```

`chol` is total cholesterol, in mmol/L, and 2,589 participants have no value recorded for it, the same kind of gap `vpa_w` had above. `na.rm = TRUE` drops those missing values before averaging the rest, exactly as it did back in IES Week 4's `mean()` example. Leave it out and the whole calculation returns `NA`, since R won't silently guess what a missing value should have been.

2.6.2 A first taste of a grouped summary

`nhanes$chol` pulls the whole cholesterol column out as a bare vector, exactly as `$` did in IES Week 4. Adding square brackets straight after it, with a condition inside, keeps only the values where that condition is `TRUE`:

```r
summary(nhanes$chol[nhanes$smoking == "Never"])
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs 
  1.780   4.220   4.890   4.968   5.610  15.830    1839 
```

```r
summary(nhanes$chol[nhanes$smoking == "Current"])
```

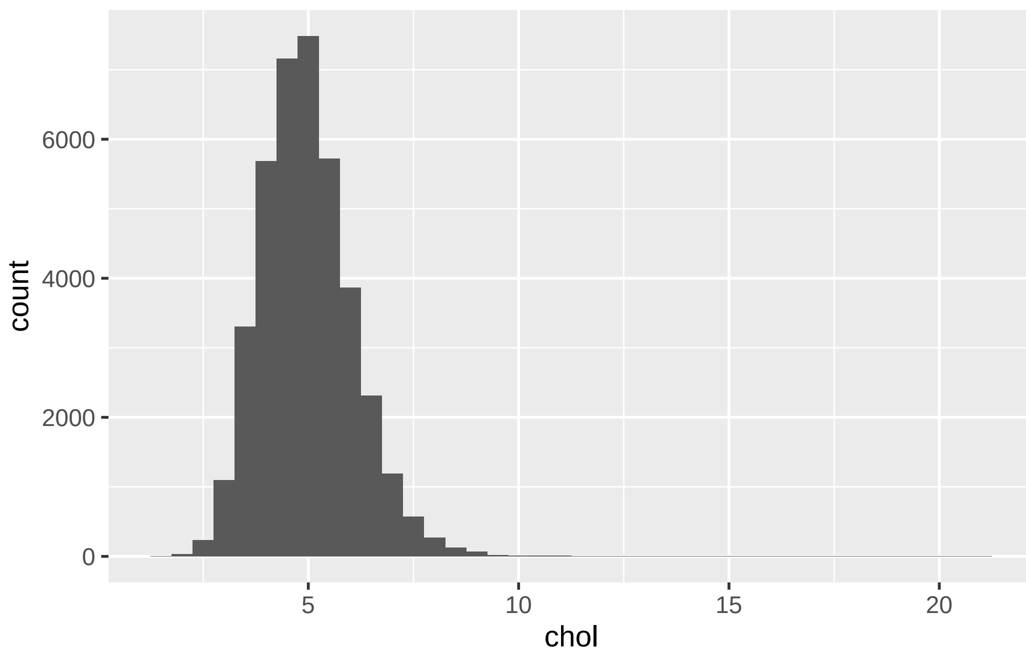
```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs 
  1.530   4.220   4.890   4.996   5.660  15.900     882 
```

That's a genuine grouped summary, built entirely from tools already familiar: `$`, `==`, and now `[` with a condition inside it. It works, but it's clunky already with just two groups, and `smoking` has three. IES Week 7 introduces `group_by()` and `summarise()`, a far nicer way to do exactly this for as many groups as you like, in one line instead of several. This is only a preview of what's coming, not the tool you'll actually use going forward.

2.6.3 A first taste of a plot

```r
library(ggplot2)

ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5)
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```



`ggplot2` code is joined with `+`, not the pipe. That's a real inconsistency in R, not a mistake in this book, and IES Week 7 explains why and unpacks the rest of the grammar properly. For today, just notice the shape: cholesterol piles up around 4 to 6 mmol/L, with a long tail of higher values trailing off to the right.

2.6.4 That warning above the plot

The plot came with a line starting `Warning:`, saying it removed 2,589 rows. Read it, because we'll see this one a lot, and then don't worry about it.

First, a warning is not an error. An error means R gave up and produced nothing. A warning means R did the job and is telling you something about how. The plot is there and it's fine.

Second, look at the number. It's 2,589, exactly the count of participants with no cholesterol value that we found a moment ago. That's not a coincidence, it's the same fact arriving twice. A histogram puts each participant somewhere along the axis, and a missing value has no "somewhere" to go, so `ggplot2` left those participants out and told us how many it dropped.

Why we leave these warnings switched on. R can be told to hide them, and plenty of people do, because they look untidy. This book doesn't, and neither should you for now. That warning is a free count of how much of your data a plot is actually based on. Hide it, and a plot built on half your dataset looks exactly like a plot built on all of it. It's the same reason we write `na.rm = TRUE` deliberately and say what it dropped, rather than typing it out of habit until it stops registering.

When it does need attention. The number is the thing to check, not the warning itself. Compare it against what you expect, the way we just did with `is.na()`. If it matches, carry on. If it's much larger than expected, stop and find out why before trusting anything the plot shows: usually it means the wrong column, an import that didn't go as planned, or an earlier step that threw away more rows than intended. And whenever you report a result, say how many observations it's based on and how many were missing. A warning R printed for you is not the same as a reader being told.

2.6.5 From console to script

Everything we've done so far in this chapter has been one line at a time. That's the right way to *try* something: type it in the console, look at what comes back, fix it if it's wrong. But IES Week 4 already made the argument against leaving it there. You will need to do this again, and you will not remember what you typed.

So the working habit for the rest of this course is two steps, not one. Try the line in the console until it does what you want. Then move the line that worked into a script, and save it. The console is where you experiment; the script is what you keep.

To build one now:

1. In RStudio, go to **File → New File → R Script**. A blank file opens in the source pane, top-left.
2. Type the lines you want to keep. For a first script, that's the `library()` calls, the `read_csv()` line, one summary, and one plot.
3. Save it into the `scripts/` folder you built in IES Week 4, with a name that follows its conventions: no spaces, no `&` or `#`, something that says what it does. `first_analysis.R` is fine. `Untitled1.R` is not.

Here's what a complete first script looks like, start to finish. Nothing in it is new; it's the pieces from earlier in this chapter, collected in the order they need to run:

```r
library(readr)
library(here)
library(ggplot2)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))

mean(nhanes$chol, na.rm = TRUE)
sum(is.na(nhanes$chol))

ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5)
```

Notice the shape of it. The `library()` calls come first, because nothing below them works until the packages are loaded. The `read_csv()` line comes next, because every line after it needs `nhanes` to exist. A script runs top to bottom, so the order isn't decoration, it's the thing that makes it work.

2.6.6 Proving it actually works

Here's the part people skip, and it's the part that matters. A script that runs on your machine right now, in a session where you've been typing for an hour, is not yet proof of anything. Half of what it needs might already be sitting in memory from something you typed and forgot.

So test it properly. In RStudio, go to **Session** → **Restart R**. That throws away every object and every loaded package, leaving you with an empty session, exactly like the one a colleague, a marker, or you-in-six-months would start from. Now run your script from the top.

If it runs clean, the script is genuinely self-contained. If it fails, the error tells you precisely what you'd forgotten to write down, and the fix belongs *in the script*, not typed into the console again. That restart is the reproducibility argument from IES Week 4, made concrete: not "scripts are good practice" but "here is how you find out whether yours actually is one."

2.6.7 How this book is put together

Every chapter follows the same shape: learning objectives at the top, then worked content with the code and its output both visible, then a full worked example, then numbered exercises mapped back to those objectives. Solutions sit in a collapsed section at the very end, so you can have a real go before you look.

The way to use it is to work along in a script of your own. Don't read the code, and don't copy and paste it either. Type it, run it, and see the same output appear in your console. When something breaks, that's not you falling behind, that's the part you're about to learn something from.

2.7 Exercises

Exercise 1 (ILO 2). This `read_csv()` call is supposed to read the dictionary file but doesn't work:

```
read_csv(here("data", "mph_nhanes_dictionary.csv"))
```

Work out what's wrong with the path and write the corrected call.

Exercise 2 (ILO 3). Check `mpa_w` (moderate physical activity at work): find its type with `class()`, and count how many values are missing with `is.na()` and `sum()`.

Exercise 3 (ILO 4). Produce one summary statistic and one plot for `wal_r` (walking, hours/day, outside work). Remember `na.rm = TRUE` if it turns out to have missing values.

Exercise 4 (ILO 4). Collect your answer to Exercise 3 into a proper script. Open a new R script, and write into it everything needed to go from a completely empty session to that summary and that plot: the `library()` calls, the `read_csv()` line, then the summary and the plot themselves. Save it into your `scripts/` folder with a sensible name.

Then test it the way this chapter described. Go to **Session → Restart R**, and run your script from the top. If it errors, read the message, work out what the fresh session was missing, and add that line to the script rather than typing it into the console. Repeat until it runs clean from a restart. That is the point of the exercise, not the summary itself.

2.8 Self-check

Five questions on the concepts this chapter leans on hardest: paths, file types, and reading a real import problem. Answers are in the Solutions section below.

1. `read_csv(here("data", "mhp_nhanes_20241107.csv"))` fails, even though `mph_nhanes_20241107.csv` really is in the `data` folder. Which of the four import failures is this?

a) The file is the wrong type
b) A path pasted with backslashes
c) A wrong filename, here a typo (`mhp` instead of `mph`)
d) The file is in the wrong folder

2. What's the actual difference between `dim(nhanes)` and `names(nhanes)`?

a) They return exactly the same information in two formats
b) `dim()` gives the number of rows and columns; `names()` lists every column name
c) `names()` gives row and column counts; `dim()` lists column names
d) `dim()` only works after `names()` has been run first

3. `sum(is.na(nhanes$vpa_w))` returns `7354`. What does that number actually mean?

a) The mean value of `vpa_w`
b) 7,354 participants have `vpa_w` recorded as exactly zero
c) 7,354 participants have no value recorded for `vpa_w`
d) There are 7,354 participants in the dataset in total

4. `read_csv()` is used to read both `mph_nhanes_20241107.csv` and `mph_nhanes_20241107_dictionary.csv`, in two separate calls. Why two calls, and not one?

a) `read_csv()` can only read one row at a time
b) They're two separate files, each holding a different table, so each needs its own call
c) The dictionary file is not really needed, but the call is included anyway
d) `read_csv()` automatically finds and reads both once you name one of them

5. Reading `05-prep2-messy-clinic.xlsx` with `read_csv()` produces a data frame with 1 column and 0 rows, and no error message. What does that tell you?

a) The file is empty
b) `read_csv()` refuses to read `.xlsx` files and stops immediately
c) `.xlsx` is the wrong file type for `read_csv()`, and the lack of an error is exactly why you always check what actually came in
d) The import worked correctly and the file genuinely has one column

2.9 Writing exercise

Using the mean and the histogram from Exercise 3, write a short paragraph, about 60 words, describing `wal_r` as you would in the results section of a report. State the mean and its units, say how many participants have no value recorded, and describe the shape of the distribution the histogram shows. This is the smallest version of a habit the rest of this book builds on: a number on its own, with no sentence around it, is not yet a result.

> **Solutions**
>
> **Spreadsheet exercise**
> The fixes needed, matching each problem named earlier: split the merged title row and the merged "Measurements" sub-header back into one header row naming every column individually; convert every patient's weight and height to the same units (kilograms and metres), then move those units out of the values and into the column names instead (`weight_kg`, `height_m`), leaving plain numbers behind; add a proper Smoker column holding an actual word ("Yes"/"No"), decoded from which rows were highlighted and the caption that explained what the highlighting meant; delete both blank spacer rows; delete or relocate the stray note about Patient 5's missed follow-up, since it belongs in its own column, or nowhere in this table at all; and either delete the second table (the follow-up schedule) or save it as its own, separate CSV file, since one sheet should hold one table.
> **Exercise 1**

```
read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
```

```
# A tibble: 27 x 2
   `Variable names` Description
   <chr>            <chr>
 1 SEQN             Participant identifier
 2 age              Age in years
 3 gender           Gender (1=Male; 2=Female)
 4 eth              Ethnicity (1=Mexican; 2=Other Hispanic; 3=White; 4=Blac~
 5 edu              Educaton (1=Less than high school; 2=high school gradua~
 6 smoking          Smoking status (Never/Previous/Current)
 7 alc_wk           Alcohol drinking per week
 8 vpa_w            Vigourous physical activity (hour/day) at work
 9 mpa_w            Moderate physical activity (hour/day) at work
10 wal_r            Walking (hour/day) outside work
# i 17 more rows
```

The folder and filename were both fine. The problem was the letters `mph` typed as `mhp`.

Exercise 2

```
class(nhanes$mpa_w)
```

```
[1] "numeric"
```

```
sum(is.na(nhanes$mpa_w))
```

```
[1] 7391
```

Exercise 3

```
mean(nhanes$wal_r, na.rm = TRUE)
```

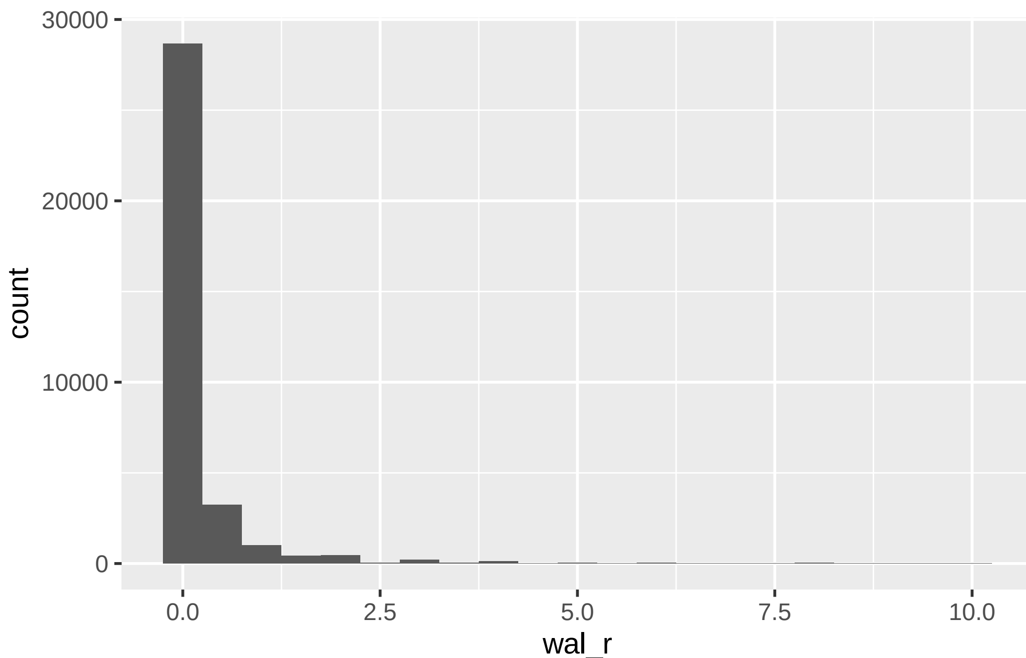
```
[1] 0.2060187
```

```
sum(is.na(nhanes$wal_r))
```

```
[1] 7347
```

```
library(ggplot2)
ggplot(nhanes, aes(x = wal_r)) +
  geom_histogram(binwidth = 0.5)
```

```
Warning: Removed 7347 rows containing non-finite outside the scale range
(`stat_bin()`).
```



Exercise 4
The finished script, in full. Saved as `scripts/wal_r_analysis.R`:

```r
library(readr)
library(here)
library(ggplot2)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))

mean(nhanes$wal_r, na.rm = TRUE)
sum(is.na(nhanes$wal_r))

ggplot(nhanes, aes(x = wal_r)) +
  geom_histogram(binwidth = 0.5)
```

The two lines people most often leave out are the ones that were already true before they started: `library(readr)` and the `read_csv()` call. Both had been run earlier in the session, so `nhanes` existed and `read_csv()` worked, and neither felt like part of "the analysis." After a restart, neither is true any more, and the script fails on its first real line with `could not find function "read_csv"` or `object 'nhanes' not found`, both of

which IES Week 4 covered. That's exactly what the restart is for: it finds the assumptions you didn't know you were making.
Self-check answers

1. **c.** `mhp` versus `mph` is a plain typo in the filename, not a folder problem, a file-type problem, or a backslash problem.
2. **b.** `dim()` returns row and column counts. `names()` returns the column names themselves. Neither substitutes for the other.
3. **c.** `is.na()` marks each missing value `TRUE`, and `sum()` adds those up. The number is a count of participants with no recorded value, nothing about the values that are present.
4. **b.** `mph_nhanes_20241107.csv` and `mph_nhanes_20241107_dictionary.csv` are two separate files on disk, each holding a different table, so each needs its own `read_csv()` call.
5. **c.** `.xlsx` is a zip archive of XML files, not a CSV, so `read_csv()` produces nonsense rather than a real dataset. The absence of an error is exactly the trap: always check what actually came in after reading a file, not just whether it "worked." `dim()`, used earlier in this chapter, is one way to do that. IES Week 7 introduces a second, `glimpse()` from `dplyr`, which shows every column with its type and first few values in one compact layout, better suited to a wide dataset like `nhanes_mph` than `dim()` alone.

Writing exercise (sample answer)
Participants reported a mean of 0.21 hours/day of walking outside work (`wal_r`), based on the 34,418 participants with a value recorded; 7,347 had none. The distribution is heavily right-skewed: most participants reported no walking outside work at all, and a small number reported several hours a day, pulling the mean above what a typical participant's value actually looks like.

3 Describing and visualising data (IES Week 7)

This practical follows the IES Week 7 seminar of the same name. That seminar covered variable types, tables and plots for describing a single variable, and how to choose between mean and SD or median and IQR. In IES Weeks 4 and 5 we already covered installing R, finding our way around a project folder, and importing a spreadsheet into R. So in this session we go straight to using that data.

3.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Use `select()`, `filter()`, `mutate()`, and `arrange()` to manipulate a dataset, and produce grouped summaries with `group_by()` and `summarise()`
2. Construct frequency and cross-tabulations
3. Build histograms, boxplots, bar charts, and scatterplots in `ggplot2`
4. Modify a plot's bin width, scale, and labels, and describe the effect on interpretation
5. Convert a character variable to a factor and set its reference level
6. Produce a basic "Table 1" of participant characteristics, and describe it in writing

3.2 The data: `nhanes_mph`

Before we load anything, it's worth knowing what we're loading. We use this dataset in every chapter from here to the end of the book, so the ten minutes spent here pays off five times over.

NHANES is the National Health and Nutrition Examination Survey, run by the National Center for Health Statistics, part of the US Centers for Disease Control and Prevention. It's designed to describe the health of the American population, and it's unusual in that it doesn't just ask people questions. Participants are interviewed at home and then examined in a mobile clinic, where measurements are taken and blood samples drawn. That's why our data can carry a real measured cholesterol value rather than "has a doctor ever told you your cholesterol is high?".

nhanes_mph is a curated extract of it, prepared for this course: 41,765 adults aged 18 to 85, described by 27 variables. Those cover demographics, education, smoking and alcohol, physical activity split by domain, sedentary time, total cholesterol, depression symptoms, and whether the participant died during follow-up. The full list is in the dictionary file, which we read in alongside the data below and use throughout.

3.2.1 One simplification, stated out loud

NHANES is not a simple random sample of Americans. It's a **complex survey**. Some groups are deliberately sampled more heavily than others, participants are recruited in geographic clusters rather than one at a time, and every participant carries a survey weight saying how many people in the population they stand for. Analysed properly, all of that has to go into the calculation.

This course ignores it. We treat nhanes_mph as though it were a simple cohort: every participant counts once, and counts the same. That's a real simplification, and it means the numbers we produce here would not be the right numbers to publish as national estimates. We do it because the statistical ideas this course teaches are hard enough to learn once, without learning them and survey weighting at the same time. Survey weighting is an Advanced Statistics topic, and that's where you'll meet it properly.

You'll see this pattern repeatedly in the book: we name the simplification, say what it costs, and point at where the honest version lives. Getting into the habit of asking "what has this analysis assumed?" matters more than any single technique in here.

3.3 Setup

As in IES Week 5, we read the data with `read_csv()` from the `readr` package. We build the file path with `here()` from the `here` package. This builds a path from the project folder down to the file, so the code runs on any machine no matter where the project sits. `nhanes_mph` arrives as two files: the data itself, and a separate dictionary file that names every variable. Both are plain CSV files, so we read both with `read_csv()`.

This chapter loads one package new to the book: `gtsummary`, for building tables. `dplyr`, for manipulating data, was loaded back in IES Week 4, and `ggplot2`, for plotting, in IES Week 5, so both are already familiar. As IES Week 4 explained, installing a package and loading it are two different actions, so if you haven't installed `gtsummary` already, run this once in the console, not inside this document:

```
install.packages("gtsummary")
```

After that, the `library()` calls below are all you need, this session and every session after it.

```r
library(readr)
library(dplyr)
library(ggplot2)
library(gtsummary)
library(here)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))
dictionary <- read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
```

Running that in your own console prints more than it does on this page. The `library()` calls announce themselves, and each `read_csv()` prints the column specification message IES Week 5 unpacked. This book hides both from here on, because they say the same thing in every chapter and that earlier chapter already read one properly. Nothing is being kept from you: glance at the row and column counts in your own console the way it showed, and carry on.

`glimpse()`, from `dplyr`, shows every column, its type, and its first few values. It's useful for checking what actually came in, rather than what we expected to come in.

```r
glimpse(nhanes)
```

```
Rows: 41,765
Columns: 27
$ SEQN     <dbl> 12, 20, 34, 66, 69, 81, 97, 107, 120, 143, 158, 167, 170,~
$ age      <dbl> 37, 23, 38, 37, 27, 30, 32, 30, 30, 22, 29, 32, 28, 31, 3~
$ gender   <dbl> 1, 2, 2, 1, 1, 1, 1, 2, 2, 2, 1, 2, 1, 2, 2, 1, 1, 2, 2, ~
$ eth      <dbl> 3, 1, 4, 1, 4, 3, 4, 1, 3, 1, 1, 2, 1, 4, 3, 3, 2, 3, 1, ~
$ edu      <dbl> 3, 1, 2, 3, 3, 3, 3, 2, 3, 1, 1, 3, 1, 1, 2, 3, 1, 2, 1, ~
$ smoking  <chr> "Never", "Previous", "Current", "Never", "Never", "Never"~
$ alc_wk   <dbl> 0.46029919, 3.00000000, 1.00000000, NA, 2.00000000, 1.000~
$ vpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ wal_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ sed      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ chol     <dbl> 4.03, 3.75, 5.04, 5.56, 5.66, 8.90, NA, 3.57, 6.57, 6.67,~
$ phq      <dbl> 0, 0, NA, 0, 0, 0, 0, 0, 0, 0, 0, NA, 0, NA, 2, 0, 0, 0, ~
```

```
$ phq.c    <chr> "0 Normal", "0 Normal", NA, "0 Normal", "0 Normal", "0 No~
$ blm.cvd  <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ dth      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ lcod     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "hrt", NA, NA, NA, NA~
$ ucod_dm  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ ucod_ht  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ fu_month <dbl> 236, 245, 244, 238, 245, 243, 242, 243, 247, 231, 235, 24~
```

Six columns describe a category but aren't usable as one yet: `gender`, `eth`, `edu`, `smoking`, `phq.c`, and `blm.cvd`. Some of them, like `gender`, are plain integers standing in for a label. Others are more mixed. `glimpse()` shows `smoking` and `phq.c` as character columns, not numbers at all. Whatever the raw form, we should never guess the *meaning* of a code from the column name. It lives in the dictionary, and that's the first place to look.

```
dictionary |>
  filter(`Variable names` %in% c("gender", "eth", "edu", "smoking", "phq.c", "blm.cvd"))
```

```
# A tibble: 6 x 2
  `Variable names` Description
  <chr>            <chr>
1 gender           Gender (1=Male; 2=Female)
2 eth              Ethnicity (1=Mexican; 2=Other Hispanic; 3=White; 4=Black~
3 edu              Educaton (1=Less than high school; 2=high school graduat~
4 smoking          Smoking status (Never/Previous/Current)
5 phq.c            Depression classification - PHQ-2 (0 Normal/1 Mild sympt~
6 blm.cvd          Baseline CVD (0=No; 1=Yes)
```

All six are documented, but not all in the same way. For `gender`, `eth`, `edu`, and `blm.cvd`, the dictionary gives a numeric code against each category: `1=Male; 2=Female`, and so on. Those are the four we have to translate. For `smoking` and `phq.c` it just lists the categories, and `glimpse()` already told us why no translation is needed: they don't arrive as bare numbers at all. `smoking` is already the text "Never", "Previous", or "Current". `phq.c` is already a label too, with the original code number kept at the front of the string, for example `"0 Normal"`.

That last detail is worth pausing on, because it shows what each source is good for. The dictionary tells us what a column *means*. It doesn't tell us what shape the values arrive in, and nothing in the entry for `phq.c` warns us that the code number is glued to the front of every label. `glimpse()` tells us the shape but not the meaning. Read together they answer the question; read on their own, either one leaves you guessing.

The dictionary isn't exhaustive, either. `dth`, the death indicator we use in IES Week 11, is described only as "Death in follow-up", with no codes given at all, so which of 0 and 1 means died has to be established some other way. That's the exception rather than the rule here, and it's still a better starting point than writing code against a column's name and hoping.

We turn every one of the six into a factor with `mutate()` and `factor()`, whether the raw values are numbers or text. `factor()` takes the raw values (`levels`) and the labels we want to give them (`labels`), in matching order. The labels are ours to choose, and we take one small liberty below: the dictionary abbreviates `eth`'s first category to "Mexican", and we write it out as "Mexican American", the full NHANES category name. Wording a label more clearly than the dictionary does is fine. Changing which code it sits against is not, and that's the part to copy exactly. One of the six, `phq.c`, is ordinal: its categories run from normal to possible depression in a natural order, so we build it with `ordered = TRUE`. `edu` is ordinal too, in principle, less education to more, but we build it as a plain factor instead. IES Week 11 fits a regression model against it and reads each category's effect directly against a reference level, which is simplest with a plain factor rather than the trend-based contrasts an ordered factor produces. `gender`, `eth`, `smoking`, and `blm.cvd` are nominal, with no natural order, so they're plain factors for the more familiar reason. By default, the first level listed becomes the *reference* level. This is the category everything else gets compared against once regression starts, in IES Week 11. For now the choice mostly affects how a bar chart or table orders its categories, but it's worth setting deliberately from the start rather than accepting whatever order R guesses.

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c("Male", "Female")),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c("Mexican American", "Other Hispanic", "White", "Black", "Others")),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c("Less than high school", "High school graduate", "Above high school")),
    smoking = factor(smoking,
      levels = c("Never", "Previous", "Current")),
    phq.c = factor(phq.c,
      levels = c("0 Normal", "1 Mild symptoms", "2 Possible depression"),
      labels = c("Normal", "Mild symptoms", "Possible depression"),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c("No", "Yes"))
  )
```

```r
glimpse(nhanes)
```

```
Rows: 41,765
Columns: 27
$ SEQN     <dbl> 12, 20, 34, 66, 69, 81, 97, 107, 120, 143, 158, 167, 170,~
$ age      <dbl> 37, 23, 38, 37, 27, 30, 32, 30, 30, 22, 29, 32, 28, 31, 3~
$ gender   <fct> Male, Female, Female, Male, Male, Male, Male, Female, Fem~
$ eth      <fct> White, Mexican American, Black, Mexican American, Black, ~
$ edu      <fct> Above high school, Less than high school, High school gra~
$ smoking  <fct> Never, Previous, Current, Never, Never, Never, Previous, ~
$ alc_wk   <dbl> 0.46029919, 3.00000000, 1.00000000, NA, 2.00000000, 1.000~
$ vpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ wal_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ mpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ vpa_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_w    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_r    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ met_t    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ sed      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ chol     <dbl> 4.03, 3.75, 5.04, 5.56, 5.66, 8.90, NA, 3.57, 6.57, 6.67,~
$ phq      <dbl> 0, 0, NA, 0, 0, 0, 0, 0, 0, 0, 0, NA, 0, NA, 2, 0, 0, 0, ~
$ phq.c    <ord> Normal, Normal, NA, Normal, Normal, Normal, Normal, Norma~
$ blm.cvd  <fct> No, No, No, No, No, No, No, No, No, No, No, No, No, No, N~
$ dth      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ lcod     <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "hrt", NA, NA, NA, NA~
$ ucod_dm  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ ucod_ht  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, 0, NA, NA, NA, NA, NA~
$ fu_month <dbl> 236, 245, 244, 238, 245, 243, 242, 243, 247, 231, 235, 24~
```

The blank and missing values in `smoking`, `edu`, `phq.c`, and `blm.cvd` have become `NA` in the process, just as they were before recoding. `factor()` relabels the values that are present and carries missingness through unchanged. From here on, every chapter opens with a short setup chunk that repeats these same steps, so each chapter stands on its own.

3.4 Checking and setting a reference level

We said above that the first level listed becomes the reference level. That was a claim, so let's actually look. `levels()`, from base R, lists a factor's categories in their current order:

```r
levels(nhanes$smoking)
```

```
[1] "Never"    "Previous" "Current"
```

`"Never"` is first, so `"Never"` is the reference. Everything else gets read against it: "Previous" means previous compared with never, and "Current" means current compared with never. That's the sensible choice here, and it isn't an accident. It's what we asked for when we wrote `levels = c("Never", "Previous", "Current")` in the recode above. Change the order in that line and you change the reference.

Sometimes we want a different reference without rebuilding the factor. `relevel()`, also from base R, takes a factor and moves the level named in `ref` to the front:

```r
eth_white_ref <- relevel(nhanes$eth, ref = "White")
levels(eth_white_ref)
```

```
[1] "White"            "Mexican American" "Other Hispanic"
[4] "Black"            "Others"
```

`"White"` has moved to the front, so it would now be the reference, with the other four groups read against it. The rest keep their order, just shifted along.

Notice we assigned the result to a **new** object, `eth_white_ref`, rather than writing it back into `nhanes`. That's deliberate. Every chapter in this book rebuilds `nhanes` with the same recode, so if we quietly changed the factor here, this chapter's `nhanes` would stop matching the one IES Weeks 8 to 11 build, and a table would come out in a different order for no visible reason. When we're demonstrating something, we work on a copy.

Right now the reference level only decides the order categories appear in a table, or along a bar chart's axis. In IES Week 11 it decides what every coefficient in a regression model means, which is a much bigger deal. Checking it is cheap, so it's worth making the habit automatic well before it starts to matter.

3.5 Manipulating a dataset: the `dplyr` verbs

Four verbs from `dplyr` cover most of what we need to get from a raw dataset to the piece of it we actually want to look at.

`select()` keeps only the named columns:

```
nhanes |> select(age, gender, sed, chol)
```

```
# A tibble: 41,765 x 4
     age gender   sed  chol
   <dbl> <fct>  <dbl> <dbl>
 1    37 Male      NA  4.03
 2    23 Female    NA  3.75
 3    38 Female    NA  5.04
 4    37 Male      NA  5.56
 5    27 Male      NA  5.66
 6    30 Male      NA  8.9
 7    32 Male      NA NA
 8    30 Female    NA  3.57
 9    30 Female    NA  6.57
10    22 Female    NA  6.67
# i 41,755 more rows
```

`filter()` keeps only the rows that satisfy a condition. Here, that's participants under 40:

```
nhanes |> filter(age < 40)
```

```
# A tibble: 16,566 x 27
    SEQN   age gender eth      edu   smoking  alc_wk vpa_w mpa_w wal_r vpa_r
   <dbl> <dbl> <fct>  <fct>    <fct> <fct>     <dbl> <dbl> <dbl> <dbl> <dbl>
 1    12    37 Male   White    Abov~ Never    0.460     NA    NA    NA    NA
 2    20    23 Female Mexican~ Less~ Previo~  3         NA    NA    NA    NA
 3    34    38 Female Black    High~ Current  1         NA    NA    NA    NA
 4    66    37 Male   Mexican~ Abov~ Never   NA         NA    NA    NA    NA
 5    69    27 Male   Black    Abov~ Never    2         NA    NA    NA    NA
 6    81    30 Male   White    Abov~ Never    1         NA    NA    NA    NA
 7    97    32 Male   Black    Abov~ Previo~ NA         NA    NA    NA    NA
 8   107    30 Female Mexican~ High~ Never    0.0385    NA    NA    NA    NA
 9   120    30 Female White    Abov~ Previo~ NA         NA    NA    NA    NA
10   143    22 Female Mexican~ Less~ Never    0         NA    NA    NA    NA
# i 16,556 more rows
# i 16 more variables: mpa_r <dbl>, mpa_t <dbl>, vpa_t <dbl>, met_w <dbl>,
#   met_r <dbl>, met_t <dbl>, sed <dbl>, chol <dbl>, phq <dbl>,
#   phq.c <ord>, blm.cvd <fct>, dth <dbl>, lcod <chr>, ucod_dm <dbl>,
#   ucod_ht <dbl>, fu_month <dbl>
```

`mutate()` adds a new column, or changes an existing one, as we already saw above in the recoding step. Here it adds a new column that classifies participants as under or over 65:

```
nhanes |> mutate(over_65 = age >= 65) |> select(age, over_65)
```

```
# A tibble: 41,765 x 2
     age over_65
   <dbl> <lgl>
 1    37 FALSE
 2    23 FALSE
 3    38 FALSE
 4    37 FALSE
 5    27 FALSE
 6    30 FALSE
 7    32 FALSE
 8    30 FALSE
 9    30 FALSE
10    22 FALSE
# i 41,755 more rows
```

`arrange()` sorts the rows by a column, lowest to highest by default:

```
nhanes |> arrange(age)
```

```
# A tibble: 41,765 x 27
    SEQN   age gender eth       edu   smoking alc_wk vpa_w mpa_w wal_r vpa_r
   <dbl> <dbl> <fct>  <fct>     <fct> <fct>    <dbl> <dbl> <dbl> <dbl> <dbl>
 1 62253    18 Male   White     <NA>  <NA>     0      0    0         0 0.857
 2 62311    18 Male   White     <NA>  <NA>     0      0    0.107     0 0.214
 3 62455    18 Male   Black     <NA>  <NA>     0      0    0         0 0
 4 62598    18 Male   Black     <NA>  <NA>     0.230  0    0         0 0.429
 5 62659    18 Male   Others    <NA>  <NA>     0.192  2.86 2.86      0 4.29
 6 62833    18 Female White     <NA>  <NA>    NA      0    4         0 0
 7 62837    18 Female White     <NA>  <NA>     0.460  0    0         0 0
 8 62925    18 Female White     <NA>  <NA>     0      0    0         0 0
 9 62976    18 Female White     <NA>  <NA>     0.25   0    0.107     0 0
10 62978    18 Female Mexican ~ <NA>  <NA>     0.690  0    0         0 1.29
# i 41,755 more rows
# i 16 more variables: mpa_r <dbl>, mpa_t <dbl>, vpa_t <dbl>, met_w <dbl>,
#   met_r <dbl>, met_t <dbl>, sed <dbl>, chol <dbl>, phq <dbl>,
#   phq.c <ord>, blm.cvd <fct>, dth <dbl>, lcod <chr>, ucod_dm <dbl>,
#   ucod_ht <dbl>, fu_month <dbl>
```

These four combine in a short pipe to answer a real question. For example, who are the ten oldest current smokers in the sample?

```
nhanes |>
  filter(smoking == "Current") |>
  arrange(desc(age)) |>
  select(age, gender, smoking) |>
  slice(1:10)
```

```
# A tibble: 10 x 3
     age gender smoking
   <dbl> <fct>  <fct>
 1    85 Female Current
 2    85 Female Current
 3    84 Female Current
 4    84 Male   Current
 5    84 Female Current
 6    83 Male   Current
 7    82 Male   Current
 8    82 Male   Current
 9    81 Male   Current
10    81 Female Current
```

`desc()` reverses the sort order. `slice()`, also from `dplyr`, keeps only the rows at the given positions. Here, that's the first ten after sorting.

3.6 Grouped summaries

`group_by()` splits a dataset into groups without changing how it looks. `summarise()` then collapses each group down to one row of statistics. Used together, they answer questions like this: what's average cholesterol, separately for men and women?

```
nhanes |>
  filter(!is.na(gender), !is.na(chol)) |>
  group_by(gender) |>
  summarise(
    mean_chol = mean(chol, na.rm = TRUE),
    sd_chol = sd(chol, na.rm = TRUE),
    median_chol = median(chol, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 2 x 5
  gender mean_chol sd_chol median_chol     n
  <fct>      <dbl>   <dbl>       <dbl> <int>
1 Male        4.89    1.09        4.81 18867
2 Female      5.05    1.08        4.97 20309
```

Three of those functions are new. `mean()` we already met in IES Week 4. `sd()`, from base R, returns the standard deviation: a measure of how spread out the values are, in the same units as the variable itself. `median()`, also base R, returns the middle value once everything is sorted, so half the participants sit below it and half above. Both are the summaries the seminar introduced, now with a function attached to each.

`na.rm = TRUE` tells `mean()` and `sd()` to drop missing values before calculating. Without it, a single `NA` in a column makes the whole result `NA`. The `filter()` step above drops rows where `gender` or `chol` itself is missing, so the group sizes reported in `n` reflect exactly the rows the means are calculated from. `n()`, used inside `summarise()`, counts the rows in each group.

The `gender` half of that filter turns out to change nothing: every participant in this extract has a gender recorded, so there are no rows for it to drop. We write it anyway, here and everywhere else in this book, and it's worth being clear that this is a deliberate habit rather than an oversight. You only know a column has no missing values because you checked, and the cost of checking every time is one short line that does nothing when there's nothing to do. The `chol` half of the same filter is doing real work: 2,589 participants have no cholesterol value, and without it they'd be counted in `n` while contributing nothing to the mean.

Whether we report the mean and SD or the median and IQR depends on shape, not habit. A skewed distribution gets misrepresented by a mean, because the mean gets pulled toward the tail. The seminar's rule was to look at the shape before choosing. We'll build the plot we need for that in a moment, but categorical variables need a different kind of summary first: a count, not a mean.

3.7 Frequency and cross-tabulations

`table()`, from base R, counts how many rows fall into each category of a variable. Given one variable, it produces a **frequency table**:

```r
table(nhanes$smoking)
```

```

   Never Previous  Current 
   23394     9468     8580 
```

`table()` drops `NA` values by default rather than counting them as their own category, so this count is out of everyone with a recorded smoking status, not out of everyone in the dataset. Raw counts are hard to compare across groups of different sizes, so we usually want percentages instead. `prop.table()`, also from base R, takes a table and turns its counts into proportions of the total:

```r
table(nhanes$smoking) |> prop.table()
```

```

    Never  Previous   Current 
0.5644998 0.2284639 0.2070363 
```

Multiplying by 100 and rounding turns those proportions into percentages, in whatever way is easiest to read.

Giving `table()` two variables instead of one produces a **cross-tabulation**: a two-way table with one variable's categories as rows and the other's as columns, one cell per combination. Here's smoking status by gender:

```r
smoking_gender <- table(nhanes$smoking, nhanes$gender)
smoking_gender
```

```

           Male Female
  Never    9302  14092
  Previous 5704   3764
  Current  4913   3667
```

The rows are `smoking`, the columns are `gender`, because that's the order the two variables were given to `table()` in. As with the frequency table above, raw counts are easier to compare as percentages, but now there's a choice to make: percentage of what? `prop.table()` takes a `margin` argument for this. `margin = 1` divides each row by its own row total, giving **row percentages**, the breakdown of gender within each smoking category:

```r
prop.table(smoking_gender, margin = 1)
```

```

                Male    Female
  Never    0.3976233 0.6023767
  Previous 0.6024504 0.3975496
  Current  0.5726107 0.4273893
```

`margin = 2` divides each column by its own column total instead, giving **column percentages**, the breakdown of smoking status within each gender:

```
prop.table(smoking_gender, margin = 2)
```

```
               Male    Female
  Never    0.4669913 0.6547414
  Previous 0.2863598 0.1748827
  Current  0.2466489 0.1703759
```

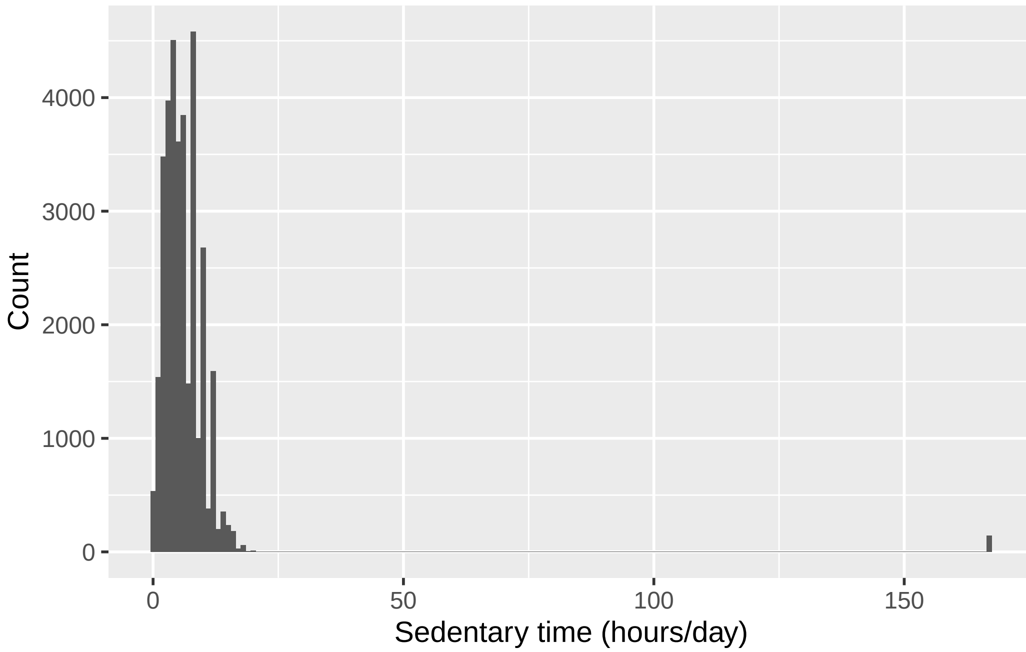
Both tables are built from the same counts, but they answer different questions, and the research question decides which one belongs in a report. This is the same two-by-two logic the epidemiology half already uses for exposure-by-outcome tables, just with more than two categories on one side.

3.8 Plotting with `ggplot2`

Every `ggplot2` plot is built from the same three pieces: the **data**, the **aesthetic mapping** (`aes()`, which decides which variables go on which axis or colour), and the **geometry** (`geom_*()`, which decides what kind of shape represents the data). These pieces are joined with `+`, not the pipe `|>` we've been using so far. That's a genuine inconsistency, not a typo to ignore. `ggplot2` was written before the pipe existed, and `+` has stuck ever since as the way to add one layer of a plot on top of another. Read `+` here as "and add this layer", the same way `|>` reads as "and then". The two aren't interchangeable, though. `ggplot2` code is joined with `+`, and everything else in this course is joined with `|>`. Here's a histogram of sedentary time:

```
ggplot(nhanes, aes(x = sed)) +
  geom_histogram(binwidth = 1) +
  labs(x = "Sedentary time (hours/day)", y = "Count")
```

```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_bin()`).
```



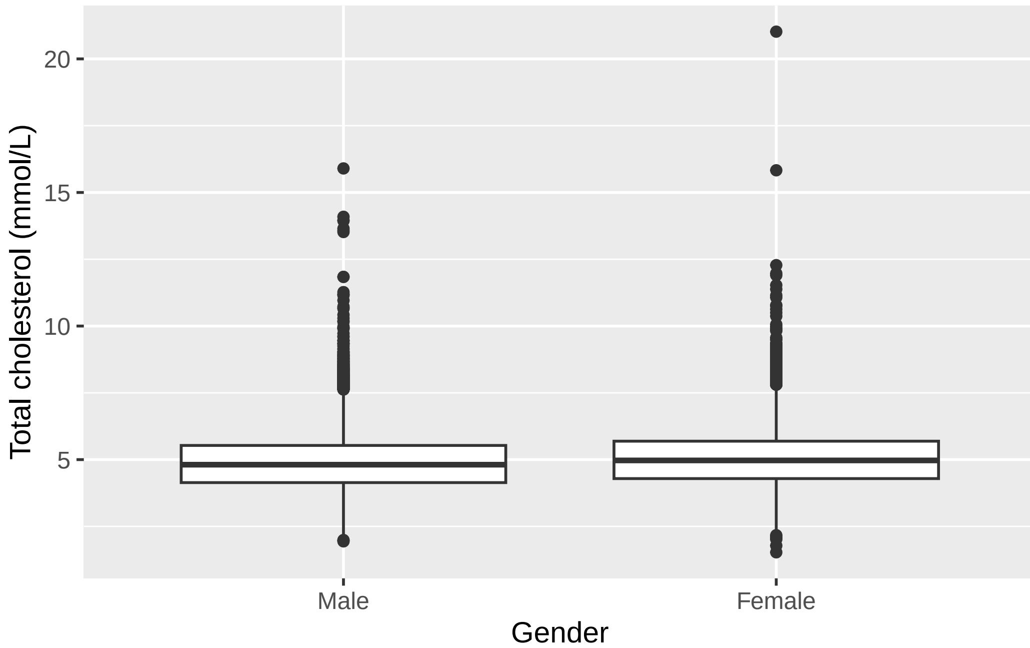
There are three layers there. `ggplot()` names the data and the mapping, `geom_histogram()` says to draw it as a histogram, and `labs()` sets the axis labels. `labs()` is worth using every single time. Left out, the axes are labelled with the raw column name, so this one would have read `sed` rather than "Sedentary time (hours/day)", and nobody outside this course knows what `sed` means or what units it's in. A plot that needs explaining out loud isn't finished. Every plot from here on sets its labels, and so should yours.

That warning about removed rows is the one IES Week 5 explained: `sed` has 7,331 participants with no recorded value, and they can't be placed on the axis, so `ggplot2` left them out and said so. It will appear above most plots in this chapter, with a different number each time depending on the variable. Glance at the number, check it's the size you expect, and carry on.

A boxplot of cholesterol by gender:

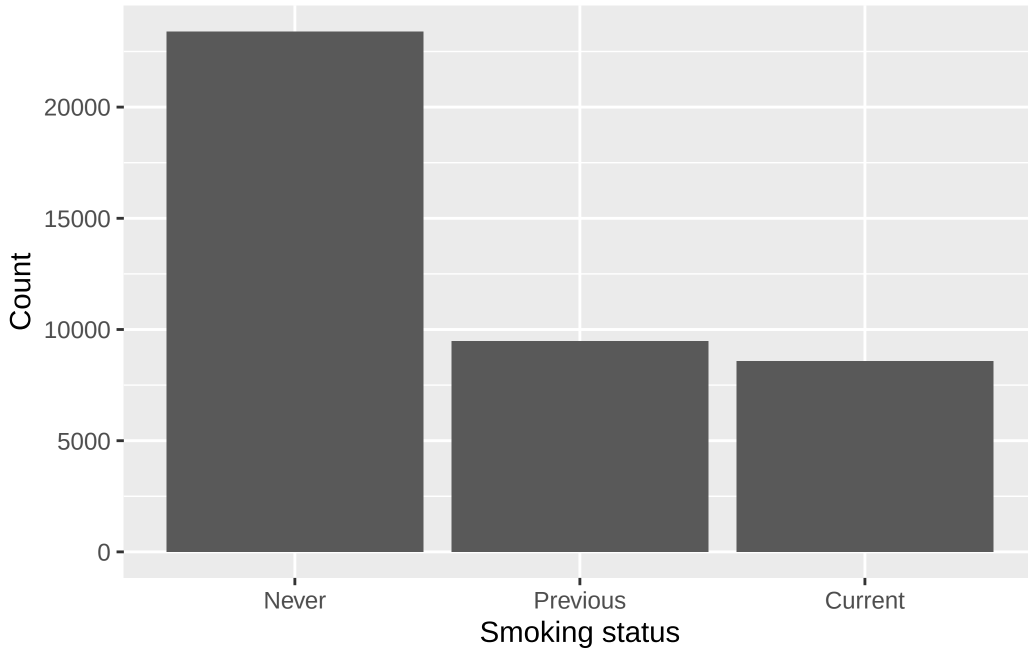
```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = chol)) +
  geom_boxplot() +
  labs(x = "Gender", y = "Total cholesterol (mmol/L)")
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



A bar chart of smoking status:

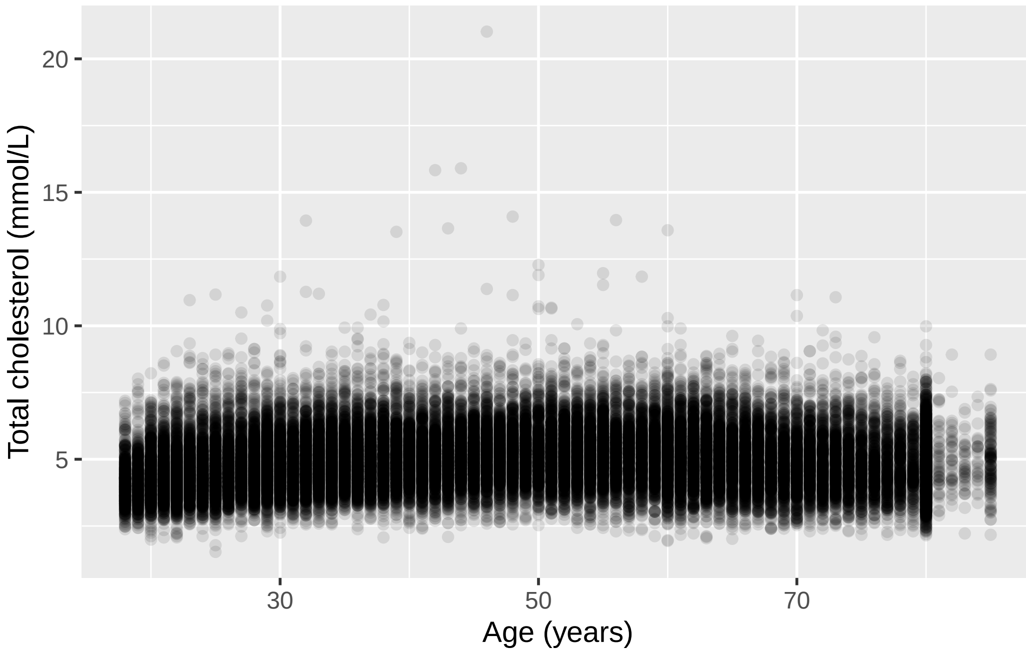
```
ggplot(nhanes |> filter(!is.na(smoking)), aes(x = smoking)) +
  geom_bar() +
  labs(x = "Smoking status", y = "Count")
```



A scatterplot of cholesterol against age:

```
ggplot(nhanes, aes(x = age, y = chol)) +
  geom_point(alpha = 0.1) +
  labs(x = "Age (years)", y = "Total cholesterol (mmol/L)")
```

```
Warning: Removed 2589 rows containing missing values or values outside the scale
range (`geom_point()`).
```



`alpha = 0.1` makes each point mostly transparent. Where thousands of points overlap, the darker regions show where the data are dense. A plain scatterplot of this many points would otherwise just be a solid block of ink.

> **Try it yourself: bin width and scale**
>
> Take the histogram of `sed` above. Change `binwidth` to `0.1`, then to `5`, and re-run it each time. Then add `+ ylim(0, 500)` to the original plot. `ylim()`, from `ggplot2`, fixes the lower and upper limits of the y-axis by hand instead of letting the plot choose them to fit the data. Be careful with it: anything falling outside the limits you set is dropped from the plot altogether rather than being drawn cut off at the edge, so a bar taller than the limit doesn't get shortened, it disappears. For each change, write one sentence on what altered in the plot, and one sentence on what the data themselves did not change. This is the same distinction the seminar drew between a plot design choice and the underlying distribution.
>
> This is the first of several "Try it yourself" boxes in the book, and none of them has a written solution. Running the code is the answer. Exercise 3 below covers the same ground in a form that does have one.

3.9 A first Table 1

A "Table 1" is the standard first table in a health research report. It shows participant characteristics, often split by a grouping variable. `tbl_summary()`, from `gtsummary`, builds one directly from a data frame.

```
nhanes |>
  select(age, gender, eth, smoking, chol) |>
  tbl_summary()
```

Characteristic	**N = 41,765**
age	46 (31, 63)
gender	
Male	20,085 (48%)
Female	21,680 (52%)
eth	
Mexican American	6,849 (16%)
Other Hispanic	3,893 (9.3%)
White	17,297 (41%)
Black	9,086 (22%)
Others	4,640 (11%)
smoking	
Never	23,394 (56%)
Previous	9,468 (23%)
Current	8,580 (21%)
Unknown	323
chol	4.89 (4.22, 5.61)
Unknown	2,589

[1] Median (Q1, Q3); n (%)

Adding `by =` splits the table by a grouping variable, giving one column per group:

```
nhanes |>
  select(age, gender, eth, smoking, chol) |>
  tbl_summary(by = gender)
```

Characteristic	**Male** N = 20,085	**Female** N = 21,680
age	47 (32, 63)	45 (31, 62)
eth		
Mexican American	3,284 (16%)	3,565 (16%)
Other Hispanic	1,717 (8.5%)	2,176 (10%)
White	8,482 (42%)	8,815 (41%)
Black	4,339 (22%)	4,747 (22%)
Others	2,263 (11%)	2,377 (11%)
smoking		
Never	9,302 (47%)	14,092 (65%)
Previous	5,704 (29%)	3,764 (17%)
Current	4,913 (25%)	3,667 (17%)
Unknown	166	157
chol	4.81 (4.14, 5.53)	4.97 (4.29, 5.69)
Unknown	1,218	1,371

[1] Median (Q1, Q3); n (%)

`tbl_summary()` automatically chooses mean and SD or median and IQR for each numeric column, based on its distribution. That's the same decision we made by eye above, now made for every column at once.

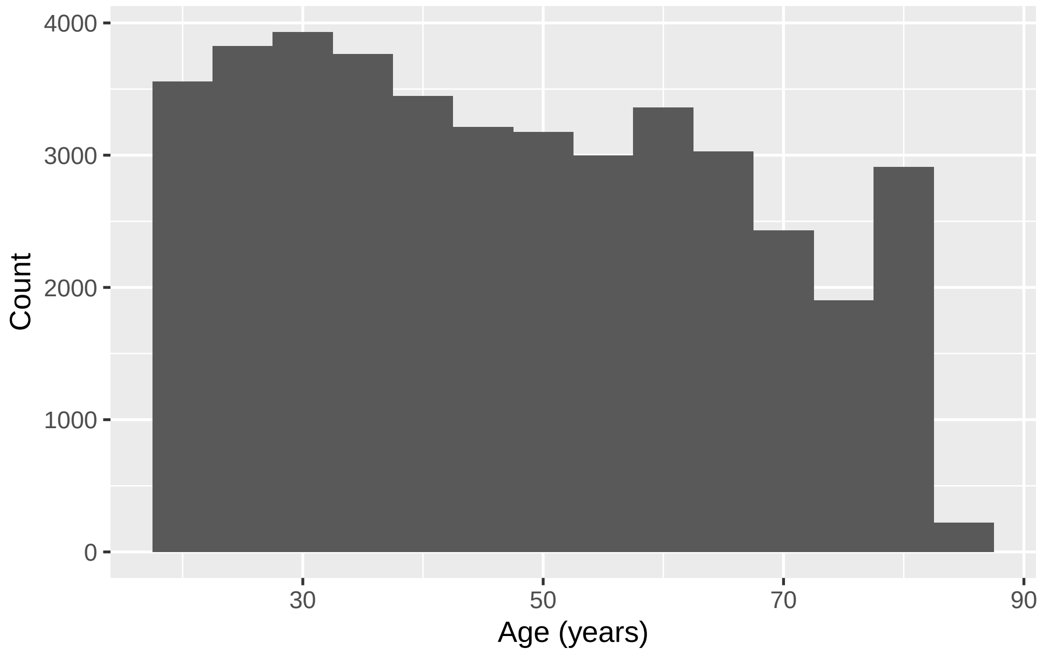
3.10 Worked example: describing age

Now let's bring the tools above together, on a variable we haven't used elsewhere in this chapter: `age`. Alongside `mean()`, `sd()`, and `median()`, we bring in one new function: `IQR()`, from base R, returns the interquartile range, the gap between the 25th and 75th percentile, in the same units as the variable itself. It's the number the "IQR" in "median and IQR" refers to.

```
nhanes |>
  summarise(
    mean_age = mean(age, na.rm = TRUE),
    sd_age = sd(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    iqr_age = IQR(age, na.rm = TRUE)
  )
```

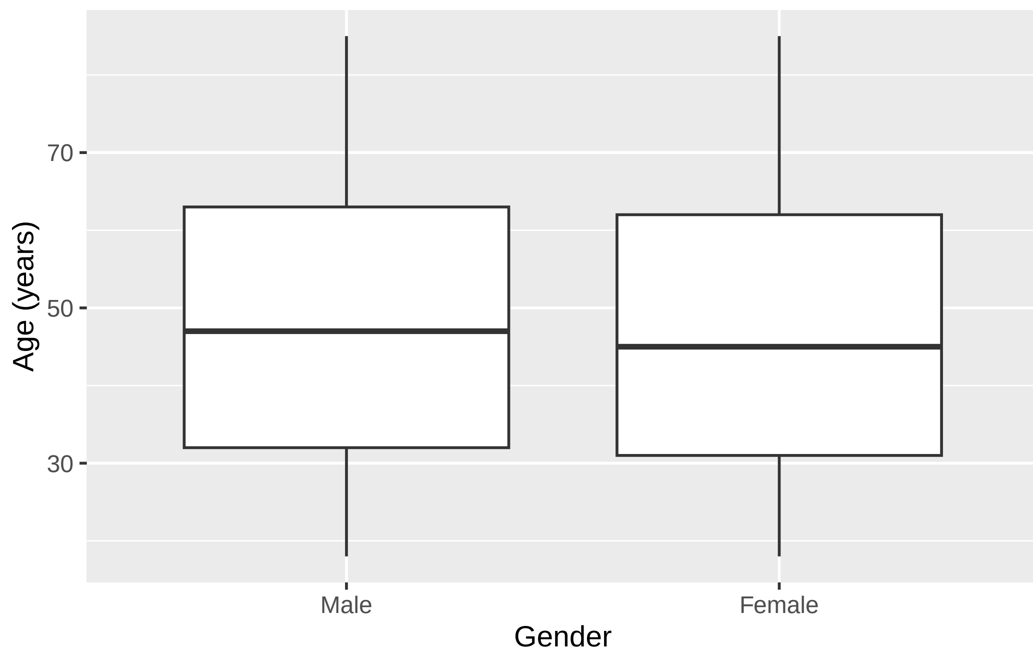
```
# A tibble: 1 x 4
  mean_age sd_age median_age iqr_age
     <dbl>  <dbl>      <dbl>   <dbl>
1     47.5   18.5         46      32
```

```r
ggplot(nhanes, aes(x = age)) +
  geom_histogram(binwidth = 5) +
  labs(x = "Age (years)", y = "Count")
```



The histogram is close to symmetric, with no strong tail in either direction. So the mean and SD are a fair summary here. Median and IQR would tell much the same story. Now let's split by gender:

```r
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = age)) +
  geom_boxplot() +
  labs(x = "Gender", y = "Age (years)")
```




```
nhanes |>
  select(age, gender) |>
  tbl_summary(by = gender)
```

Characteristic	Male N = 20,085	Female N = 21,680
age	47 (32, 63)	45 (31, 62)

[1] Median (Q1, Q3)

Age looks similar across the two gender groups, both in the boxplot and in the Table 1. There's no obvious difference to report here, and it would be wrong to manufacture one.

3.11 Exercises

Exercise 1 (ILO 1). Using the four `dplyr` verbs, and a pipe, answer this: among participants aged 65 or over who have never smoked, what are the ten highest cholesterol values? Keep only `age`, `gender`, and `chol` in your answer. Then add a column called `chol_high` that is `TRUE` when `chol` is above 6 mmol/L and `FALSE` otherwise.

Exercise 2 (ILOs 1, 3). Describe the distribution of sedentary time (`sed`). Report the mean and SD or the median and IQR, whichever the shape justifies, and support your choice with a histogram and a boxplot. Then produce a scatterplot of `chol` against `sed`, label both axes with their units, and write one sentence on whether the plot suggests any relationship between the two.

Exercise 3 (ILOs 3, 4). Build a histogram of total cholesterol (`chol`) with a `binwidth` of 0.5, labelled properly with `labs()`. Then produce two more versions of it: one with a `binwidth` of 0.05, and one with the original bin width but with `+ ylim(0, 1000)` added. Write one sentence for each of the two changes saying what it does to the impression the plot gives, and one sentence saying which of the three you would put in a report and why.

Exercise 4 (ILOs 1, 3, 5). Compare total cholesterol (`chol`) by gender. Produce a grouped summary and a boxplot, and write one sentence comparing the two groups. Then use `levels()` to state which gender is currently the reference level, and `relevel()` to make the other one the reference instead, assigning the result to a new object rather than overwriting `nhanes`.

Exercise 5 (ILOs 2, 5, 6). Produce a Table 1 of `age`, `gender`, `eth`, `smoking`, and `chol`, stratified by depression classification (`phq.c`).

Exercise 6 (ILOs 2, 3). Build a cross-tabulation of smoking status (`smoking`) against education (`edu`) with `table()`. Produce both the row percentages and the column percentages with `prop.table()`.

Then answer this question using whichever of the two is appropriate: *among people who never smoked, how is education distributed?* State which of your two tables answers it, and say in one sentence why the other one doesn't. Finally, produce a bar chart of `edu`, labelled properly.

3.12 Writing exercise

Every practical in this book ends with one of these, and it's worth knowing why before you write the first one. The module's assessment is a structured paper on a dataset you haven't seen, where every question asks for an analysis *and* the sentence that interprets it, and the interpretation carries half the marks. So these exercises are the assessment in miniature, five times over, before it counts. Output without a sentence around it is an unfinished answer here and an incomplete one there.

Using the Table 1 from Exercise 5, write a short paragraph of about 150 words, as you would for the results section of a report. Describe how participant characteristics differ, or don't, across the three depression classification groups. State which characteristics look similar across groups and which look different, and avoid overstating a difference the table doesn't support.

Start by stating how many participants the table is actually based on, and how many were excluded because they had no depression classification recorded. The number is in the message printed above the table. A results paragraph that doesn't say what it's counting is incomplete, however good the rest of the sentence is.

Solutions

Exercise 1

```
nhanes |>
  filter(age >= 65, smoking == "Never") |>
  arrange(desc(chol)) |>
  select(age, gender, chol) |>
  slice(1:10)
```

```
# A tibble: 10 x 3
     age gender  chol
   <dbl> <fct>  <dbl>
 1    73 Female 11.1
 2    70 Female 10.4
 3    80 Female  9.98
 4    76 Female  9.57
 5    73 Female  9.36
 6    65 Female  9.13
 7    71 Female  9.05
 8    71 Female  9.05
 9    82 Female  8.92
10    85 Female  8.92
```

`filter()` takes two conditions separated by a comma, which means the same as `&`: both have to be true. Then `arrange(desc(chol))` sorts from highest cholesterol down, `select()` keeps the three columns asked for, and `slice(1:10)` takes the top ten rows. The `mutate()` step is separate, because adding a column to ten hand-picked rows isn't much use. Applied to the whole dataset it is:

```
nhanes |>
  mutate(chol_high = chol > 6) |>
  select(age, chol, chol_high)
```

```
# A tibble: 41,765 x 3
     age  chol chol_high
   <dbl> <dbl> <lgl>
 1    37  4.03 FALSE
 2    23  3.75 FALSE
 3    38  5.04 FALSE
 4    37  5.56 FALSE
 5    27  5.66 FALSE
```

```
 6    30  8.9  TRUE
 7    32 NA    NA
 8    30  3.57 FALSE
 9    30  6.57 TRUE
10    22  6.67 TRUE
# i 41,755 more rows
```

`chol > 6` is a comparison, exactly like the ones in IES Week 4, and it returns `TRUE` or `FALSE` for every row. Participants with no cholesterol value recorded get `NA` rather than `FALSE`, which is correct: we don't know whether they're above 6, and guessing "no" would be inventing data.

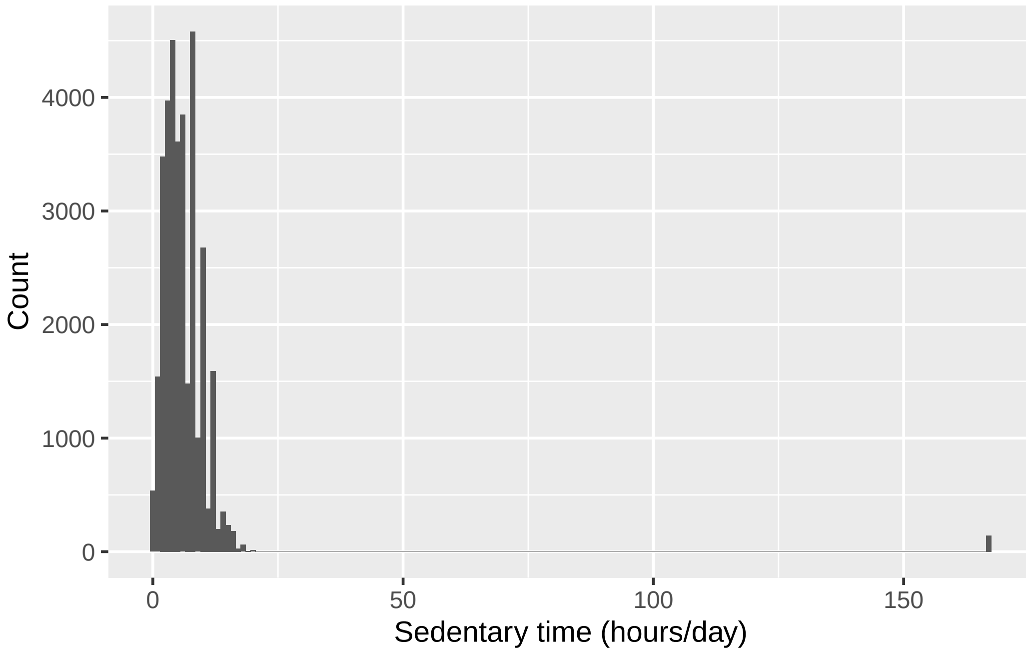
Exercise 2

```r
nhanes |>
  summarise(
    median_sed = median(sed, na.rm = TRUE),
    iqr_sed = IQR(sed, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 2
  median_sed iqr_sed
       <dbl>   <dbl>
1          5       5
```

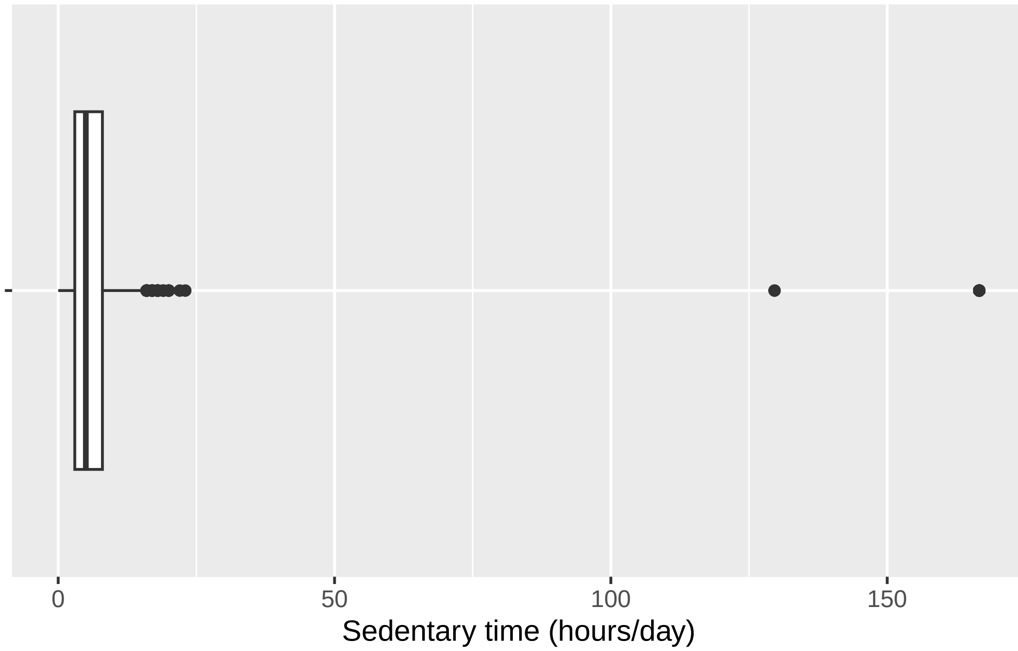
```r
ggplot(nhanes, aes(x = sed)) +
  geom_histogram(binwidth = 1) +
  labs(x = "Sedentary time (hours/day)", y = "Count")
```

```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_bin()`).
```




```
ggplot(nhanes, aes(x = sed, y = "")) +
  geom_boxplot() +
  labs(x = "Sedentary time (hours/day)", y = NULL)
```

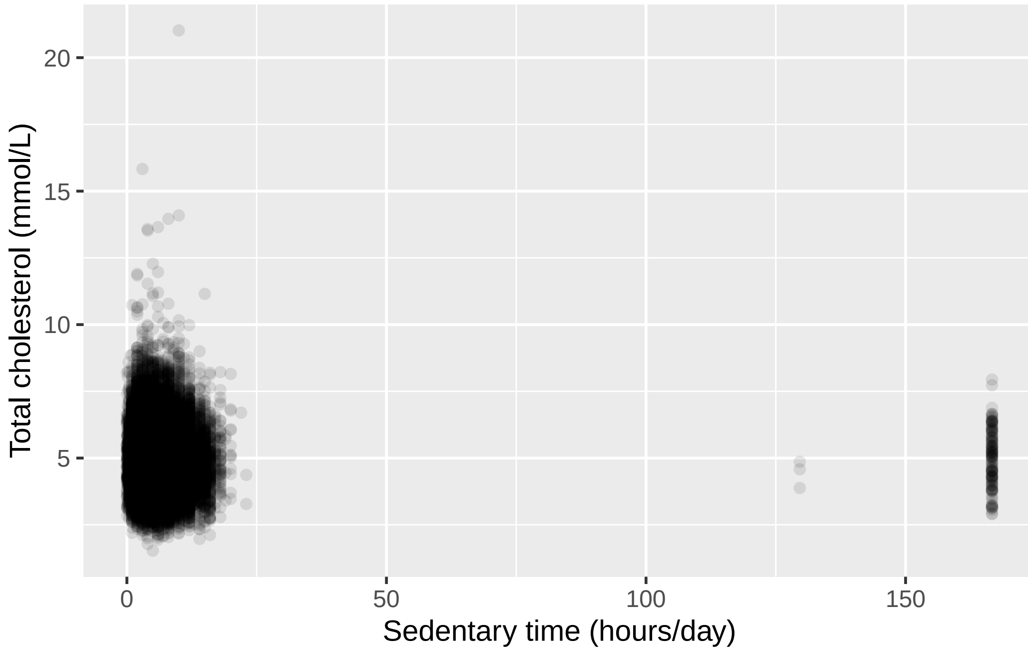
```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



The histogram is right-skewed, with a long tail of participants reporting many hours of sedentary time. So the median and IQR describe it more fairly than the mean and SD would.

```
ggplot(nhanes, aes(x = sed, y = chol)) +
  geom_point(alpha = 0.1) +
  labs(x = "Sedentary time (hours/day)",
       y = "Total cholesterol (mmol/L)")
```

```
Warning: Removed 9456 rows containing missing values or values outside the scale
range (`geom_point()`).
```

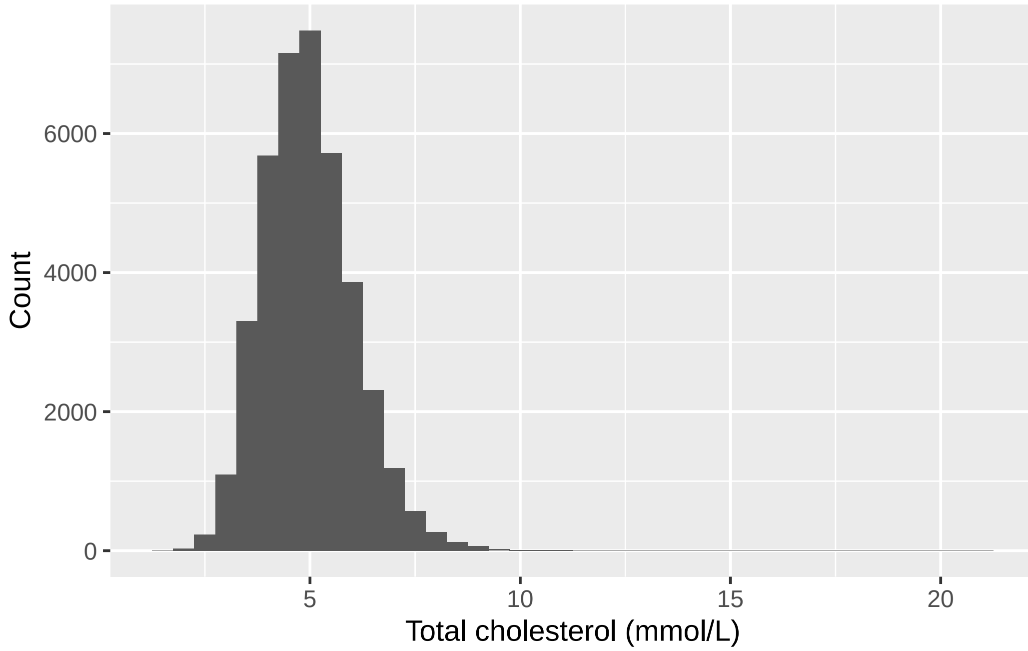


The scatterplot shows no clear relationship. The cloud of points is roughly the same height right across the range of sedentary time, rather than drifting up or down as you move left to right. Whatever else predicts cholesterol here, sedentary time on its own doesn't look like much of it. IES Week 10 puts a number and a confidence interval on that impression instead of leaving it to the eye.

Exercise 3

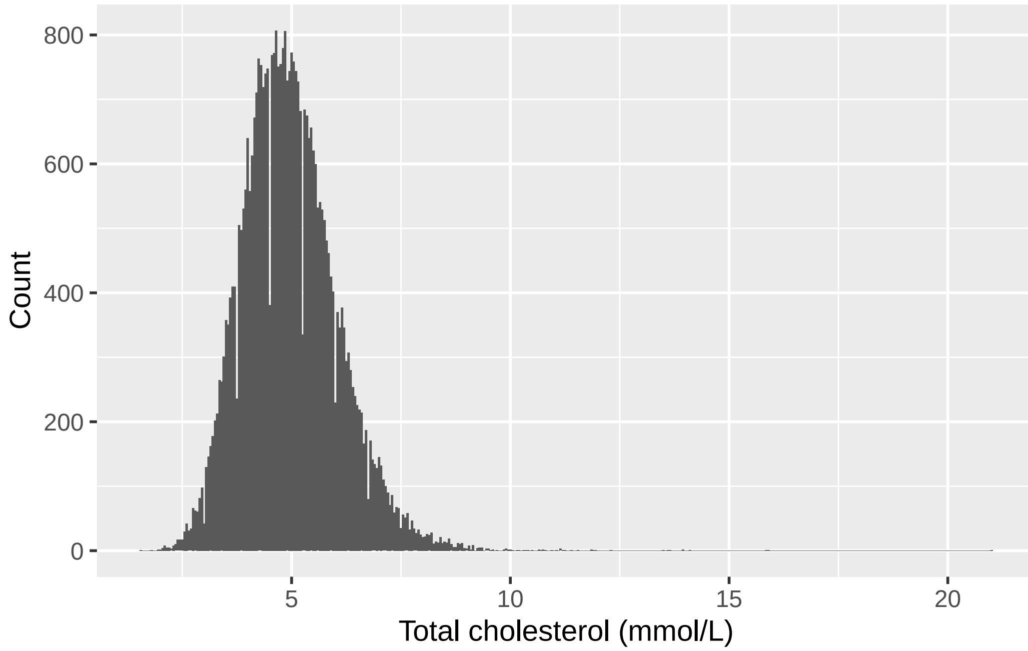
```
ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5) +
  labs(x = "Total cholesterol (mmol/L)", y = "Count")
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```




```r
ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.05) +
  labs(x = "Total cholesterol (mmol/L)", y = "Count")
```

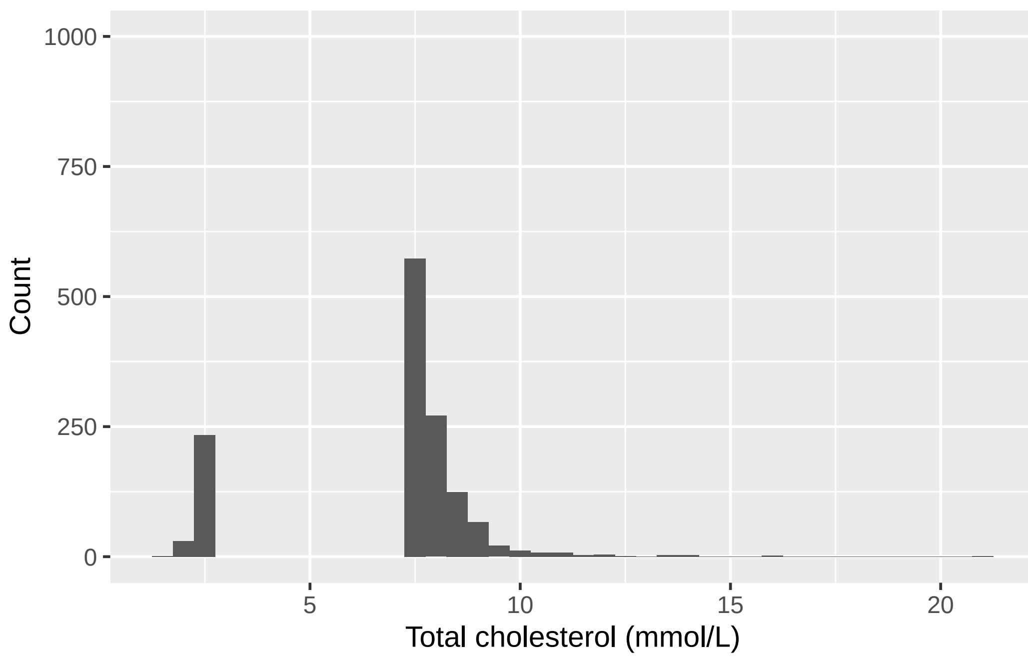
```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```




```
ggplot(nhanes, aes(x = chol)) +
  geom_histogram(binwidth = 0.5) +
  ylim(0, 1000) +
  labs(x = "Total cholesterol (mmol/L)", y = "Count")
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_bin()`).
```

```
Warning: Removed 9 rows containing missing values or values outside the scale range
(`geom_bar()`).
```



Narrowing the bins to 0.05 breaks the smooth shape into a spiky, gappy outline. Cholesterol is recorded to a limited number of decimal places, so at this bin width many bins fall in the gaps between recorded values and come out empty. Those spikes are an artefact of how the values were written down, not a real feature of cholesterol.

Capping the y-axis at 1,000 does something worse, and worth seeing once. Every bar taller than 1,000 doesn't get its top trimmed off, it vanishes from the plot completely. The tallest bar here counts over 7,000 participants, and the whole middle of the distribution is taller than the cap, so what's left is the two tails with a hole where most of the data should be. A reader who didn't check the code would have no way of knowing that anything was missing.

The first version is the one to put in a report. It shows the whole distribution at a bin width where the shape is readable, and it doesn't drop anything. Notice that all three plots are drawn from exactly the same numbers: nothing about cholesterol changed between them, only the choices we made about how to draw it. That's the seminar's point about misleading figures, made with our own data and our own code rather than someone else's published chart.

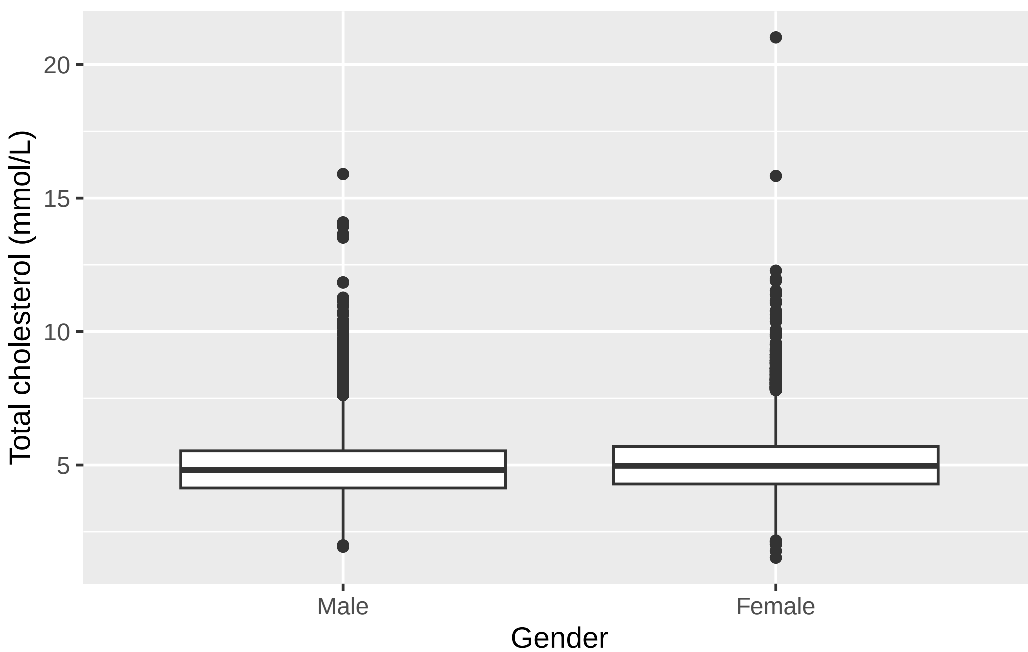
Exercise 4

```r
nhanes |>
  filter(!is.na(gender)) |>
  group_by(gender) |>
  summarise(
    mean_chol = mean(chol, na.rm = TRUE),
    sd_chol = sd(chol, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 2 x 4
  gender mean_chol sd_chol     n
  <fct>      <dbl>   <dbl> <int>
1 Male        4.89    1.09 20085
2 Female      5.05    1.08 21680
```

```r
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = chol)) +
  geom_boxplot() +
  labs(x = "Gender", y = "Total cholesterol (mmol/L)")
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



Mean cholesterol is slightly higher in women than in men, and the boxplots overlap substantially. That's a small difference in the centre, not a separation of the two distributions.

```
levels(nhanes$gender)
```

```
[1] "Male"   "Female"
```

```
gender_female_ref <- relevel(nhanes$gender, ref = "Female")
levels(gender_female_ref)
```

```
[1] "Female" "Male"
```

`"Male"` is listed first, so `"Male"` is currently the reference level, which follows from the recode listing the numeric code `1` first and labelling it `"Male"`. After `relevel()`, `"Female"` is at the front of the new object and would be the reference instead. `nhanes` itself is untouched.

Exercise 5

```
nhanes |>
  select(age, gender, eth, smoking, chol, phq.c) |>
  tbl_summary(by = phq.c)
```

```
4499 missing rows in the "phq.c" column have been removed.
```

Characteristic	Normal N = 25,178	Mild symptoms N = 8,628	Possible depression N = 3,460
age	46 (31, 63)	46 (30, 61)	51 (36, 63)
gender			
Male	13,041 (52%)	3,755 (44%)	1,406 (41%)
Female	12,137 (48%)	4,873 (56%)	2,054 (59%)
eth			
Mexican American	4,102 (16%)	1,448 (17%)	564 (16%)
Other Hispanic	2,101 (8.3%)	917 (11%)	437 (13%)
White	10,955 (44%)	3,509 (41%)	1,348 (39%)
Black	5,304 (21%)	1,870 (22%)	828 (24%)
Others	2,716 (11%)	884 (10%)	283 (8.2%)
smoking			
Never	14,617 (58%)	4,546 (53%)	1,491 (43%)
Previous	5,978 (24%)	1,894 (22%)	786 (23%)
Current	4,396 (18%)	2,090 (25%)	1,164 (34%)
Unknown	187	98	19
chol	4.89 (4.22, 5.61)	4.89 (4.22, 5.64)	4.91 (4.22, 5.69)
Unknown	1,251	394	185

[1] Median (Q1, Q3); n (%)

Notice the message above the table: `4499 missing rows in the "phq.c" column have been removed.` That's `gtsummary` telling us something important, not complaining. It isn't an error, and the table is fine.

What it means is this. We asked for the table split three ways, by `phq.c`. A participant with no depression classification recorded can't go in any of the three columns, because we don't know which one they belong in. So `tbl_summary()` left them out, and told us how many: 4,499 of the 41,765 participants in the dataset. The table describes the 37,266 who remain.

This is the same idea as the removed-rows warnings above the plots, arriving in a different form. There, missing values couldn't be placed on an axis; here, they can't be placed in a column. Either way the software drops them and says so, and either way the number is the thing to look at rather than the message.

It matters here more than usual, because this is the table the writing exercise asks you to describe. Describing it as though it covered everybody would overstate what it shows by 4,499 people. Whenever you report a table like this, say what it's based on.

Exercise 6

```
smoking_edu <- table(nhanes$smoking, nhanes$edu)
smoking_edu
```

```
           Less than high school High school graduate Above high school
  Never                     5164                 4656             12778
  Previous                  2405                 2221              4811
  Current                   2707                 2504              3284
```

```r
prop.table(smoking_edu, margin = 1)
```

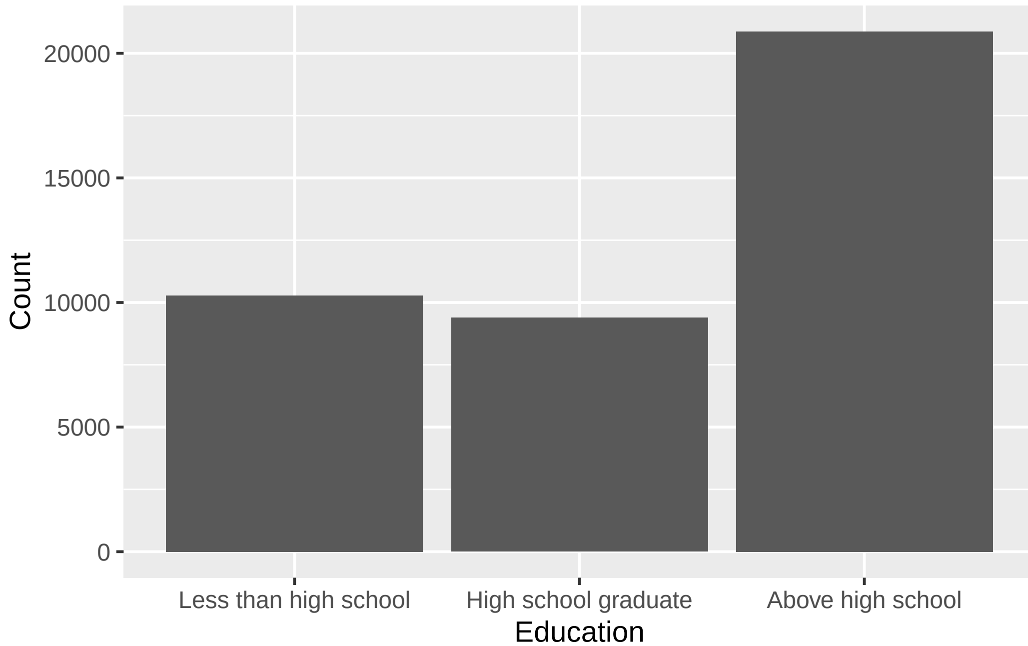
```
           Less than high school High school graduate Above high school
  Never                0.2285158            0.2060359         0.5654483
  Previous             0.2548479            0.2353502         0.5098018
  Current              0.3186580            0.2947616         0.3865803
```

```r
prop.table(smoking_edu, margin = 2)
```

```
           Less than high school High school graduate Above high school
  Never                0.5025302            0.4963224         0.6121784
  Previous             0.2340405            0.2367551         0.2304891
  Current              0.2634293            0.2669225         0.1573324
```

The row percentages are the ones that answer the question. `smoking` was given to `table()` first, so smoking is the rows, and `margin = 1` divides each row by its own total. The "Never" row then reads as the education breakdown *within* people who never smoked, which is exactly what was asked, and those three numbers add to 100%.
The column percentages answer a different question: within each education group, what share smoke, used to smoke, and never smoked. That's a perfectly good question, just not this one. Both tables come from the same counts, so neither is more correct than the other in the abstract. What decides it is which total you want the percentages to be a share of, and that comes from the question, not from the data.

```r
ggplot(nhanes |> filter(!is.na(edu)), aes(x = edu)) +
  geom_bar() +
  labs(x = "Education", y = "Count")
```



Writing exercise (sample answer)
Depression classification was recorded for 37,266 of the 41,765 participants; the remaining 4,499 had no classification recorded and are not included in this table. Among those classified, age, gender, and ethnicity distributions were broadly similar across the three groups. Current smoking was more common among participants classified with possible depression than among those classified as normal. Mean total cholesterol was similar across all three groups. These are descriptive comparisons only, based on group summaries and counts, not any formal test of difference. Hypothesis tests come in IES Week 9.

4 Estimation, precision, and study size (IES Week 8)

This practical follows the IES Week 8 seminar of the same name. That seminar introduced sampling variability through an interactive demonstration, stated the central limit theorem as a one-sentence summary of what that demonstration showed, and drew a sharp line between the standard deviation and the standard error. The SD describes the spread of the *data*. The SE describes the precision of an *estimate*. In this session we put those ideas to work. We'll calculate a standard error, build a confidence interval from it, check the calculation against R's own functions, and then use the same logic in reverse to size a study before it is run.

4.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Calculate a standard error and construct a confidence interval by formula for a mean and for a proportion, and reproduce each using `t.test()` and `prop.test()`
2. Obtain confidence intervals for a difference in means and a difference in proportions
3. Interpret each interval in the units and context of the problem, in writing
4. Describe how an interval's width responds to sample size
5. Obtain a sample size for a two-group comparison using `power.prop.test()` and `power.t.test()`, and describe how the requirement changes with the assumed effect size

4.2 Setup

As in IES Week 7, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary. This chapter also brings in a second dataset for the first time: `strep_tb`, from the `medicaldata` package. Unlike `nhanes_mph`, `strep_tb` doesn't live in a CSV file. It ships built into the package itself, so `library(medicaldata)` is enough to make it available. There is no `read_csv()` step for it. We'll explain what `strep_tb` contains properly below, the first time we use it.

`medicaldata` is the first package this book uses that doesn't come with the tidyverse, so it almost certainly isn't on your machine yet. As IES Week 4 explained, installing a package and loading it are two different actions, and the data living inside the package doesn't change that: you still have to install the package before you can load it. Run `install.packages("medicaldata")` once in the console, not inside this document. Once it's installed, `library(medicaldata)` below works for this session, exactly like every other package we load.

```
library(readr)
library(dplyr)
library(gtsummary)
library(here)
library(medicaldata)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))
dictionary <- read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
```

Two small things about that chunk, worth noticing once. There's no `library(ggplot2)`, because this chapter draws no plots and there's no reason to load a package you aren't going to use. And `dictionary` is read in even though no code below refers to it, because it's the thing you reach for the moment you're unsure what a column means. Every chapter reads it for that reason, not because the chapter's own code needs it.

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c("Male", "Female")),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c("Mexican American", "Other Hispanic", "White", "Black", "Others")),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c("Less than high school", "High school graduate", "Above high school")),
    smoking = factor(smoking,
      levels = c("Never", "Previous", "Current")),
    phq.c = factor(phq.c,
      levels = c("0 Normal", "1 Mild symptoms", "2 Possible depression"),
      labels = c("Normal", "Mild symptoms", "Possible depression"),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c("No", "Yes"))
  )
```

This is the same recode we introduced in IES Week 7. See IES Week 7 for the reasoning behind each variable and each choice of `ordered = TRUE`. We repeat it here, unchanged, so this chapter stands on its own.

4.3 Point estimate and standard error, by hand

A point estimate on its own says nothing about how precise it is. The standard error attaches a precision to it, and we calculate it from three quantities already familiar from IES Week 7: the standard deviation, `sd()`, and the sample size, `n()`. Let's apply this to sedentary time (`sed`):

```
sed_stats <- nhanes |>
  filter(!is.na(sed)) |>
  summarise(
    mean_sed = mean(sed),
    sd_sed = sd(sed),
    n_sed = n()
  ) |>
  mutate(se_sed = sd_sed / sqrt(n_sed))

sed_stats
```

```
# A tibble: 1 x 4
  mean_sed sd_sed n_sed se_sed
     <dbl>  <dbl> <int>  <dbl>
1     6.56   10.9 34434 0.0588
```

`sqrt()` is base R's square root function. The standard error of a mean is the standard deviation divided by the square root of the sample size. As `n` grows, the denominator grows and the standard error shrinks. That's the algebra behind IES Week 8's precision argument.

`sed` was shown in IES Week 7 to be right-skewed, with a long tail of participants reporting many sedentary hours. That skew doesn't disqualify it from a confidence interval on its *mean*. The seminar's interactive demonstration made the point with dice: the value of a single roll is flat, every face equally likely and nothing bell-shaped about it, yet the *average* of 25 rolls piles up into a bell anyway. The shape of the thing you're averaging and the shape of the average are two different questions.

That's what licenses what we just did. `sed` is skewed where the die was flat, but both are a long way from normal, and neither has to be normal for the interval on the mean to behave. What does the work is the sample size, and with 34,434 participants behind it, `sed` has far more of it than 25 dice.

4.4 From standard error to a confidence interval

A 95% confidence interval for a mean is the point estimate plus or minus 1.96 standard errors. 1.96 is the value that cuts off the middle 95% of a normal distribution:

```
sed_stats <- sed_stats |>
  mutate(
    ci_lower = mean_sed - 1.96 * se_sed,
    ci_upper = mean_sed + 1.96 * se_sed
  )

sed_stats
```

```
# A tibble: 1 x 6
  mean_sed sd_sed n_sed se_sed ci_lower ci_upper
     <dbl>  <dbl> <int>  <dbl>    <dbl>    <dbl>
1     6.56   10.9 34434 0.0588     6.45     6.68
```

R has a function that does this directly. `t.test()` expects a single numeric vector, not a data frame, so we pull `sed` out of `nhanes` with `$`. `nhanes$sed` reads as "the `sed` column, inside `nhanes`", and it returns that column on its own, as a plain vector of numbers, rather than a data frame containing it. That's different from `select(sed)`, which keeps a one-column data frame instead of pulling the column out. `t.test()` then returns the mean and its 95% confidence interval without any of the arithmetic above:

```
t.test(nhanes$sed)
```

```

	One Sample t-test

data:  nhanes$sed
t = 111.73, df = 34433, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 6.449124 6.679429
sample estimates:
mean of x
 6.564277
```

`t.test()` also reports a test statistic, degrees of freedom, and a p-value. These belong to a hypothesis test, which is IES Week 9's subject. For now, look only at the confidence interval, and check that it closely matches the interval we calculated by hand above.

One thing `t.test()` does quietly is worth making loud. We passed it `nhanes$sed` whole, missing values and all, with no `filter()` and no `na.rm = TRUE`, and it ran anyway. It drops missing values itself, before calculating. The evidence is in the output: the degrees of freedom are 34,433, one less than the 34,434 participants we filtered down to by hand, which is why the two intervals agree at all. That's a sensible default, but it's the same silent drop the histogram warnings in IES Week 5 and IES Week 7 made noisy, this time with nothing printed to tell you it happened. So count the missing values yourself before you report a result from `t.test()`, the way we did with `is.na()` and `sum()` in IES Week 5. The function won't do it for you.

The two intervals are close but not identical, because `t.test()` doesn't use a fixed multiplier of 1.96. It uses a value from the t-distribution, which depends on the sample size through its degrees of freedom (`n - 1`), and which is very slightly wider than 1.96 for any finite sample. `qt()`, from base R, looks up that exact multiplier:

```
qt(0.975, df = sed_stats$n_sed - 1)
```

```
[1] 1.960033
```

`qt(0.975, df = ...)` asks for the value below which 97.5% of the t-distribution lies. Combined with the 2.5% in the opposite tail, this brackets the middle 95%, matching the interval `t.test()` reports. With a sample this large, the t-multiplier and 1.96 agree to several decimal places. With a small sample, the difference would matter more.

4.4.1 Saying what the interval means, and what it doesn't

We now have an interval. Writing it down correctly is a separate skill from calculating it, and it's the one the seminar spent time on, contrasting the right sentence with the wrong one. Here is the right one, for the interval we just built:

> Mean sedentary time was 6.56 hours/day (95% CI: 6.45 to 6.68 hours/day).

And here is the wrong one, which is the single most common error in this part of the course:

> There is a 95% probability that the true mean sedentary time lies between 6.45 and 6.68 hours/day.

The difference matters, and it's worth being clear about why. The true mean sedentary time in the population is a fixed number. We don't know it, but it isn't moving around, and it doesn't have a 95% chance of being anywhere. What moves is the *interval*. Draw a different sample and you get a different mean, a different standard error, and so a different pair of numbers. The 95% belongs to that procedure: of all the intervals we could build this way, 95% of them would contain the true value. This particular interval either contains it or it doesn't, and we have no way of knowing which.

In practice the fix is easy, because the correct sentence is the shorter one. State the estimate, put the interval after it, and stop. The moment you write "probability", "chance", or "there's a 95% likelihood the true value is", you have said something about the parameter that the interval doesn't support. Every interval in the rest of this book gets reported the first way, and so should yours.

4.5 A confidence interval for a proportion

`strep_tb`, from the `medicaldata` package, records the 1948 MRC streptomycin trial: a randomised controlled trial of streptomycin against tuberculosis in 107 patients, generally considered the first modern randomised controlled trial. Each patient was assigned to the `Streptomycin` or `Control` arm and followed up. Patients in the control arm received bed rest alone rather than a placebo, since streptomycin was in very short supply in post-war Britain and injecting a dummy drug was never part of the design. `improved` records whether their chest X-ray had improved at six months. As with any new dataset, it's worth looking at what actually came in before writing code against it:

```r
glimpse(strep_tb)
```

```
Rows: 107
Columns: 13
$ patient_id          <chr> "0001", "0002", "0003", "0004", "0005", "0006"~
$ arm                 <fct> Control, Control, Control, Control, Control, C~
$ dose_strep_g        <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ dose_PAS_g          <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ gender              <fct> M, F, F, M, F, M, F, M, F, M, F, M, F, M, F, M~
$ baseline_condition  <fct> 1_Good, 1_Good, 1_Good, 1_Good, 1_Good, 1_Good~
$ baseline_temp       <fct> 1_98-98.9F, 3_100-100.9F, 1_98-98.9F, 1_98-98.~
$ baseline_esr        <fct> 2_11-20, 2_11-20, 3_21-50, 3_21-50, 3_21-50, 3~
$ baseline_cavitation <fct> yes, no, no, no, no, no, yes, yes, yes, yes, n~
$ strep_resistance    <fct> 1_sens_0-8, 1_sens_0-8, 1_sens_0-8, 1_sens_0-8~
$ radiologic_6m       <fct> 6_Considerable_improvement, 5_Moderate_improve~
$ rad_num             <dbl> 6, 5, 5, 5, 5, 6, 5, 5, 5, 5, 6, 5, 5, 5, 5, 6~
$ improved            <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE~
```

`arm` and `gender` are already factors. `improved`, the outcome we care about here, is **logical**: `TRUE` or `FALSE`, rather than a factor. R stores `TRUE` and `FALSE` as 1 and 0, which will matter again shortly. `table()`, already met in IES Week 7, counts how many patients fall into each category of `improved`, and `prop.table()` converts those counts to proportions:

```
table(strep_tb$improved)
```

```
FALSE  TRUE
   52    55
```

```
prop.table(table(strep_tb$improved))
```

```
    FALSE      TRUE
0.4859813 0.5140187
```

4.5.1 The standard error of a proportion, by hand

A proportion is an estimate like any other, so it has a standard error too. The formula is different from the one for a mean, because a proportion carries its own variability: once you know p, you know how spread out a yes/no variable is, so there's no separate SD to measure. The standard error of a proportion is the square root of $p \times (1 - p) \div n$.

`sum()` of a logical vector counts how many values are `TRUE`, since R stores `TRUE` as 1 and `FALSE` as 0. So `sum(strep_tb$improved)` is the number of patients who improved, out of `nrow(strep_tb)` in total. `nrow()` was met briefly in IES Week 5, returning just the row count on its own rather than rows and columns together the way `dim()` does:

```
n_improved <- sum(strep_tb$improved)
n_total <- nrow(strep_tb)

p_improved <- n_improved / n_total
se_improved <- sqrt(p_improved * (1 - p_improved) / n_total)

p_improved
```

```
[1] 0.5140187
```

```r
se_improved
```

```
[1] 0.04831782
```

The confidence interval is then built exactly the way it was for a mean: the estimate, plus or minus 1.96 standard errors.

```r
p_improved - 1.96 * se_improved
```

```
[1] 0.4193158
```

```r
p_improved + 1.96 * se_improved
```

```
[1] 0.6087216
```

4.5.2 The same interval from `prop.test()`

`prop.test()` returns a confidence interval for a proportion directly, given a count of successes (`x`) and a total (`n`), with none of the arithmetic above:

```r
prop.test(x = n_improved, n = n_total)
```

```

	1-sample proportions test with continuity correction

data:  n_improved out of n_total, null probability 0.5
X-squared = 0.037383, df = 1, p-value = 0.8467
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.4159553 0.6110633
sample estimates:
        p 
0.5140187 
```

The two intervals are close but not identical, and for the same kind of reason the hand-built and `t.test()` intervals differed earlier. The $\pm$ 1.96 version above is the textbook approximation, and it works well when n is large and p isn't near 0 or 1. `prop.test()` uses a better method that behaves more sensibly in small samples and never runs past 0 or 1, which the approximation happily will if you push it. With 107 patients and a proportion near a half, both are perfectly reasonable and they agree to within about half a percentage point. Report the one from `prop.test()`; the hand calculation is here so the formula is a real thing you have used, not a line on a slide.

Giving `prop.test()` the count and total directly, rather than a cross-tabulated table, avoids any doubt about which of the two categories we're counting. A table of `improved` alone would still show both proportions clearly, but a table fed straight into `prop.test()` reports the proportion for only whichever category R lists first, which is not necessarily the one we want.

By default, `prop.test()` also tests whether this proportion differs from 50%. That default comparison isn't of interest here, and it isn't the subject of this chapter. The 95% confidence interval it reports alongside is what we want.

4.6 Confidence interval for a difference in means

`t.test()` also compares two groups directly, using **formula syntax**. `chol ~ gender` reads as "`chol`, modelled by `gender`". R splits `chol` into the groups defined by `gender` and compares them:

```r
t.test(chol ~ gender, data = nhanes |> filter(!is.na(gender)))
```

```
    Welch Two Sample t-test

data:  chol by gender
t = -15.318, df = 38908, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Male and group Female is not
95 percent confidence interval:
 -0.1901172 -0.1469821
sample estimates:
  mean in group Male mean in group Female
            4.885939             5.054489
```

Notice there's no `$` anywhere in that call, even though `chol` and `gender` both live inside `nhanes`. The `data =` argument tells `t.test()` which data frame to search for bare names like `chol` and `gender`, so it looks them up there itself instead of us pulling each one out by hand first with `$`. `$` is still the right tool for getting a single column on its own, the way `nhanes$sed` was above; a formula's `data =` argument does that lookup for every variable named in the formula at once.

This is the same pairing we showed descriptively in IES Week 7's boxplot and Table 1: cholesterol by gender. Now it's attached to a confidence interval for the *difference* in means between the two groups. By default, `t.test()` doesn't assume the two groups have equal variance. That's a sensible default and not something we need to adjust here.

4.6.1 Which way round is the difference?

Both ends of that interval are negative, and a negative difference in cholesterol isn't obviously meaningful until we know what has been subtracted from what. R tells us, on the line beginning "alternative hypothesis": *true difference in means between group Male and group Female*. So it has taken the men's mean and subtracted the women's.

That order isn't R's choice, and it isn't alphabetical. It's the order the factor's levels are in, which we set ourselves in the setup chunk:

```r
levels(nhanes$gender)
```

```
[1] "Male"   "Female"
```

`"Male"` is first, so men are group 1 and women are group 2, and the difference runs first minus second. A negative interval therefore says the first group is *lower*: mean cholesterol is lower in men than in women, by somewhere between 0.15 and 0.19 mmol/L.

Two habits follow from this, and they save a lot of confusion later. Check the alternative-hypothesis line before reading any two-group interval, every time. And if you'd rather write the comparison the other way round, which is often more natural to read, flip the whole thing together: "cholesterol was 0.17 mmol/L higher in women than in men (95% CI: 0.15 to 0.19)" says exactly the same as R's output with every sign reversed. What you must never do is flip the words and leave the numbers, or flip one end of the interval and not the other. The direction of the estimate and the direction of the interval have to agree.

4.7 Confidence interval for a difference in proportions

The same idea extends to two proportions: how many improved in each trial arm. `group_by()` and `summarise()`, already familiar from IES Week 7, give us the count and the number improved in each arm:

```r
strep_arm_summary <- strep_tb |>
  group_by(arm) |>
  summarise(
    n_arm = n(),
    n_improved = sum(improved),
    prop_improved = n_improved / n_arm
  )

strep_arm_summary
```

```
# A tibble: 2 x 4
  arm          n_arm n_improved prop_improved
  <fct>        <int>      <int>         <dbl>
1 Streptomycin    55         38         0.691
2 Control         52         17         0.327
```

`prop_improved` here is exactly what `mean(improved)` would give directly. `mean()` of a logical vector is a shortcut worth knowing: R treats `TRUE` as 1 and `FALSE` as 0, so the mean of a column of `TRUE`/`FALSE` values is the proportion that are `TRUE`.

If we pass the two counts and the two totals to `prop.test()`, rather than a cross-tabulated table, it compares the two proportions unambiguously. `prop 1` and `prop 2` below correspond, in order, to the two rows of `strep_arm_summary` above:

```r
prop.test(x = strep_arm_summary$n_improved, n = strep_arm_summary$n_arm)
```

```

	2-sample test for equality of proportions with continuity
	correction

data:  strep_arm_summary$n_improved out of strep_arm_summary$n_arm
X-squared = 12.756, df = 1, p-value = 0.0003548
alternative hypothesis: two.sided
95 percent confidence interval:
 0.168726 0.559246
sample estimates:
   prop 1    prop 2 
0.6909091 0.3269231 
```

The confidence interval is for the difference in the proportion improved between the two arms. Before reading it, check the direction, exactly as we did for the difference in means above. The same rule applies: R takes the first group and subtracts the second, and the order comes from the factor's levels.

```
levels(strep_tb$arm)
```

```
[1] "Streptomycin" "Control"
```

`"Streptomycin"` is first, so `prop 1` is the streptomycin arm and `prop 2` is the control arm, and the difference runs streptomycin minus control. Both ends of the interval are positive, which therefore says the streptomycin arm improved *more*. Had the two levels been the other way round, every number in that interval would have carried a minus sign and meant exactly the same thing. IES Week 9 puts this same factor the other way round on purpose, so it's worth having looked at the levels here first.

Two more things in that output are worth naming rather than skipping. The heading says "with continuity correction": `prop.test()` applies a small adjustment by default that nudges the p-value up and the interval slightly wider. IES Week 9 explains what it does and when it earns its place, so leave it for now, but notice that the p-value printed here, 0.0004, is the one that adjustment produces. And the p-value itself belongs to a hypothesis test, which is still IES Week 9's subject. Read the interval; the rest keeps.

Because `strep_tb` is a randomised trial rather than an observational study, a difference this large and this precisely estimated supports a causal reading in a way that an association in `nhanes_mph` would not. Randomisation, not statistical adjustment, is what licenses that conclusion here.

4.8 `gtsummary::tbl_summary()` as shorthand

`tbl_summary()`, met in IES Week 7, can be told which summary to use for a given variable with the `statistic` argument, which takes a `list()`. This is a way of bundling several named settings together, pairing a variable with the display it should use.

The display itself is written as a piece of text with curly braces in it: `"{mean} ({sd})"`. The braces are placeholders. `gtsummary` works out the mean and the standard deviation, then drops each number into the spot where its name appears, and everything outside the braces, here a space and a pair of round brackets, is printed exactly as written. So `"{mean} ({sd})"` comes out as something like `5.05 (1.08)`. Writing the layout out this way means you decide how the cell reads, rather than accepting a default.

```
nhanes |>
  select(chol, gender) |>
  tbl_summary(by = gender, statistic = list(chol ~ "{mean} ({sd})"))
```

Table 4.1

Characteristic	Male N = 20,085	Female N = 21,680
chol	4.89 (1.09)	5.05 (1.08)
Unknown	1,218	1,371

[1] Mean (SD)

This forces mean (SD) display for `chol`, overriding whatever `tbl_summary()` would otherwise have chosen automatically.

4.9 Width and sample size, without programming

The seminar's rule was that a confidence interval narrows as sample size grows. We can show this directly, without simulating anything, by computing the same interval on the full sample and on two smaller subsets of it. `slice_sample()`, from `dplyr`, keeps a random fraction of the rows. `set.seed()`, from base R, fixes the sequence of "random" numbers R uses, so the same subset gets drawn every time this code is run, rather than a different one each time.

The full sample. This time we save the result into an object with `<-` as well as printing it, because we want to reuse its numbers in a moment:

```r
tt_full <- t.test(nhanes$chol)
tt_full
```

```

	One Sample t-test

data:  nhanes$chol
t = 902.16, df = 39175, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.962511 4.984121
sample estimates:
mean of x
 4.973316
```

A 50% subset:

```
set.seed(1)
chol_50 <- nhanes |>
  filter(!is.na(chol)) |>
  slice_sample(prop = 0.5)

tt_50 <- t.test(chol_50$chol)
tt_50
```

```

	One Sample t-test

data:  chol_50$chol
t = 637.89, df = 19587, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.955667 4.986217
sample estimates:
mean of x
 4.970942
```

A 10% subset:

```
set.seed(1)
chol_10 <- nhanes |>
  filter(!is.na(chol)) |>
  slice_sample(prop = 0.1)

tt_10 <- t.test(chol_10$chol)
tt_10
```

```

	One Sample t-test

data:  chol_10$chol
t = 289.21, df = 3916, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.933782 5.001132
sample estimates:
mean of x
 4.967457
```

The point estimates are similar across all three, because all three are samples from the same population. But the confidence interval widens as the sample size drops.

That's hard to see across three blocks of output, so let's put the three intervals in one small table. A `t.test()` result isn't a data frame, but `$` still works on it, pulling out one named piece at a time exactly as it pulls one column out of a data frame. `tt_full$conf.int` is the pair of numbers at the ends of the interval, and `[1]` and `[2]` take the lower and the upper one, the same square-bracket indexing we used in IES Week 5. `data.frame()`, from IES Week 4, builds the table out of them, and `mutate()` adds the column we actually care about:

```
widths <- data.frame(
  sample = c("Full", "50% subset", "10% subset"),
  n = c(sum(!is.na(nhanes$chol)), nrow(chol_50), nrow(chol_10)),
  lower = c(tt_full$conf.int[1], tt_50$conf.int[1], tt_10$conf.int[1]),
  upper = c(tt_full$conf.int[2], tt_50$conf.int[2], tt_10$conf.int[2])
) |>
  mutate(width = upper - lower)

widths
```

```
      sample     n    lower    upper      width
1       Full 39176 4.962511 4.984121 0.02161000
2 50% subset 19588 4.955667 4.986217 0.03054926
3 10% subset  3917 4.933782 5.001132 0.06734956
```

Now the pattern is one column instead of six numbers on three screens. Cut the sample in half and the interval gets about 40% wider; cut it to a tenth and it's roughly three times wider than we started. That last number is the useful one to hold onto. Ten times less data did not make the interval ten times wider, it made it about three times wider, and three is roughly the square root of ten. That's the SE = $s/\sqrt{n}$ formula showing up in an actual result: precision improves with the square root of the sample size, not with the sample size itself, which is exactly why doubling a study's size buys much less than it feels like it should.

4.10 Sample size and power

The same logic runs in reverse. Instead of asking how precise an estimate from a given sample size will be, we can size a study in advance to detect a given effect. `strep_tb` supplies the natural example, since it's a trial with a binary outcome, and `strep_arm_summary` above already has the proportion improved in each arm: roughly 33% in the control arm and 69% in the streptomycin arm.

`power.prop.test()`, from base R, answers a practical question: how large would a two-arm trial need to be to reliably detect a gap this size? We give it the two proportions, a significance level, and a target power:

```r
power.prop.test(p1 = 0.33, p2 = 0.69, sig.level = 0.05, power = 0.8)
```

```

     Two-sample comparison of proportions power calculation

              n = 29.0615
             p1 = 0.33
             p2 = 0.69
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

`sig.level = 0.05` is the Type I error rate the seminar introduced. `power = 0.8` is the conventional target: an 80% chance of detecting the effect if it's really there. `n` is the number **per group** required. The gap observed in the original trial was unusually large. A smaller assumed gap needs a correspondingly larger trial:

```r
power.prop.test(p1 = 0.33, p2 = 0.45, sig.level = 0.05, power = 0.8)
```

```

     Two-sample comparison of proportions power calculation

              n = 258.158
             p1 = 0.33
             p2 = 0.45
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

The same function, `power.t.test()`, sizes a study around a continuous outcome instead, using the variability of `chol`:

```r
chol_sd <- nhanes |>
  filter(!is.na(chol)) |>
  summarise(sd_chol = sd(chol)) |>
  pull(sd_chol)

power.t.test(delta = 0.5, sd = chol_sd, sig.level = 0.05, power = 0.8)
```

```

     Two-sample t test power calculation

              n = 75.72811
          delta = 0.5
             sd = 1.091122
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

`pull()`, from `dplyr`, extracts a single column as a plain vector rather than a one-column tibble. We need this because `power.t.test()` expects a single number for `sd`, not a table containing one. `delta` is the assumed difference in means the study is being sized to detect. As with the proportions above, a smaller assumed difference requires a larger sample:

```r
power.t.test(delta = 0.2, sd = chol_sd, sig.level = 0.05, power = 0.8)
```

```

     Two-sample t test power calculation

              n = 468.1853
          delta = 0.2
             sd = 1.091122
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Both functions size a study *before* it is run, from an assumed effect. Power calculated after the fact, from a result already in hand, only re-expresses the p-value already obtained and adds nothing. The seminar's point about post-hoc power applies to both `power.prop.test()` and `power.t.test()` equally.

> **Try it yourself: assumed effect size**
>
> Re-run `power.prop.test()` with `p2` set closer and closer to `p1` (for example 0.40, then 0.35), keeping `power = 0.8`. Watch what happens to `n`. Write one sentence on what this means for a researcher planning a trial to detect a genuinely small effect.

4.11 Worked example: a confidence interval for age

Let's bring the tools above together, on a variable we haven't used elsewhere in this chapter: `age`.

```
age_stats <- nhanes |>
  filter(!is.na(age)) |>
  summarise(
    mean_age = mean(age),
    sd_age = sd(age),
    n_age = n()
  ) |>
  mutate(
    se_age = sd_age / sqrt(n_age),
    ci_lower = mean_age - 1.96 * se_age,
    ci_upper = mean_age + 1.96 * se_age
  )

age_stats
```

```
# A tibble: 1 x 6
  mean_age sd_age n_age se_age ci_lower ci_upper
     <dbl>  <dbl> <int>  <dbl>    <dbl>    <dbl>
1     47.5   18.5 41765 0.0905     47.3     47.7
```

```
t.test(nhanes$age)
```

```


	One Sample t-test

data:  nhanes$age
```

```
t = 524.82, df = 41764, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 47.33953 47.69445
sample estimates:
mean of x
 47.51699
```

The hand-calculated interval and the one from `t.test()` agree closely. Written as it would appear in a results section: the mean age of participants was 47.5 years (95% CI: 47.3 to 47.7 years). This is a statement about the precision of the *estimate* of the mean, not a description of how spread out individual ages are. That's what the SD, not the SE, is for.

4.12 Exercises

Exercise 1 (ILOs 1, 2, 3). Calculate the mean and standard error of total cholesterol (`chol`), construct a 95% confidence interval by hand, and confirm it with `t.test()`. Write one sentence reporting the result as it would appear in a results section.

Then use `t.test()` with formula syntax to obtain a 95% confidence interval for the *difference* in mean cholesterol between smoking groups, comparing current smokers with those who have never smoked. You'll need `filter()` to restrict `smoking` to those two categories first, since `t.test()` compares exactly two groups. Write one sentence interpreting that interval, and one sentence saying how it differs in meaning from the interval you built in the first half of this exercise.

Exercise 2 (ILOs 1, 2, 3). In `strep_tb`, find the proportion of patients with baseline cavitation present (`baseline_cavitation`) and calculate its standard error by hand, then build a 95% confidence interval from it and check the result against `prop.test()`. Then construct a 95% confidence interval for the difference in the proportion improved between men and women (`gender`). Write one interpretive sentence for each interval.

Exercise 3 (ILOs 3, 4). Repeat the width exploration above (full sample, 50% subset, 10% subset) using `sed` instead of `chol`, and collect the three intervals into one table the way the chapter did, so their widths sit side by side. Describe what changes and what stays about the same as the sample size drops.

Exercise 4 (ILOs 3, 5). Using `power.t.test()`, find the sample size needed to detect a difference in `chol` of 0.3 mmol/L, and again for a difference of 0.8 mmol/L, both at `power = 0.8`. Write one sentence explaining why the two required sample sizes differ so much.

4.13 Writing exercise

Using the cholesterol confidence interval from Exercise 1 and the `strep_tb` arm-improvement confidence interval from earlier in this chapter, write one sentence for each, interpreting it as it would appear in the results section of a report. Use the correct units, and don't describe the interval as a probability statement about where the true value falls.

Solutions

Exercise 1

```
chol_stats <- nhanes |>
  filter(!is.na(chol)) |>
  summarise(
    mean_chol = mean(chol),
    sd_chol = sd(chol),
    n_chol = n()
  ) |>
  mutate(
    se_chol = sd_chol / sqrt(n_chol),
    ci_lower = mean_chol - 1.96 * se_chol,
    ci_upper = mean_chol + 1.96 * se_chol
  )

chol_stats
```

```
# A tibble: 1 x 6
  mean_chol sd_chol n_chol se_chol ci_lower ci_upper
      <dbl>   <dbl>  <int>   <dbl>    <dbl>    <dbl>
1      4.97    1.09  39176 0.00551     4.96     4.98
```

```
t.test(nhanes$chol)
```

```

	One Sample t-test

data:  nhanes$chol
t = 902.16, df = 39175, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 4.962511 4.984121
sample estimates:
```

```
mean of x
 4.973316
```

Mean total cholesterol in this sample was 4.97 mmol/L (95% CI: 4.96 to 4.98 mmol/L). For the difference between the two smoking groups:

```r
chol_smoking <- nhanes |>
  filter(smoking %in% c("Never", "Current"))

t.test(chol ~ smoking, data = chol_smoking)
```

```

	Welch Two Sample t-test

data:  chol by smoking
t = -1.9231, df = 13580, p-value = 0.05449
alternative hypothesis: true difference in means between group Never and group Current is
95 percent confidence interval:
 -0.0561225720  0.0005350405
sample estimates:
  mean in group Never mean in group Current
             4.967995              4.995789
```

`smoking` still has three levels after the `filter()`, but only two of them have any rows left, and `t.test()` rebuilds the factor from the data it's actually given, so the empty third group drops out on its own.

Check the direction before reading the interval, as above. `"Never"` comes first in `smoking`'s levels, so R has subtracted current smokers from never smokers, and the difference is never minus current.

Mean total cholesterol was -0.028 mmol/L in never smokers compared with current smokers (95% CI: -0.056 to 0.001 mmol/L), so a little lower on average in those who had never smoked. Notice that the estimate sits inside its own interval, which is the check worth running on any result you write down: if it doesn't, a sign has been flipped somewhere. The interval includes zero, so these data are consistent with no difference between the two groups, though they don't rule out a small one either.

The two intervals mean different things. The first is about a single quantity, mean cholesterol, and says how precisely this sample pins it down. The second is about a *comparison* between two groups, and the value that matters in it is zero: an interval containing zero is one where the data can't establish which group is higher. From IES Week 9 onward, almost every interval in this course is of the second kind.

Exercise 2

```r
n_cav <- sum(strep_tb$baseline_cavitation == "yes")
p_cav <- n_cav / nrow(strep_tb)
se_cav <- sqrt(p_cav * (1 - p_cav) / nrow(strep_tb))

p_cav
```

```
[1] 0.5794393
```

```r
se_cav
```

```
[1] 0.04772286
```

```r
p_cav - 1.96 * se_cav
```

```
[1] 0.4859025
```

```r
p_cav + 1.96 * se_cav
```

```
[1] 0.6729761
```

```r
prop.test(x = n_cav, n = nrow(strep_tb))
```

```

    1-sample proportions test with continuity correction

data:  n_cav out of nrow(strep_tb), null probability 0.5
X-squared = 2.3925, df = 1, p-value = 0.1219
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
 0.4801030 0.6729991
sample estimates:
        p
0.5794393
```

```r
gender_summary <- strep_tb |>
  group_by(gender) |>
  summarise(n_gender = n(), n_improved = sum(improved))
gender_summary
```

```
# A tibble: 2 x 3
  gender n_gender n_improved
  <fct>     <int>      <int>
1 F            59         28
2 M            48         27
```

```
prop.test(x = gender_summary$n_improved, n = gender_summary$n_gender)
```

```

	2-sample test for equality of proportions with continuity
	correction

data:  gender_summary$n_improved out of gender_summary$n_gender
X-squared = 0.50491, df = 1, p-value = 0.4773
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.2963678  0.1205203
sample estimates:
   prop 1    prop 2 
0.4745763 0.5625000 
```

The hand-built interval and the one from `prop.test()` agree closely, as they did for the proportion improved earlier in the chapter.
Just over half of patients (58%) had cavitation present on their baseline chest X-ray, but the 95% confidence interval includes 50%, so these data don't rule out the true proportion being exactly one half. The confidence interval for the difference in improvement between women (`prop 1`) and men (`prop 2`) includes zero, consistent with little or no difference in outcome by gender in this trial.
Exercise 3

```r
tt_sed_full <- t.test(nhanes$sed)

set.seed(1)
sed_50 <- nhanes |>
  filter(!is.na(sed)) |>
  slice_sample(prop = 0.5)
tt_sed_50 <- t.test(sed_50$sed)

set.seed(1)
sed_10 <- nhanes |>
  filter(!is.na(sed)) |>
  slice_sample(prop = 0.1)
tt_sed_10 <- t.test(sed_10$sed)

data.frame(
  sample = c("Full", "50% subset", "10% subset"),
  n = c(sum(!is.na(nhanes$sed)), nrow(sed_50), nrow(sed_10)),
  lower = c(tt_sed_full$conf.int[1], tt_sed_50$conf.int[1],
            tt_sed_10$conf.int[1]),
  upper = c(tt_sed_full$conf.int[2], tt_sed_50$conf.int[2],
            tt_sed_10$conf.int[2])
) |>
  mutate(width = upper - lower)
```

```
      sample     n    lower    upper     width
1       Full 34434 6.449124 6.679429 0.2303057
2 50% subset 17217 6.403063 6.728992 0.3259286
3 10% subset  3443 6.219105 6.983209 0.7641046
```

As with `chol`, the point estimate is broadly similar across all three samples, but the confidence interval widens substantially in the 10% subset, and by almost exactly the same factor of about three. The same pattern holds for a skewed variable as for a symmetric one, because it's the *sample size*, not the shape of the raw data, that governs the width of the interval on the mean.

Exercise 4

```r
power.t.test(delta = 0.3, sd = chol_stats$sd_chol, sig.level = 0.05, power = 0.8)
```

```
     Two-sample t test power calculation
```

```
              n = 208.6194
          delta = 0.3
             sd = 1.091122
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

```r
power.t.test(delta = 0.8, sd = chol_stats$sd_chol, sig.level = 0.05, power = 0.8)
```

```

     Two-sample t test power calculation

              n = 30.19253
          delta = 0.8
             sd = 1.091122
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Detecting the smaller difference (0.3 mmol/L) needs a much larger sample than detecting the larger difference (0.8 mmol/L). Smaller effects are harder to distinguish from chance variation, so we need more participants to have the same 80% chance of detecting them.

Writing exercise (sample answer)

Mean total cholesterol was 4.97 mmol/L (95% CI: 4.96 to 4.98 mmol/L). In the streptomycin trial, the proportion of patients improved was higher in the streptomycin arm than in the control arm, with a 95% confidence interval for the difference that excludes zero. This is consistent with a real difference in improvement between the two arms, rather than one attributable to chance alone.

5 Hypothesis tests and measures of association (IES Week 9)

This practical follows the IES Week 9 seminar of the same name. That seminar set out the logic of a hypothesis test: stating a null and an alternative hypothesis, turning a test statistic into a p-value, and being precise about what a p-value is and what it is not. It also showed that a confidence interval and a hypothesis test carry the same information, viewed two ways, and it established the reporting standard we use from here on: an effect size, a 95% confidence interval, and a p-value, reported together. Finally, it reactivated the risk ratio and odds ratio from the epidemiology half of the programme, attaching a confidence interval and a p-value to numbers you can already calculate by hand. In this session we put all of that into R.

5.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Perform and interpret an independent-samples t-test in R
2. Perform and interpret a paired t-test, recognise from a data structure which is required, and distinguish paired measurements taken over time from two related measurements taken at once
3. Construct a contingency table and perform a chi-squared test
4. Obtain risk ratios and odds ratios with confidence intervals, and reconcile them with the hand calculation taught in the epidemiology half
5. Write up a result in the style expected in a public health journal

5.2 Setup

As in IES Weeks 7 and 8, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary. We also bring back `strep_tb`, from the `medicaldata` package, for the categorical half of this chapter.

This chapter adds one new package, `epitools`. It isn't part of the tidyverse, and it won't already be installed alongside the packages we've used so far. As IES Week 4 explained, installing a package and loading it are two different actions. If `epitools` isn't already on your machine, run `install.packages("epitools")` once in the console, not inside this document. Once it's installed, `library(epitools)` below makes its functions available for this session, exactly like every other package we load.

```
library(readr)
library(dplyr)
library(ggplot2)
library(gtsummary)
library(here)
library(medicaldata)
library(epitools)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))
dictionary <- read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
```

As in IES Week 8, `dictionary` is read in even though no code below refers to it. It's there so it's to hand the moment you're unsure what a column means.

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c("Male", "Female")),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c("Mexican American", "Other Hispanic", "White", "Black", "Others")),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c("Less than high school", "High school graduate", "Above high school")),
    smoking = factor(smoking,
      levels = c("Never", "Previous", "Current")),
    phq.c = factor(phq.c,
      levels = c("0 Normal", "1 Mild symptoms", "2 Possible depression"),
      labels = c("Normal", "Mild symptoms", "Possible depression"),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c("No", "Yes"))
  )
```

This is the same recode we introduced in IES Week 7 and repeated in IES Week 8. See IES Week 7 for the reasoning behind each variable and each choice of `ordered = TRUE`. We repeat it here, unchanged, so this chapter stands on its own too.

5.3 Independent-samples t-test: cholesterol by gender

`t.test()` compares two independent groups directly. We already used it in IES Week 8 to build a confidence interval for the difference in total cholesterol (`chol`, in mmol/L) between men and women:

```
t.test(chol ~ gender, data = nhanes |> filter(!is.na(gender)))
```

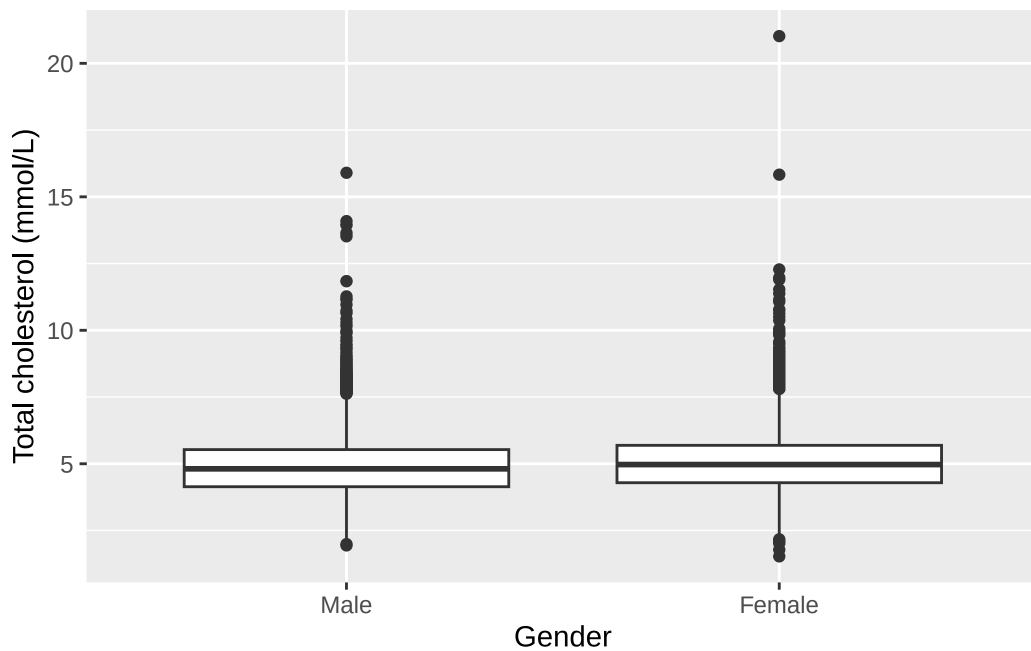
```

	Welch Two Sample t-test

data:  chol by gender
t = -15.318, df = 38908, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Male and group Female is not e
95 percent confidence interval:
 -0.1901172 -0.1469821
sample estimates:
  mean in group Male mean in group Female 
            4.885939             5.054489 
```

This is the exact same call we made in IES Week 8. `t.test()` has been giving us a hypothesis test the whole time. In IES Week 8, hypothesis testing hadn't been introduced yet, so we only looked at the confidence interval and set the rest of the output aside. Now we bring the rest into view: a t-statistic, degrees of freedom, and a p-value.

Before trusting a t-test, it's worth checking that the two groups aren't wildly different in spread or shape, the way IES Week 7's boxplot of `chol` by `gender` already let us do:

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = chol)) +
  geom_boxplot() +
  labs(x = "Gender", y = "Total cholesterol (mmol/L)")
```

```
Warning: Removed 2589 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



The two boxplots are similar in spread and shape, just shifted slightly. Nothing here rules out a t-test. `t.test()` also doesn't assume equal variance between the two groups by default, which protects us even where the spreads differ more than they do here.

Before reading the interval, check the direction, exactly as IES Week 8 set out. The alternative-hypothesis line says *group Male and group Female*, because `"Male"` is the first level of `gender`, so R has printed men minus women and both ends come out negative. Reversing every sign together turns it into the more natural reading: mean total cholesterol was about 0.17 mmol/L higher in women than in men (95% CI: 0.15 to 0.19 mmol/L, $p < 0.001$). The two statements are the same result written from opposite ends. This is the reporting standard the seminar introduced: effect size, confidence interval, and p-value, together, every time, from here on. `p < 0.001` is how we write a p-value that R prints as something like `< 2.2e-16`. Convention reports a very small p-value this way rather than typing out the full number.

5.4 Paired t-test: two domains of activity in one person

`nhanes_mph` records physical activity as two separate totals for each participant, both in MET-hours/day: `met_w`, activity at work, and `met_r`, activity outside work. These are two different domains of activity, measured at the same time, in the same person. They are not the same activity measured twice at two different points in time.

That distinction decides which test applies. An independent-samples t-test, like the one we just ran on `chol` by `gender`, compares two separate groups of people. Here we have one group of people, each measured twice. The right tool is a paired t-test. Rather than comparing the two columns directly, it collapses them into a single column of within-person differences (`met_w` minus `met_r`) and tests whether that difference is, on average, zero.

`t.test()` runs a paired test when we set `paired = TRUE`:

```r
t.test(nhanes$met_w, nhanes$met_r, paired = TRUE)
```

```

	Paired t-test

data:  nhanes$met_w and nhanes$met_r
t = 43.111, df = 34284, p-value < 2.2e-16
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 2.927984 3.206905
sample estimates:
mean difference 
       3.067444 
```

On average, work activity ran about 3.07 MET-hours/day higher than activity outside work (95% CI: 2.93 to 3.21 MET-hours/day, $p < 0.001$).

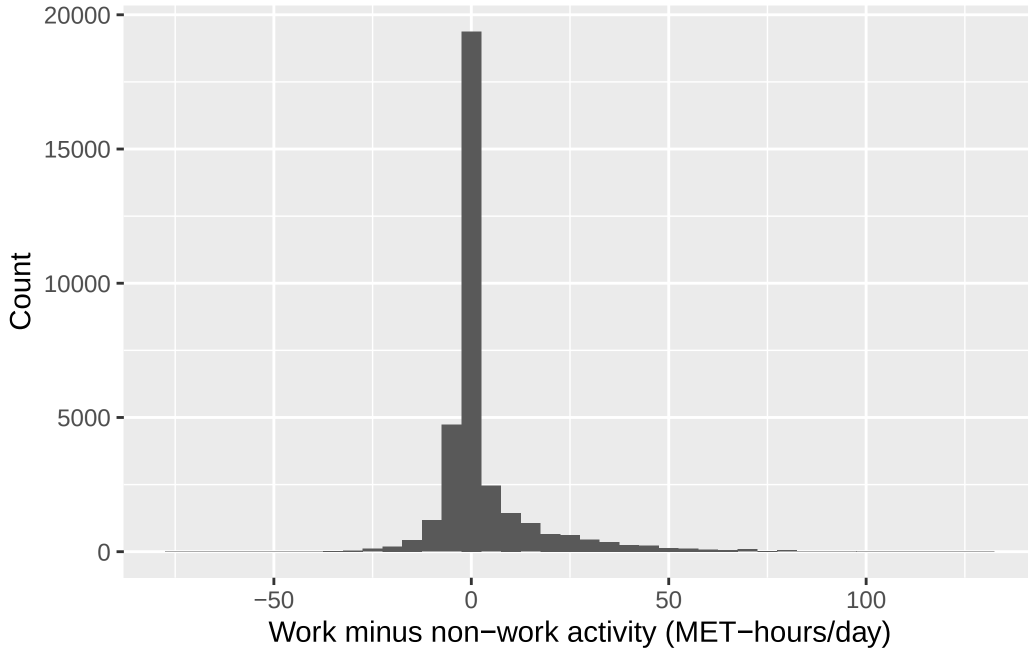
Framing this as two domains of activity within a person, rather than as a change over time, is deliberate. A paired t-test also arises in a before-and-after design, where the same measurement is taken at two time points on the same person. That is a genuinely different study design from the one here, even though the R code looks identical. There is no repeated measurement over time in this data, and describing the result as though there were would misdescribe the study and plant a habit that causes real trouble in a longitudinal design later on.

5.5 A non-parametric alternative: the Wilcoxon signed-rank test

A paired t-test assumes the within-person differences are at least roughly normally distributed. Let's check that assumption directly, the way we checked spread with a boxplot earlier:

```r
nhanes |>
  mutate(met_diff = met_w - met_r) |>
  ggplot(aes(x = met_diff)) +
  geom_histogram(binwidth = 5) +
  labs(x = "Work minus non-work activity (MET-hours/day)", y = "Count")
```

```
Warning: Removed 7480 rows containing non-finite outside the scale range
(`stat_bin()`).
```



The distribution of differences is heavily skewed, and a large share of participants sit exactly at zero. Many people report no occupational activity at all, so `met_w` and `met_r` are equal for them. This is exactly the shape a paired t-test's normality assumption doesn't cope well with.

`wilcox.test()`, from base R, tests the same null hypothesis (no systematic difference within the pair) without assuming a normal distribution. It works with the ranks of the differences rather than their average, so one extreme value or an oddly-shaped distribution can't dominate the result the way it can dominate a mean. With `paired = TRUE`, it runs the paired version, known as the Wilcoxon signed-rank test:

```
wilcox.test(nhanes$met_w, nhanes$met_r, paired = TRUE)
```

```

	Wilcoxon signed rank test with continuity correction

data:  nhanes$met_w and nhanes$met_r
V = 169408552, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
```

The conclusion doesn't change here: both tests reject the null hypothesis convincingly, each with a p-value far below 0.001. With a sample this large, the t-test and the Wilcoxon test agree despite the skew. That won't always be true. With a smaller sample, or a more extreme distribution, the two tests can disagree, and the non-parametric result is usually the one to trust when the normality assumption is in real doubt.

5.6 Chi-squared test: `strep_tb`'s primary result

`strep_tb`'s primary outcome is binary: did chest X-ray improve at six months (`improved`), and does that depend on trial arm (`arm`)? `table()` builds a two-way count, the same way it did in IES Week 8, and `chisq.test()`, from base R, tests whether the two variables are associated:

```r
tab_arm <- table(strep_tb$arm, strep_tb$improved)
tab_arm
```

```
              FALSE TRUE
  Streptomycin    17   38
  Control         35   17
```

```r
chisq.test(tab_arm, correct = FALSE)
```

```

	Pearson's Chi-squared test

data:  tab_arm
X-squared = 14.176, df = 1, p-value = 0.0001665
```

The chi-squared statistic is large and the p-value is small: the proportion improved is not the same in the two arms.

Before trusting a chi-squared test, it's worth checking the assumption behind it. Every **expected** count, not observed count, should be reasonably large, conventionally at least 5. `chisq.test()` calculates these expected counts internally and stores them in `$expected`:

```r
chisq.test(tab_arm, correct = FALSE)$expected
```

```
                 FALSE     TRUE
Streptomycin 26.72897 28.27103
Control      25.27103 26.72897
```

All four expected counts are comfortably above 5, so the test is on solid ground here. When expected counts run low, typically in a small or very unevenly split sample, `fisher.test()`, also from base R, tests the same association exactly rather than by approximation, and doesn't depend on that condition.

5.6.1 Why `correct = FALSE`

We passed `correct = FALSE` above, and it's worth saying what that switches off. Left to itself on a 2×2 table, `chisq.test()` applies **Yates' continuity correction**, which shrinks the test statistic slightly to compensate for using a smooth curve to approximate counts that can only be whole numbers. It's a conservative adjustment: it makes the p-value larger.

You've already seen it in action, without being told what it was. IES Week 8 ran `prop.test()` on these same two arms and its output was headed "with continuity correction", because that function applies it by default too. The p-value it printed was 0.0003548. The uncorrected test we just ran gives 0.000166. Same data, same comparison, two different numbers, and the whole difference is this one default.

That correction earns its place when the table is thin, which is the situation it was designed for: small totals, or any expected cell count down around 5 to 10, where the approximation is under real strain. Look at the expected counts we just printed. The smallest is above 25, on 107 patients split fairly evenly four ways. Nothing here is straining, so the correction adjusts for a problem this table doesn't have, and we switch it off and read the plain Pearson test instead.

There's a practical reason too, which the next section depends on. Turning the correction off is also what makes this p-value match the one `epitools` is about to report for the same table.

5.7 Risk ratios and odds ratios, with confidence intervals

The epidemiology half already taught how to calculate a risk ratio and an odds ratio from a 2×2 table like `tab_arm` above, and what each one means. We don't re-derive those formulas here. What's new is attaching a confidence interval and a p-value to numbers you can already compute by hand, and checking that R's version agrees with yours.

`riskratio()` and `oddsratio()`, both from `epitools`, take a 2×2 table and compare its rows, treating the first row as the reference category everything else is measured against.

They expect the table a particular way round, and that's worth knowing before building one, because getting it wrong produces a perfectly ordinary-looking answer rather than an error. The **exposure goes in the rows**, with the reference group first. The **outcome goes in the columns**, and the second column is taken as the event: the thing whose risk we're comparing. Our table has `arm` in the rows and `improved` in the columns, and `improved` holds `TRUE`/`FALSE` values, which `table()` orders `FALSE` then `TRUE`, so improvement lands in the second column exactly as needed. Print the table and look at it before passing it on. If the reference row or the event column is the wrong one, these functions still return a risk ratio, just an upside-down one.

`levels()`, from base R, lists a factor's categories in their current order, which is exactly what we need to check before comparing rows:

```r
levels(strep_tb$arm)
```

```
[1] "Streptomycin" "Control"
```

`Streptomycin` is listed first, which would make it the reference, backwards from what we want here. `relevel()`, met in IES Week 7 when we checked the reference level on `eth`, moves the level named in `ref` to the front, making it the new reference level. There we were only demonstrating it; here it changes the answer:

```r
arm_control_ref <- relevel(strep_tb$arm, ref = "Control")
levels(arm_control_ref)
```

```
[1] "Control"      "Streptomycin"
```

`Control` is now first, so it becomes the reference row: the one everything else is compared against. This is the same reference level idea we met with factors in IES Week 7, now doing real work rather than being demonstrated on a copy, and it's worth having used twice before IES Week 11, where setting a sensible reference level is part of fitting a regression model.

We rebuild the table with the releveled factor, and pass it to `riskratio()`:

```r
tab_arm_releveled <- table(arm_control_ref, strep_tb$improved)
tab_arm_releveled
```

```
arm_control_ref FALSE TRUE
   Control         35   17
   Streptomycin    17   38
```

```r
riskratio(tab_arm_releveled)
```

```
$data

arm_control_ref FALSE TRUE Total
   Control         35   17    52
   Streptomycin    17   38    55
   Total           52   55   107

$measure
               risk ratio with 95% C.I.
arm_control_ref estimate    lower    upper
   Control      1.000000       NA       NA
   Streptomycin 2.113369 1.377267 3.242893

$p.value
               two-sided
arm_control_ref   midp.exact fisher.exact  chi.square
   Control                NA           NA          NA
   Streptomycin 0.0001868959 0.0002217708 0.000166482

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

`oddsratio()` works the same way. We ask for `method = "wald"`, which uses the same normal-approximation logic behind the $\pm$ 1.96 $\times$ SE intervals from IES Week 8, and which matches the simple cross-product formula from the hand calculation directly:

```r
oddsratio(tab_arm_releveled, method = "wald")
```

```
$data

arm_control_ref FALSE TRUE Total
   Control         35   17    52
   Streptomycin    17   38    55
   Total           52   55   107

$measure
```

```
                odds ratio with 95% C.I.
arm_control_ref estimate    lower   upper
   Control      1.000000       NA      NA
   Streptomycin 4.602076 2.038863 10.3877

$p.value
                two-sided
arm_control_ref   midp.exact fisher.exact  chi.square
   Control                NA           NA          NA
   Streptomycin 0.0001868959 0.0002217708 0.000166482

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

From the table above: 38 of 55 patients improved in the streptomycin arm (risk = 0.69), and 17 of 52 improved in the control arm (risk = 0.33). By hand, risk ratio = 0.69 / 0.33 = 2.11, and odds ratio = (38 × 35) / (17 × 17) = 4.60. Both match what `riskratio()` and `oddsratio()` just reported, with a confidence interval attached to each: a risk ratio of 2.11 (95% CI: 1.38 to 3.24) and an odds ratio of 4.60 (95% CI: 2.04 to 10.39). Neither interval comes close to including 1, the null value for both a risk ratio and an odds ratio, consistent with the chi-squared test above.

`riskratio()` and `oddsratio()` each also report three p-values alongside the estimate: a mid-p exact test, Fisher's exact test, and a chi-squared test. All three agree closely here. The chi-squared one is 0.000166, exactly the p-value `chisq.test(tab_arm, correct = FALSE)` gave us above, to every digit printed. That's the payoff from switching the continuity correction off: the same test, run by two different packages, now gives one number instead of two. Had we left the correction on, `chisq.test()` would have reported 0.0003548, the number IES Week 8's `prop.test()` printed, while `epitools` reported 0.000166, and there'd be no way to tell from the output that the two functions had simply made different default choices.

5.8 `gtsummary::tbl_summary()` with a test attached

`tbl_summary()`, already met in IES Weeks 7 and 8, can attach a test to a stratified table with `add_p()`, piped on afterwards:

```
strep_tb |>
  select(arm, improved) |>
```

Table 5.1

Characteristic	**Streptomycin** N = 55	**Control** N = 52	**p-value**
improved	38 (69%)	17 (33%)	<0.001

[1] n (%)

[2] Pearson's Chi-squared test

```
  tbl_summary(by = arm) |>
  add_p()
```

This single call reproduces the count, the percentage, and the p-value we built by hand above, in the one-line form we'll reach for once the manual steps are familiar. The p-value matches to every digit because `add_p()` happens to default to the *uncorrected* chi-squared test for a table like this one, the same choice we made by hand with `correct = FALSE`. That's worth checking rather than assuming whenever a shortcut and a manual calculation agree: they agree here because their defaults happen to line up, not because a shortcut is guaranteed to do what you just did by hand.

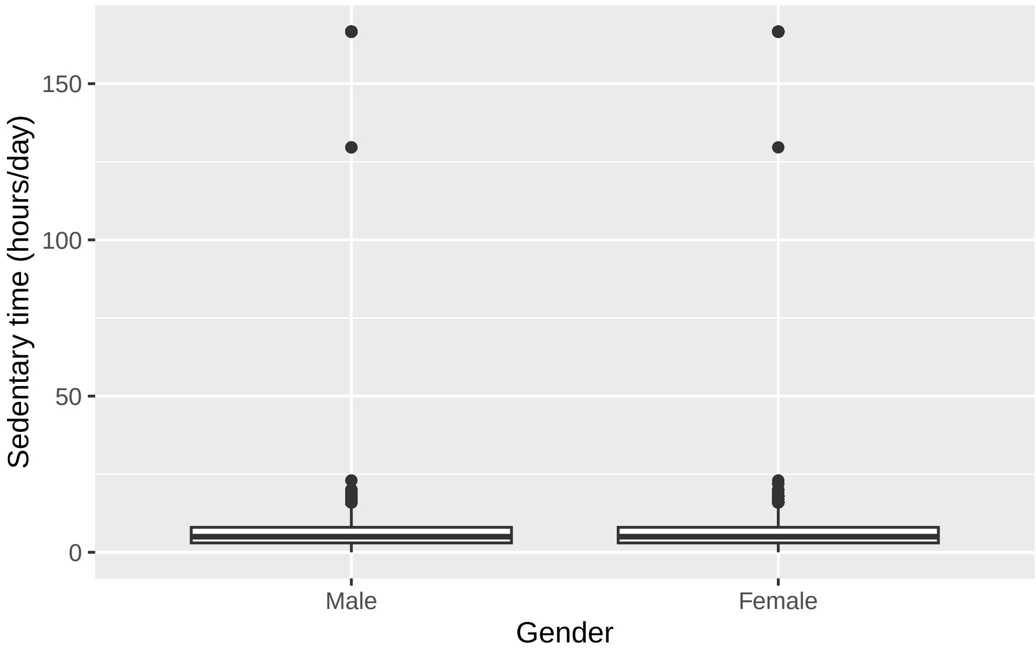
> **Try it yourself: sample size and significance**
>
> Using `slice_sample()` from IES Week 8, take a 10% random subsample of `nhanes` (remember to `set.seed()` first, so the subsample is reproducible) and re-run the `chol ~ gender` t-test from earlier in this chapter on it. Compare the p-value and the confidence interval to the ones from the full sample. Write one sentence on what changed and one sentence on what this means for judging whether a small p-value reflects a large effect or simply a large sample.

5.9 Worked example: sedentary time by gender

Let's bring the independent-samples t-test together on a pairing we haven't used elsewhere in this chapter: sedentary time (`sed`, in hours/day) by gender.

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = sed)) +
  geom_boxplot() +
  labs(x = "Gender", y = "Sedentary time (hours/day)")
```

```
Warning: Removed 7331 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```




```
t.test(sed ~ gender, data = nhanes |> filter(!is.na(gender)))
```

```

	Welch Two Sample t-test

data:  sed by gender
t = -2.2193, df = 33827, p-value = 0.02648
alternative hypothesis: true difference in means between group Male and group Female is not e
95 percent confidence interval:
 -0.48830744 -0.03028725
sample estimates:
  mean in group Male mean in group Female
            6.430592             6.689889
```

The boxplots overlap substantially, with similar spread in each group, so a t-test is reasonable here too. R prints the difference men minus women once again, so the interval on the page runs from -0.49 to -0.03. Written as it would appear in a results section, with the whole thing reversed: women reported slightly more sedentary time than men (mean difference about 0.26 hours/day, 95% CI: 0.03 to 0.49 hours/day, p = 0.026). The interval is entirely on one side of zero, so the difference is statistically significant, but it's a small effect on a scale of many hours a day.

5.10 Exercises

Exercise 1 (ILO 1). Perform an independent-samples t-test comparing age (`age`) between men and women. Check the assumption graphically first with a boxplot. Write one sentence reporting the result as it would appear in a results section, and one sentence commenting on whether a statistically significant result here is also a practically important one.

Exercise 2 (ILO 2). Using `filter()`, restrict `nhanes` to participants whose sedentary time (`sed`) is above the sample median. On this subgroup, run both a paired t-test and a Wilcoxon signed-rank test comparing `met_w` and `met_r`. Write one sentence stating whether your conclusion changes in this more sedentary subgroup, and one sentence explaining why `met_w` and `met_r` are still a paired comparison here, not an independent one, even after filtering.

Exercise 3 (ILO 3). Construct a contingency table of baseline cavitation (`baseline_cavitation`) against improvement (`improved`) in `strep_tb`. Run a chi-squared test, check the expected counts, and write one sentence interpreting the result.

Exercise 4 (ILOs 4, 5). Using the same method as the `arm`/`improved` example above, check with `levels()` which category of `gender` is already the reference, then use `relevel()` to set it explicitly (even if it turns out to already be the one you want). Obtain a risk ratio and an odds ratio, each with a 95% confidence interval, for improvement in men compared with women. Reconcile the software output with a hand calculation from the same 2×2 table, and write a two-sentence results statement.

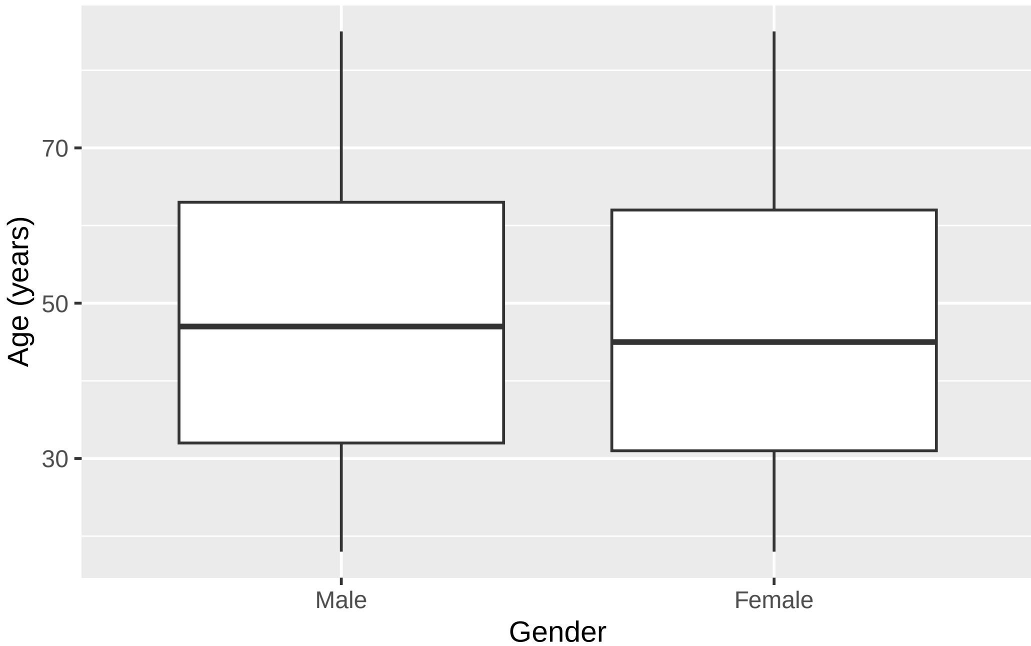
5.11 Writing exercise

Using the `arm`/`improved` result from earlier in this chapter, write two sentences, as they would appear in a results section, for `strep_tb`'s primary outcome. State the proportion improved in each arm, and report the risk ratio or the odds ratio (your choice), its 95% confidence interval, and its p-value, together.

Solutions

Exercise 1

```
ggplot(nhanes |> filter(!is.na(gender)), aes(x = gender, y = age)) +
  geom_boxplot() +
  labs(x = "Gender", y = "Age (years)")
```




```r
t.test(age ~ gender, data = nhanes |> filter(!is.na(gender)))
```

```

	Welch Two Sample t-test

data:  age by gender
t = 4.0035, df = 41503, p-value = 6.251e-05
alternative hypothesis: true difference in means between group Male and group Female is not
95 percent confidence interval:
 0.3702765 1.0805737
sample estimates:
  mean in group Male mean in group Female 
            47.89355             47.16813 
```

This time R's interval is already positive, and that's informative rather than incidental: the difference is still men minus women, so a positive interval means the first group is the higher one. Men in this sample were on average about 0.7 years older than women (95% CI: 0.4 to 1.1 years, $p < 0.001$), and no signs need reversing to say so. The difference is statistically significant but tiny relative to a population averaging around 47 years old. With a sample this large, tens of thousands of participants, even a very small true difference will produce a small p-value. Statistical significance here is not the same as a practically important difference, and it doesn't really contradict IES Week 7's impression that age looked similar across the two groups.

Exercise 2

```
sed_above_median <- nhanes |>
  filter(!is.na(sed), sed > median(sed, na.rm = TRUE))

t.test(sed_above_median$met_w, sed_above_median$met_r, paired = TRUE)
```

```

	Paired t-test

data:  sed_above_median$met_w and sed_above_median$met_r
t = 7.4471, df = 16724, p-value = 1e-13
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.3593059 0.6160133
sample estimates:
mean difference 
      0.4876596 
```

```
wilcox.test(sed_above_median$met_w, sed_above_median$met_r, paired = TRUE)
```

```

	Wilcoxon signed rank test with continuity correction

data:  sed_above_median$met_w and sed_above_median$met_r
V = 25597758, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
```

Both tests still reject the null hypothesis in this more sedentary subgroup, though the mean difference is much smaller than in the whole sample (about 0.49 MET-hours/day here, versus about 3.07 MET-hours/day overall). `met_w` and `met_r` remain two measurements taken on the same person at the same time, just restricted to a different set of people; filtering the rows doesn't change what the columns are measuring, so the comparison is still paired.

Exercise 3

```
tab_cavitation <- table(strep_tb$baseline_cavitation, strep_tb$improved)
tab_cavitation
```

```
      FALSE TRUE
  no     17   28
  yes    35   27
```

```
chisq.test(tab_cavitation, correct = FALSE)
```

```

	Pearson's Chi-squared test

data:  tab_cavitation
X-squared = 3.6399, df = 1, p-value = 0.05641
```

```
chisq.test(tab_cavitation, correct = FALSE)$expected
```

```
         FALSE     TRUE
  no  21.86916 23.13084
  yes 30.13084 31.86916
```

All expected counts are comfortably above 5, the smallest being just under 22, so `correct = FALSE` is the right call here for the same reason it was for the `arm` table.

The chi-squared test gives a p-value of about 0.056. That's above the conventional 0.05 threshold, but only just, and it would be a mistake to read much into which side of it the number landed on. The honest reading is the same either way: this is weak evidence of an association between baseline cavitation and improvement. The data lean towards there being one, they don't establish it, and they certainly don't rule it out. A trial of 107 patients simply cannot settle a question this size, and reporting it as "no association" because 0.056 exceeds 0.05 would misrepresent what the analysis found.

Exercise 4

```
levels(strep_tb$gender)
```

```
[1] "F" "M"
```

```
gender_f_ref <- relevel(strep_tb$gender, ref = "F")
tab_gender <- table(gender_f_ref, strep_tb$improved)
tab_gender
```

```
            
gender_f_ref FALSE TRUE
           F    31   28
           M    21   27
```

```r
riskratio(tab_gender)
```

```
$data

gender_f_ref FALSE TRUE Total
       F        31   28    59
       M        21   27    48
       Total    52   55   107

$measure
            risk ratio with 95% C.I.
gender_f_ref estimate     lower    upper
           F 1.000000        NA       NA
           M 1.185268 0.8215655 1.709979

$p.value
            two-sided
gender_f_ref midp.exact fisher.exact chi.square
           F         NA           NA         NA
           M  0.3745488    0.4378977   0.365451

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

```r
oddsratio(tab_gender, method = "wald")
```

```
$data

gender_f_ref FALSE TRUE Total
       F        31   28    59
       M        21   27    48
       Total    52   55   107

$measure
            odds ratio with 95% C.I.
gender_f_ref estimate     lower    upper
           F 1.000000        NA       NA
           M 1.423469 0.6619167 3.061208
```

```
$p.value
           two-sided
gender_f_ref midp.exact fisher.exact chi.square
           F         NA           NA         NA
           M  0.3745488    0.4378977   0.365451

$correction
[1] FALSE

attr(,"method")
[1] "Unconditional MLE & normal approximation (Wald) CI"
```

`levels()` shows that `"F"` is already listed first, so `relevel()` here confirms that choice explicitly rather than changing anything. By hand, from the table: 27 of 48 men improved (risk = 0.5625) and 28 of 59 women improved (risk = 0.4746), giving a risk ratio of 0.5625 / 0.4746 = 1.19, and an odds ratio of $(27 \times 31) / (21 \times 28) = 1.42$. Both match the software output. The proportion improved was slightly higher in men than in women (risk ratio 1.19, 95% CI: 0.82 to 1.71; odds ratio 1.42, 95% CI: 0.66 to 3.06, chi-squared $p = 0.37$), but both confidence intervals include the null value of 1, so this trial gives no good evidence of a difference in improvement by gender. `oddsratio()` prints three p-values, as it did for the `arm` table; the chi-squared one is quoted here because that's the one that reconciles with `chisq.test()`.

Writing exercise (sample answer)

Among patients receiving streptomycin, 69% (38 of 55) showed radiological improvement at six months, compared with 33% (17 of 52) among those receiving the control treatment. This corresponds to a risk ratio of 2.11 (95% CI: 1.38 to 3.24, $p < 0.001$), indicating substantially greater improvement with streptomycin than with control in this trial.

6 Correlation and simple linear regression (IES Week 10)

This practical follows the IES Week 10 seminar of the same name. That seminar opened with scatterplots and Pearson's correlation coefficient, then made a deliberate shift: from describing how strongly two variables move together to modelling one variable as a function of the other. It stated the simple linear regression model, Y = + X + , introduced least squares and residuals, and showed how to interpret a slope, an intercept, and R^2. It also covered the assumptions behind the model, the diagnostic plots that check them, and the danger of using a fitted line to predict somewhere the data never reached. In this session we fit that model in R, check it properly, and use it to predict.

6.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Produce and interpret a scatterplot with a fitted regression line
2. Fit a simple linear regression with `lm()`
3. Generate and interpret standard diagnostic plots
4. Generate predictions and identify when a prediction constitutes extrapolation
5. Extract coefficients and confidence intervals, and report a simple regression result in journal format

6.2 Setup

As in IES Weeks 7 to 9, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary.

This chapter adds one new package, `broom`. It isn't part of the tidyverse core, so if you haven't used it in an earlier chapter, it likely isn't installed yet. As IES Week 4 explained, installing a package and loading it are two different actions. If `broom` isn't already on your machine, run `install.packages("broom")` once in the console, not inside this document. Once it's installed, `library(broom)` below makes its functions available for this session, exactly like every other package we load. `gtsummary` isn't used anywhere in this chapter, so it isn't loaded here. `dictionary` is read in even though no code below refers to it, for the reason IES Week 8 gave: it's there so it's to hand the moment you're unsure what a column means.

```
library(readr)
library(dplyr)
library(ggplot2)
library(here)
library(broom)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))
dictionary <- read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
```

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c("Male", "Female")),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c("Mexican American", "Other Hispanic", "White", "Black", "Others")),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c("Less than high school", "High school graduate", "Above high school")),
    smoking = factor(smoking,
      levels = c("Never", "Previous", "Current")),
    phq.c = factor(phq.c,
      levels = c("0 Normal", "1 Mild symptoms", "2 Possible depression"),
      labels = c("Normal", "Mild symptoms", "Possible depression"),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c("No", "Yes"))
  )
```

This is the same recode we introduced in IES Week 7 and repeated in IES Weeks 8 and 9. See IES Week 7 for the reasoning behind each variable and each choice of `ordered = TRUE`. We repeat it here, unchanged, so this chapter stands on its own too.

6.3 Correlation: Pearson's r and Spearman's rho

The seminar opened with scatterplots and Pearson's correlation coefficient, before shifting to regression. It's worth computing both correlation coefficients here in R before we make that same shift.

`cor()`, from base R, calculates a correlation coefficient between two numeric variables. Its default method is Pearson's r, which measures the strength of a straight-line relationship between two continuous variables. We'll use the same pairing this chapter's first model uses, `age` and `chol`, so the connection between correlation and regression stays visible rather than assumed:

```
cor(nhanes$age, nhanes$chol, use = "complete.obs")
```

```
[1] 0.09398053
```

The `use = "complete.obs"` argument tells `cor()` to drop any participant missing either `age` or `chol` before calculating the coefficient; without it, a single missing value in either variable makes the whole calculation return `NA`. `cor.test()` adds a confidence interval and a p-value testing whether the true correlation in the population is zero, and it drops incomplete pairs automatically, so it needs no `use` argument of its own:

```
cor.test(nhanes$age, nhanes$chol)
```

```

	Pearson's product-moment correlation

data:  nhanes$age and nhanes$chol
t = 18.684, df = 39174, p-value < 2.2e-16
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.08415645 0.10378634
sample estimates:
       cor
0.09398053
```

Pearson's r here is about 0.094 (95% CI 0.084 to 0.104, $p < 0.001$): a weak but clearly non-zero positive correlation between age and cholesterol. Hold onto that t-value, 18.68 on 39,174 degrees of freedom: the next section fits `chol ~ age` with `lm()`, and its slope's t-value comes out identical. That's not a coincidence. With one continuous predictor and one continuous outcome, testing whether r differs from zero and testing whether the regression slope differs from zero are the same test, read on two different scales. r is unitless and treats both variables symmetrically, correlating `age` with `chol` gives the same number as correlating `chol` with `age`, while a regression slope is in the outcome's own units and only makes sense read in one direction. That's exactly the shift the seminar made, from describing how strongly two variables move together to modelling one as a function of the other.

Pearson's r assumes both variables sit on a scale where equal-sized steps mean the same thing throughout its range, an interval or ratio scale. `phq.c`, the depression symptom category, doesn't: "Possible depression" isn't a fixed distance from "Mild symptoms" the way one extra year of age is from another, it's an ordered category, not a measurement. Spearman's rank correlation is the alternative for exactly this situation. Instead of correlating the raw values, it correlates their ranks, which only needs putting observations in order, not measuring a distance between them. `phq.c` is already an ordered factor, so `as.numeric()`, from base R, which converts whatever it's given into plain numbers, turns it into the numbers 1, 2, and 3 in the order the factor was built. That's exactly the ranking Spearman's correlation needs:

```r
phq_numeric <- as.numeric(nhanes$phq.c)
cor.test(nhanes$age, phq_numeric, method = "spearman")
```

```
Warning in cor.test.default(nhanes$age, phq_numeric, method = "spearman"):
cannot compute exact p-value with ties
```

```

    Spearman's rank correlation rho

data:  nhanes$age and phq_numeric
S = 8.5857e+12, p-value = 0.3725
alternative hypothesis: true rho is not equal to 0
sample estimates:
        rho
0.004619514
```

Spearman's rho here is about 0.005, $p = 0.37$: close enough to zero, and far from significant, that age and depression symptom category show no clear monotonic relationship in this sample. The warning printed above, "cannot compute exact p-value with ties," is a computational detail rather than a problem: with tens of thousands of participants spread across only three `phq.c` categories, many participants necessarily share the same rank, so R falls back on an approximate p-value instead of an exact one. The approximate value is still valid here.

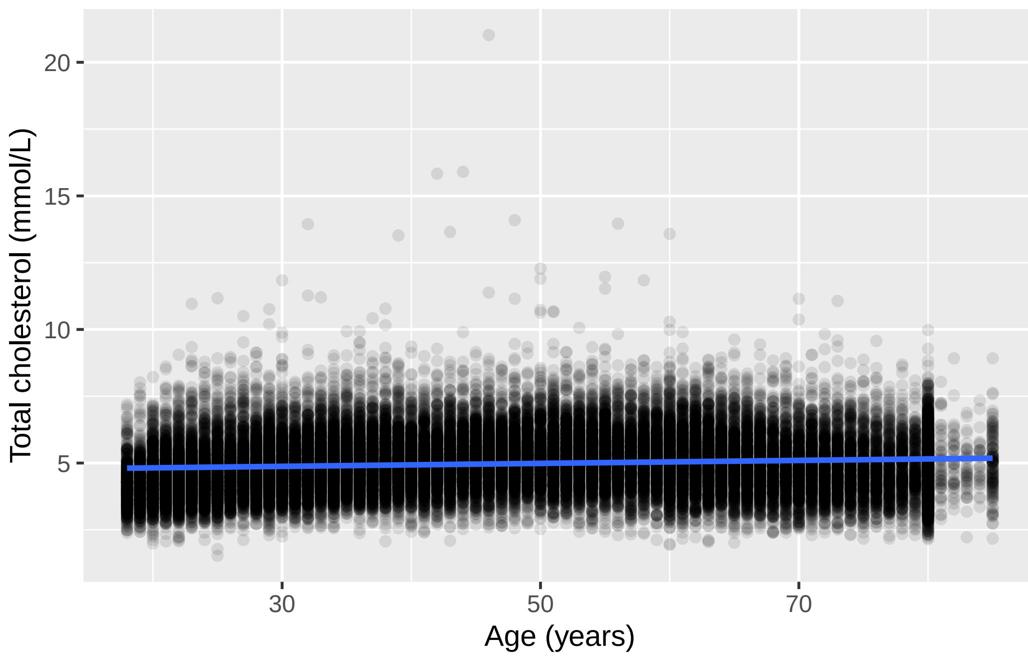
Correlation is as far as this chapter takes `phq.c`. The rest of the chapter follows the seminar's own shift, from measuring association to modelling it directly with regression.

6.4 A scatterplot with a fitted line: cholesterol by age

IES Week 7 plotted total cholesterol (`chol`, in mmol/L) against age (in years) as a plain scatterplot, with no line through it. `geom_smooth()`, from `ggplot2`, adds one. With `method = "lm"`, it fits exactly the simple linear regression this chapter is about, and draws the result:

```
ggplot(nhanes |> filter(!is.na(age), !is.na(chol)), aes(x = age, y = chol)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = "lm") +
  labs(x = "Age (years)", y = "Total cholesterol (mmol/L)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



Above the plot, `ggplot2` printed `` `geom_smooth()` using formula = 'y ~ x' ``. That's a message rather than a warning, and nothing has gone wrong: it's simply reporting which formula it fitted, because `geom_smooth()` picks one for you if you don't say. Here it chose a straight line in `x`, which is exactly the simple linear regression we asked for with `method = "lm"`. It's worth a glance the first time, because on a plot where you meant to fit something more elaborate it would tell you that you hadn't. It appears above every `geom_smooth()` plot in this chapter and always says the same thing.

The shaded band around the line is a 95% confidence interval for the line itself, the same idea as every other confidence interval in this course, just drawn as a band rather than printed as two numbers. The line slopes gently upward: cholesterol tends to be a little higher in older participants. The cloud of points around it is wide, though. Age clearly doesn't determine cholesterol on its own, and the next section puts a number on exactly how much of the picture the line accounts for.

> **Try it yourself: what the band shows**
>
> Re-run the plot above with `geom_smooth(method = "lm", se = FALSE)`. Then try `geom_smooth(method = "lm", level = 0.80)`. Write one sentence on what disappeared in the first change, and one sentence on what changed in the second.

6.5 Fitting the model with `lm()`

`lm()`, from base R, fits a linear model. Its first argument is a formula, in the same `y ~ x` form `t.test()` used in IES Week 9: the variable on the left is what we're modelling, and the variable on the right is what we're modelling it with. We drop the rows where either variable is missing first, the same habit we've used for every model and test so far:

```
age_complete <- nhanes |> filter(!is.na(age), !is.na(chol))
m_age <- lm(chol ~ age, data = age_complete)
summary(m_age)
```

```

Call:
lm(formula = chol ~ age, data = age_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.3177 -0.7469 -0.0877  0.6401 16.0554 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 4.7085437  0.0151970  309.83   <2e-16 ***
age         0.0055658  0.0002979   18.68   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.086 on 39174 degrees of freedom
Multiple R-squared:  0.008832,	Adjusted R-squared:  0.008807 
F-statistic: 349.1 on 1 and 39174 DF,  p-value: < 2.2e-16
```

As with `t.test()` in IES Week 8, there's no `$` anywhere in that call. `lm()`'s `data =` argument tells it to look for `chol` and `age` inside `age_complete`, so it finds them there directly rather than us pulling either one out by hand first. Every model in this course, `lm()` and `glm()` alike, follows this same pattern: name the variables bare in the formula, and point `data =` at wherever they live.

`summary()` prints a lot at once. The `Coefficients` table has the two numbers that define the fitted line: the `(Intercept)`, the model's predicted cholesterol at age zero, and the slope on `age`, the predicted change in cholesterol for each additional year of age. Each has a standard error, a t-value, and a p-value attached, exactly the reporting standard from IES Week 9. The slope here is small but clearly non-zero: cholesterol tends to be about 0.0056 mmol/L higher for every extra year of age, and the p-value is far below 0.001.

`Multiple R-squared`, R^2, says what fraction of the variation in cholesterol this model accounts for. Here it's under 0.01: age explains under 1% of the variation in cholesterol between participants. That isn't a contradiction of the significant slope above. A relationship can be real, precisely estimated, and still leave most of the variation in the outcome unexplained. R^2 and a p-value answer different questions, and neither one substitutes for the other.

`coef()` and `confint()`, both base R, pull out just the coefficients and just their confidence intervals, without the rest of the printout:

```r
coef(m_age)
```

```
(Intercept)         age
4.708543749 0.005565764
```

```r
confint(m_age)
```

```
                  2.5 %      97.5 %
(Intercept) 4.678757321 4.738330177
age         0.004981885 0.006149643
```

`broom::tidy()` puts the same information into a tidy data frame, one row per coefficient, which is easier to work with in code than `summary()`'s printed output. Adding `conf.int = TRUE` includes the confidence interval as two extra columns:

```r
tidy(m_age, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
```

```
1 (Intercept)  4.71     0.0152       310.  0          4.68      4.74
2 age          0.00557  0.000298      18.7 1.46e-77   0.00498   0.00615
```

Written as it would appear in a results section: total cholesterol was on average 0.0056 mmol/L higher for each additional year of age (95% CI: 0.0050 to 0.0061 mmol/L, p < 0.001), although age accounted for less than 1% of the variation in cholesterol between participants.

6.6 Checking assumptions: diagnostic plots

A fitted line is only trustworthy if the model's assumptions hold: linearity (a straight line is the right shape), constant variance (the scatter around the line doesn't change as you move along it), and normally distributed residuals (the vertical distances between the points and the line). A fourth assumption, independence between observations, isn't something a plot from one fitted model can check. It's a property of how the data were collected, and NHANES's sampling design is what would need inspecting for that, not this model's output.

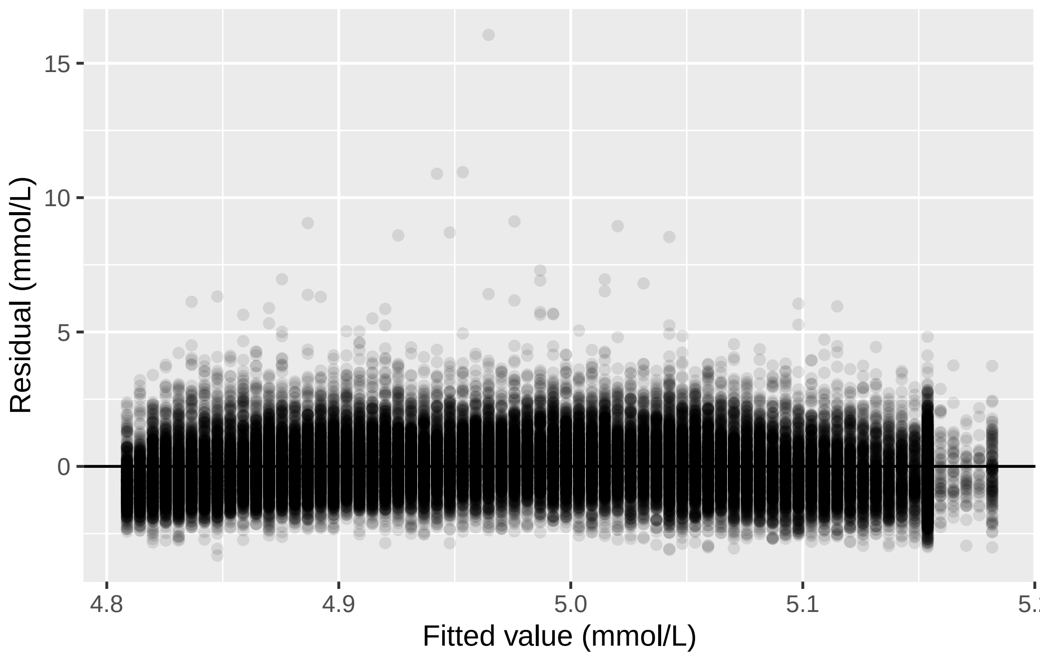
`broom::augment()` adds each observation's fitted value and residual back onto the original data, in columns named `.fitted` and `.resid`:

```r
age_aug <- augment(m_age)
age_aug
```

```
# A tibble: 39,176 x 8
    chol   age .fitted .resid      .hat .sigma     .cooksd .std.resid
   <dbl> <dbl>   <dbl>  <dbl>     <dbl>  <dbl>       <dbl>      <dbl>
 1  4.03    37    4.91 -0.884 0.0000339   1.09 0.0000112       -0.814
 2  3.75    23    4.84 -1.09  0.0000709   1.09 0.0000355       -1.00
 3  5.04    38    4.92  0.120 0.0000324   1.09 0.000000198      0.110
 4  5.56    37    4.91  0.646 0.0000339   1.09 0.00000599       0.594
 5  5.66    27    4.86  0.801 0.0000573   1.09 0.0000156        0.738
 6  8.9     30    4.88  4.02  0.0000487   1.09 0.000335         3.70
 7  3.57    30    4.88 -1.31  0.0000487   1.09 0.0000352       -1.20
 8  6.57    30    4.88  1.69  0.0000487   1.09 0.0000593        1.56
 9  6.67    22    4.83  1.84  0.0000747   1.09 0.000107         1.69
10  5.9     29    4.87  1.03  0.0000515   1.09 0.0000231        0.948
# i 39,166 more rows
```

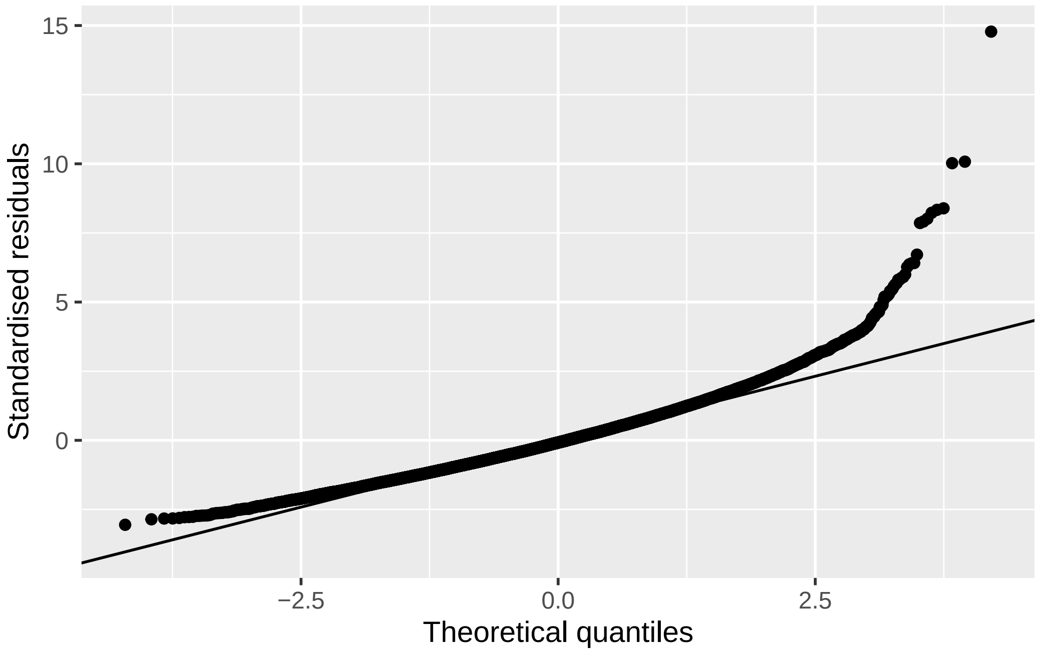
Plotting residuals against fitted values checks linearity and constant variance together. If the model is a good fit, the points should scatter evenly above and below zero, with no obvious curve and no obvious change in spread from left to right. `geom_hline()`, from `ggplot2`, draws a single horizontal line across the plot at whatever height `yintercept` names, and we put one at zero to make "above and below" easy to judge by eye:

```
ggplot(age_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = "Fitted value (mmol/L)", y = "Residual (mmol/L)")
```



Most points sit in a fairly even band around zero, consistent with linearity and roughly constant variance. A small number of participants with very high cholesterol readings sit well above the line, though, visible here as points a long way above zero. Those same participants are worth keeping in mind for the QQ plot, which checks whether the residuals are approximately normally distributed. `.std.resid`, also added by `augment()`, standardises each residual to the same scale a normal distribution would use. `stat_qq()` and `stat_qq_line()`, both from `ggplot2`, plot the standardised residuals against what a perfect normal distribution would produce, with a reference line for comparison:

```
ggplot(age_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = "Theoretical quantiles", y = "Standardised residuals")
```



Points that fall on the line are consistent with normally distributed residuals. Here, the middle of the distribution sits close to the line, but the upper tail pulls away from it. That's the same small group of participants with unusually high cholesterol showing up again: the residuals are a little more heavy-tailed than a normal distribution would predict. With a sample this large, the model's estimates are still reliable despite this. With a much smaller sample, tail behaviour like this would be more of a concern.

6.7 Predicting from the model, and the danger of extrapolation

`predict()`, from base R, uses a fitted model to generate a predicted value for a new set of inputs. Those inputs need to arrive as a data frame, with a column named to match each predictor in the model. `data.frame()`, also from base R, builds one directly from the values we supply, rather than reading one in from a file:

```
predict(m_age, newdata = data.frame(age = c(45, 150)), interval = "confidence")
```

```
       fit      lwr      upr
1 4.959003 4.948142 4.969865
2 5.543408 5.482643 5.604174
```

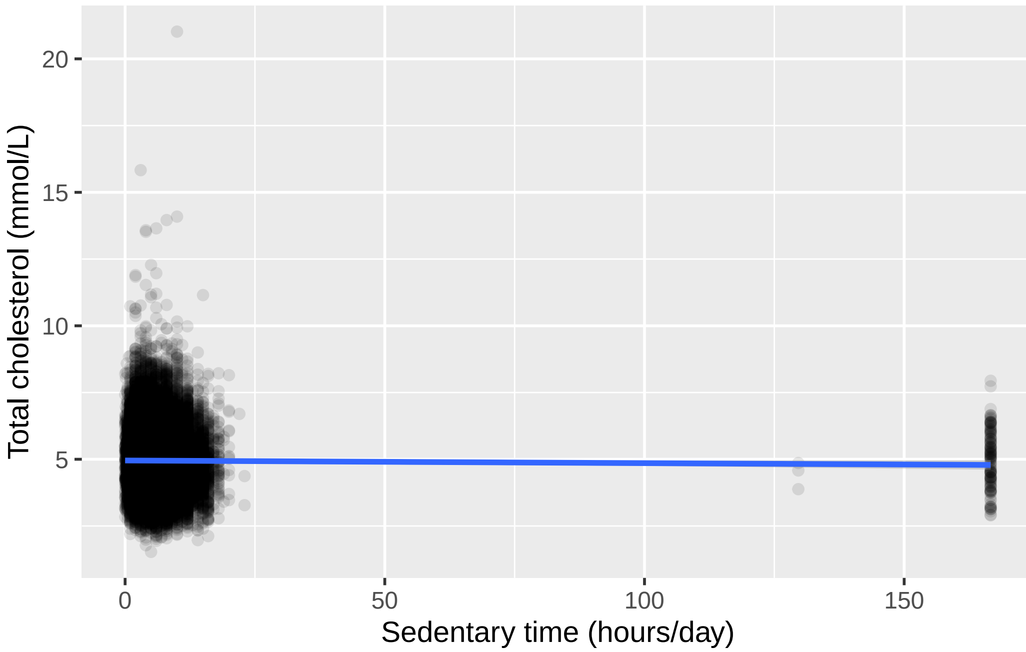
`interval = "confidence"` adds a confidence interval around each prediction. The first row, age 45, sits comfortably inside the range of ages actually observed in this sample (18 to 85), so this prediction is supported by data the model was fitted on. The second row, age 150, is not. Nobody in this dataset, or in any verified human population, is 150 years old. The model has no information about what happens at that age, and fitting a straight line through the data we do have says nothing reliable about what lies far outside it. The predicted value it returns looks like an ordinary number, which is exactly what makes extrapolation dangerous: a plausible-looking output doesn't mean the input that produced it was reasonable. Always check a prediction's input against the range the model was actually fitted on before trusting its output.

6.8 Worked example: cholesterol by sedentary time

Let's bring the whole workflow together on the pairing this course actually needs: total cholesterol (`chol`) modelled by sedentary time (`sed`, in hours/day).

```r
ggplot(nhanes |> filter(!is.na(sed), !is.na(chol)), aes(x = sed, y = chol)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = "lm") +
  labs(x = "Sedentary time (hours/day)", y = "Total cholesterol (mmol/L)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```



The fitted line is nearly flat, and the confidence band around it is wide enough to include a line with no slope at all. Fitting the model directly confirms what the plot suggests:

```
sed_complete <- nhanes |> filter(!is.na(sed), !is.na(chol))
m_sed <- lm(chol ~ sed, data = sed_complete)
summary(m_sed)
```

```

Call:
lm(formula = chol ~ sed, data = sed_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4193 -0.7563 -0.0893  0.6467 16.0757 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept)  4.9543425  0.0070430 703.445   <2e-16 ***
sed         -0.0010015  0.0005582  -1.794   0.0728 .  
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.083 on 32307 degrees of freedom
```

```
Multiple R-squared:  9.961e-05, Adjusted R-squared:  6.866e-05
F-statistic: 3.218 on 1 and 32307 DF,  p-value: 0.07282
```

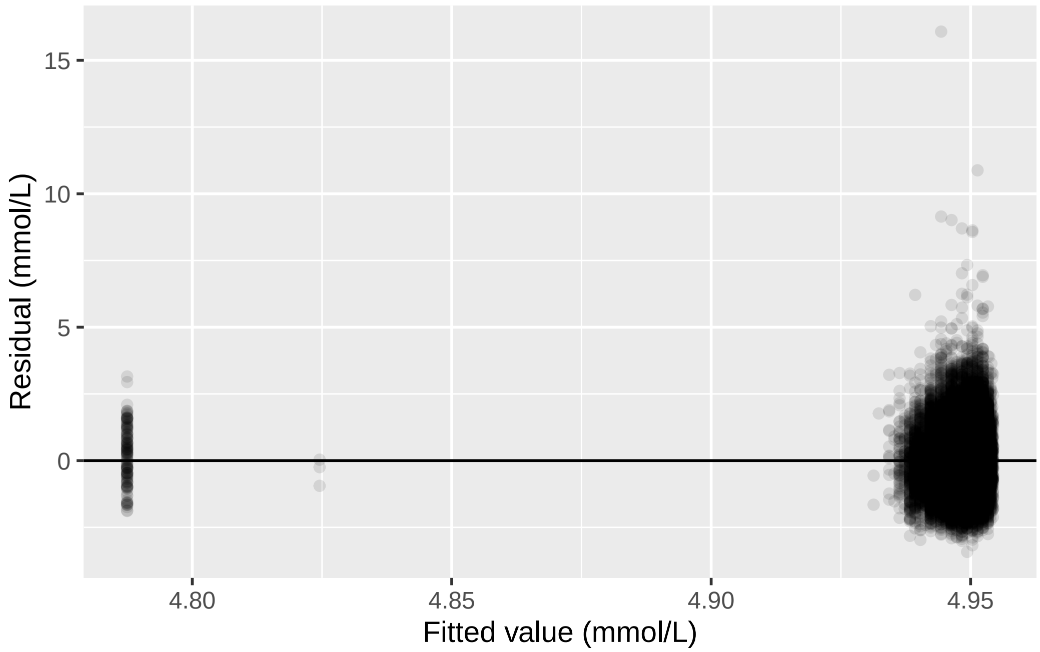
```
confint(m_sed)
```

```
                   2.5 %       97.5 %
(Intercept)  4.940538020 4.968147e+00
sed         -0.002095651 9.269529e-05
```

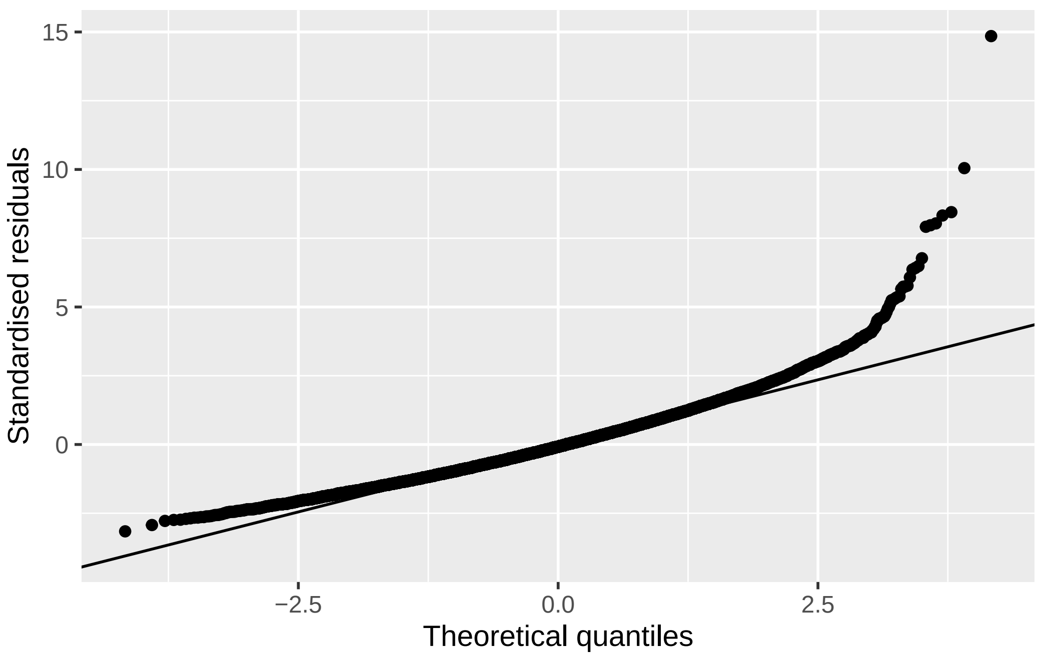
The slope is small and negative, about -0.0010 mmol/L for each additional hour/day of sedentary time, and the p-value is 0.073: above the conventional 0.05 threshold. The 95% confidence interval for the slope, -0.0021 to 0.0001 mmol/L, includes zero. R^2 is essentially zero. On this evidence alone, sedentary time and cholesterol show no clear linear relationship in this sample.

The diagnostic plots are worth checking anyway, since a weak result is not the same as a well-behaved one:

```
sed_aug <- augment(m_sed)

ggplot(sed_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = "Fitted value (mmol/L)", y = "Residual (mmol/L)")
```




```
ggplot(sed_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = "Theoretical quantiles", y = "Standardised residuals")
```



The same pattern from the age model shows up again, more visibly: a handful of participants sit far from the fitted line, and `sed` itself carries a long right tail, including some participants recording well over 24 hours of sedentary time a day, which cannot be literally true and points to some noise in how this variable was recorded or reported. None of this overturns the conclusion above. It's a reason to hold the result loosely rather than to distrust the direction of the estimate.

Written as it would appear in a results section: sedentary time was not associated with total cholesterol in this sample (slope -0.0010 mmol/L per hour/day, 95% CI: -0.0021 to 0.0001 mmol/L, p = 0.073). This is a genuinely unsatisfying result to end a chapter on, and that's deliberate. This model has exactly one predictor. It says nothing about age, nothing about gender, and nothing about anything else that might differ between people with more and less sedentary time. IES Week 11 puts those back in.

6.9 Exercises

Exercise 1 (ILOs 1, 2, 5). Using `vpa_r`, vigorous physical activity outside work (hours/day), produce a scatterplot of `chol` against `vpa_r` with a fitted regression line. Fit the model with `lm()`, and extract its coefficients and their 95% confidence intervals with `confint()` and `broom::tidy()`. Write one sentence reporting the slope as it would appear in a results section.

Exercise 2 (ILO 3). For the model fitted in Exercise 1, produce a residuals-vs-fitted plot and a QQ plot. Write one sentence describing what each plot shows, and one sentence commenting on anything the plots reveal about the shape of `vpa_r` itself.

Exercise 3 (ILO 4). Using the model fitted in Exercise 1, predict total cholesterol at `vpa_r` = 1 hour/day and at `vpa_r` = 24 hours/day. Write one sentence stating which of the two predictions can be trusted, and one sentence explaining why the other cannot, referring to the range of `vpa_r` actually observed in the data.

6.10 Writing exercise

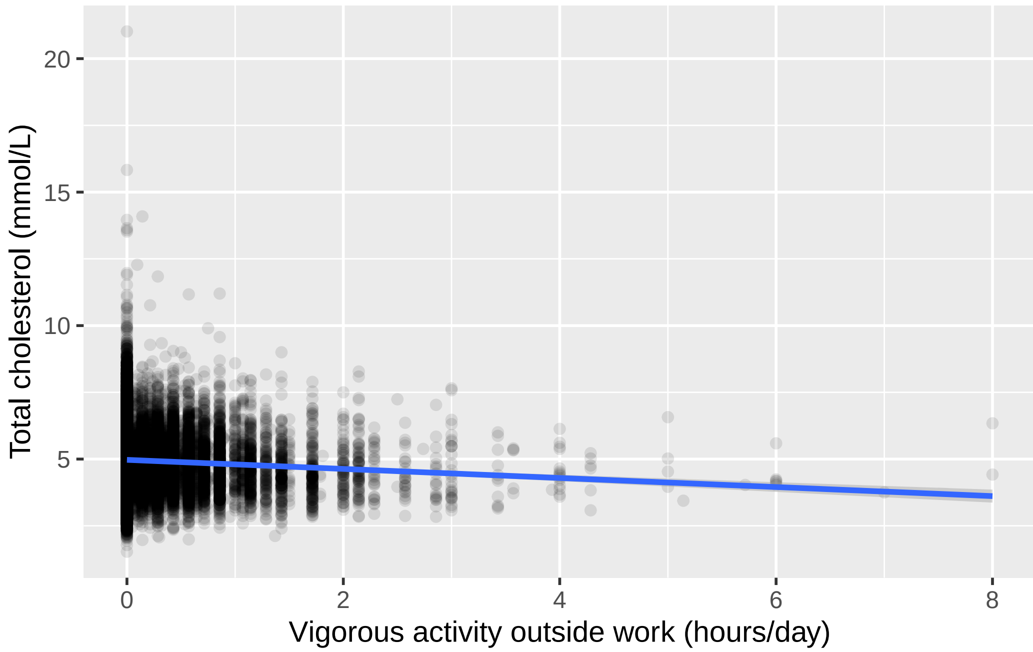
Using the `chol ~ sed` result from the worked example, write a short results paragraph of two to three sentences, as it would appear in a report. State the estimated slope, its 95% confidence interval, and its p-value, and say plainly what this evidence does and does not support. Avoid two common mistakes: don't describe a non-significant result as proof there is no relationship at all, and don't write around the p-value without stating it.

Solutions

Exercise 1

```
vpa_r_complete <- nhanes |> filter(!is.na(vpa_r), !is.na(chol))

ggplot(vpa_r_complete, aes(x = vpa_r, y = chol)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = "lm") +
  labs(x = "Vigorous activity outside work (hours/day)", y = "Total cholesterol (mmol/L)")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
m_vpa_r <- lm(chol ~ vpa_r, data = vpa_r_complete)
summary(m_vpa_r)
```

```

Call:
lm(formula = chol ~ vpa_r, data = vpa_r_complete)

Residuals:
    Min      1Q  Median      3Q     Max 
-3.4408 -0.7508 -0.0808  0.6392 16.0492 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  4.970791   0.006375  779.74   <2e-16 ***
vpa_r       -0.169796   0.015756  -10.78   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.08 on 32329 degrees of freedom
Multiple R-squared:  0.003579,	Adjusted R-squared:  0.003549 
F-statistic: 116.1 on 1 and 32329 DF,  p-value: < 2.2e-16
```

```r
confint(m_vpa_r)
```

```
                 2.5 %     97.5 %
(Intercept)  4.9582959  4.9832860
vpa_r       -0.2006789 -0.1389139
```

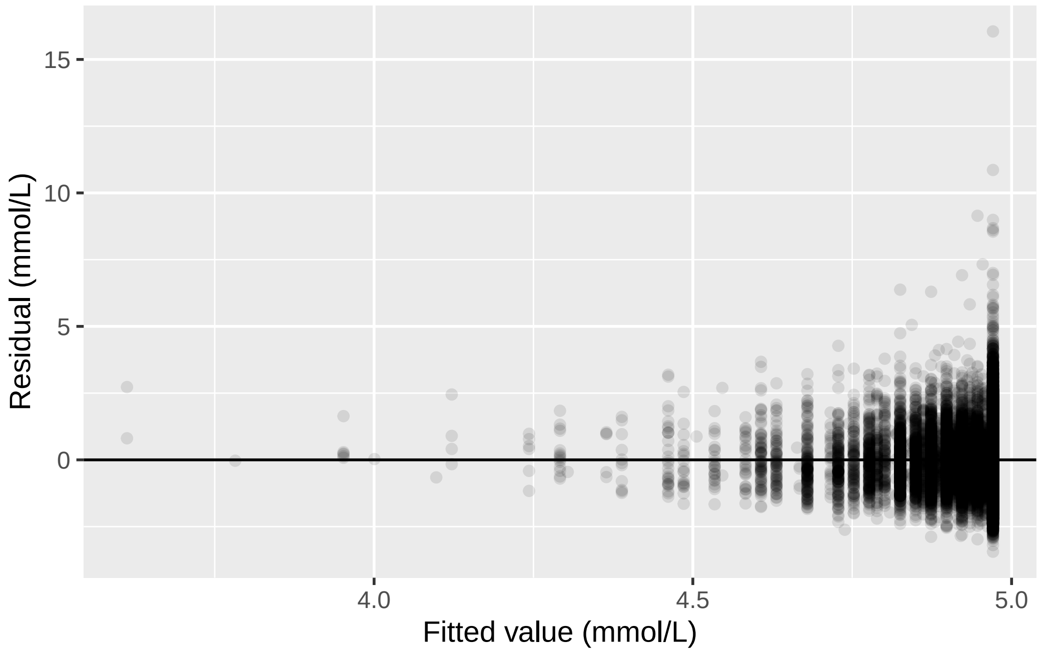
```r
tidy(m_vpa_r, conf.int = TRUE)
```

```
# A tibble: 2 x 7
  term        estimate std.error statistic  p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)    4.97    0.00637     780.  0            4.96      4.98
2 vpa_r         -0.170   0.0158      -10.8 4.94e-27   -0.201    -0.139
```

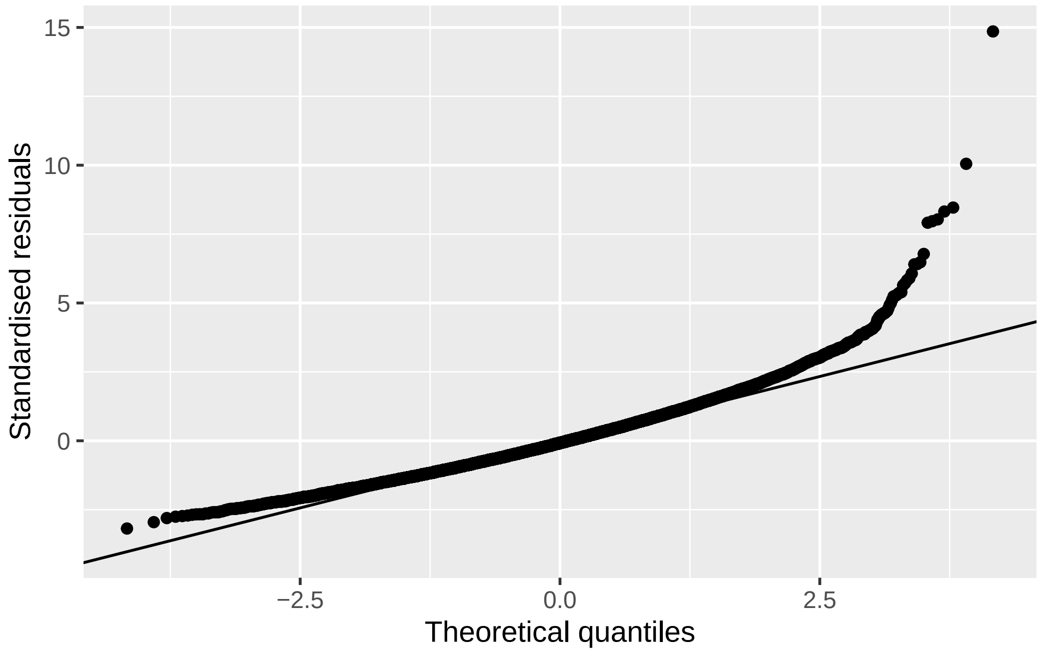
Total cholesterol was about 0.170 mmol/L lower for each additional hour/day of vigorous activity outside work (95% CI: -0.201 to -0.139 mmol/L, $p < 0.001$).

Exercise 2

```r
vpa_r_aug <- augment(m_vpa_r)

ggplot(vpa_r_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = "Fitted value (mmol/L)", y = "Residual (mmol/L)")
```




```r
ggplot(vpa_r_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = "Theoretical quantiles", y = "Standardised residuals")
```



The residuals-vs-fitted plot checks linearity and constant variance; the QQ plot checks whether the residuals are approximately normally distributed. Most points in the residual plot fall into a small number of vertical strips rather than scattering freely, because `vpa_r` itself only takes a small number of distinct values: most participants report exactly zero hours/day of vigorous activity outside work, so their fitted value and residual pattern is shared rather than spread out. This is a feature of the predictor's own distribution, not a sign that the model is wrongly specified.

Exercise 3

```
predict(m_vpa_r, newdata = data.frame(vpa_r = c(1, 24)), interval = "confidence")
```

```
        fit       lwr      upr
1 4.8009945 4.7718047 4.830184
2 0.8956766 0.1585768 1.632776
```

The prediction at `vpa_r` = 1 hour/day (about 4.80 mmol/L) can be trusted, because an hour of vigorous activity outside work a day is well within the range observed in this sample (0 to 8 hours/day). The prediction at `vpa_r` = 24 hours/day (about 0.90 mmol/L) cannot: 24 hours/day of vigorous activity is not physically possible to sustain, it is far outside the observed range, and the resulting prediction, a cholesterol level barely above zero, is not a value any living person could have. It is a consequence of extending the fitted line somewhere the data give it no support, not a real finding.

Writing exercise (sample answer)

Sedentary time was not significantly associated with total cholesterol in this sample (slope -0.0010 mmol/L per hour/day, 95% CI: -0.0021 to 0.0001 mmol/L, $p = 0.073$). The confidence interval includes zero, so these data are consistent with no true association, but they do not rule one out either; a genuinely small effect could still be present and undetected at this sample size. This model adjusts for nothing else, so it cannot say whether age, gender, or other characteristics are masking or accounting for this result.

7 Multiple linear and logistic regression (IES Week 11)

This practical follows the IES Week 11 seminar of the same name. That seminar explained confounding, connected it to the epidemiology half's own treatment of bias, and extended the simple regression model from IES Week 10 into a multiple regression with several predictors. It showed how to interpret an adjusted coefficient, how a categorical predictor's contrasts are read against a reference level, and made the connection between that and one-way analysis of variance. It then explained why a straight line is the wrong model for a 0/1 outcome, introduced the logistic model as the fix, and showed that exponentiating a coefficient turns it into an adjusted odds ratio, the same quantity the epidemiology half already taught how to calculate by hand. It closed with baseline cardiovascular disease, `blm.cvd`, as a case where the data cannot say whether a variable is a confounder or a mediator, and showed a sensitivity analysis as the honest way to report that. In this session we fit all of that in R.

7.1 Chapter intended learning outcomes

By the end of this session you will be able to:

1. Derive a fixed-window binary outcome from an event indicator and a follow-up time, and justify the exclusion rule it requires
2. Fit a multivariable linear regression, including categorical predictors with a stated reference level
3. Compare crude and adjusted estimates and quantify the change
4. Fit a logistic regression with `glm(family = binomial)`, and interpret an exponentiated coefficient as an adjusted odds ratio with a confidence interval
5. Present a pair of models differing only in one adjustment variable as a sensitivity analysis
6. Write an interpretation of an adjusted association suitable for a results section

7.2 Setup

As in IES Weeks 7 to 10, we read the data with `read_csv()` and turn the six coded variables into factors against the dictionary.

This chapter adds one new package underneath the scenes: `gtsummary`'s `tbl_regression()`, which we use for the first time here, depends on `broom.helpers`. It isn't part of the tidyverse or `gtsummary` itself, so if you haven't needed it before, it likely isn't installed yet. As IES Week 4 explained, installing a package and loading it are two different actions. If `broom.helpers` isn't already on your machine, run `install.packages("broom.helpers")` once in the console, not inside this document. We don't `library()` it directly; `gtsummary` finds it automatically once it's installed. As in IES Weeks 8 to 10, `dictionary` is read in even though no code below refers to it, so it's to hand the moment you're unsure what a column means.

```
library(readr)
library(dplyr)
library(ggplot2)
library(gtsummary)
library(here)
library(broom)

nhanes <- read_csv(here("data", "mph_nhanes_20241107.csv"))
dictionary <- read_csv(here("data", "mph_nhanes_20241107_dictionary.csv"))
```

```
nhanes <- nhanes |>
  mutate(
    gender = factor(gender,
      levels = c(1, 2),
      labels = c("Male", "Female")),
    eth = factor(eth,
      levels = c(1, 2, 3, 4, 5),
      labels = c("Mexican American", "Other Hispanic", "White", "Black", "Others")),
    edu = factor(edu,
      levels = c(1, 2, 3),
      labels = c("Less than high school", "High school graduate", "Above high school")),
    smoking = factor(smoking,
      levels = c("Never", "Previous", "Current")),
    phq.c = factor(phq.c,
      levels = c("0 Normal", "1 Mild symptoms", "2 Possible depression"),
      labels = c("Normal", "Mild symptoms", "Possible depression"),
      ordered = TRUE),
    blm.cvd = factor(blm.cvd,
      levels = c(0, 1),
      labels = c("No", "Yes"))
  )
```

This is the same recode we introduced in IES Week 7 and repeated in IES Weeks 8 to 10. See IES Week 7 for the reasoning behind each variable, including why `edu`, although its categories do have a natural order, is built as a plain factor rather than an ordered one: this chapter is exactly the reason why. We repeat the recode here, unchanged, so this chapter stands on its own too.

This chapter's shape is different from IES Weeks 7 to 10 in one respect. Every other practical in this book puts its worked content first and its exercises at the end. This one opens with an exercise instead, because the outcome variable every model in this chapter needs, `dead_10yr`, is not supplied in the data and is not built for you in the setup chunk above: you build it yourself, right now, before any of the regression content that follows. That is deliberate, not a change of habit part-way through the book. Deriving a ten-year outcome from a death indicator and a follow-up time, and deciding what to do with the participants it cannot classify either way, is itself the first thing this chapter has to teach.

7.3 Opening exercise: building a ten-year mortality outcome

`nhanes_mph` records whether each participant died during follow-up (`dth`) and how long they were followed for (`fu_month`, in months). This is the column IES Week 7 flagged as the one the dictionary doesn't fully document: it says "Death in follow-up" and stops, without telling us which value means what. So we state it here, and it's worth noticing that you had to be told rather than being able to look it up. `dth` is 1 for a participant who died during the study and 0 for one who didn't.

Neither column on its own answers the question this chapter needs: did a participant die *within ten years*? "Died" isn't a question you can answer until you say "by when."

Three groups exist in this data:

- Participants who died, and whose death happened within 120 months (10 years) of their follow-up starting. These count as an event: `1`.
- Participants known to be alive at 120 months. This includes everyone followed at least that long without dying, but it also includes anyone who died *after* 120 months: surviving to month 150 and then dying still means they were alive at month 120, so this group counts as a non-event: `0`, regardless of whether death eventually happened later on.
- Participants who did not die, but whose recorded follow-up ended before 120 months. These cannot be classified either way. They might have died the month after their follow-up ended, or lived another thirty years. Calling them "survivors" would be a guess dressed up as data, so they are excluded rather than coded as `0`.

Two tools do the work, and both are new. `case_when()`, from `dplyr`, checks a series of conditions in order and returns the first matching value, which is exactly what a three-way rule needs. Each line pairs a condition on the left of the `~` with the value to return on the right, and the last line is usually a catch-all: `TRUE` is a condition that is always met, so anything reaching it without matching a line above gets whatever that line returns. Order matters, because `case_when()` stops at the first condition a participant matches. The second tool is an argument rather than a function. `table()` normally leaves missing values out of its count, the way it did in IES Week 7; `useNA = "always"` tells it to report them as their own category instead, which is what we want here, since the participants we exclude are the point rather than a footnote.

> **! Your turn, before you read on**
>
> **Exercise (ILO 1).** Write the `mutate()` yourself. Using `case_when()`, build a new column called `dead_10yr` in `nhanes` that returns `1` for the first group above, `0` for the second, and `NA` for the third, then count the three groups with `table(nhanes$dead_10yr, useNA = "always")`.
> Two things to work out as you write it, which matter more than the syntax:
>
> 1. The rule for the second group needs no mention of `dth` at all. Why not? Think about who is in that group.
> 2. Once you have the counts, be ready to say why the third group can't simply be added to the `0`s and called survivors.
>
> Have a real go before opening the answer. Everything after this section runs on the column you're about to build, so it's worth getting your hands on it rather than reading someone else's version of it.

i Answer: building `dead_10yr`

```r
nhanes <- nhanes |>
  mutate(
    dead_10yr = case_when(
      dth == 1 & fu_month <= 120 ~ 1,
      fu_month >= 120 ~ 0,
      TRUE ~ NA
    )
  )

table(nhanes$dead_10yr, useNA = "always")
```

```

    0     1  <NA> 
14275  3748 23742 
```

The second condition is written as `fu_month >= 120` alone, with no condition on `dth`, precisely to catch both parts of that second group: `dth == 1` (died, but after month 120) and `dth == 0` (never died, followed at least that long) both belong in it. Anyone who was still being observed at month 120 was alive at month 120, and that is the whole question. The catch-all comes last and returns `NA`. That's the third group: alive, but followed for less than ten years. Had it been written first, it would have swallowed everybody.

3,748 participants died within the ten-year window, 14,275 are known to have been alive at ten years (whether or not they later died), and 23,742 are excluded because their follow-up ended too early to classify them either way. That's a large number excluded, but it's the honest cost of a fixed-window outcome: we're deliberately not using information about *when* someone died beyond whether it was inside or outside our chosen window, and we're refusing to guess about the people whose observation window closed before the ten years were up. The 3,748 events that remain are plenty to fit the regression models this chapter builds.

There is a family of methods, survival analysis, built precisely to use the follow-up time we've just thrown away and to keep the participants we've just excluded. Kaplan-Meier curves and Cox regression are its familiar tools. They're out of scope here, and Advanced Statistics is where you'll meet them.

Try it yourself: a different window

Rebuild `dead_10yr` using a five-year window (60 months) instead of ten. Report the new event and exclusion counts, and write one sentence on why a shorter window changes both numbers in the direction it does.

7.4 Extending the IES Week 10 model

IES Week 10 fitted total cholesterol (`chol`, in mmol/L) against sedentary time alone (`sed`, in hours/day), `chol ~ sed`, and found a small, non-significant slope: it adjusted for nothing else, so it couldn't rule out age (in years) or gender accounting for the pattern. We now add them, along with education (`edu`), the same way `+` adds any predictor to an `lm()` formula:

```
chol_adj_complete <- nhanes |>
  filter(!is.na(sed), !is.na(chol), !is.na(age), !is.na(gender), !is.na(edu))

m_adj <- lm(chol ~ sed + age + gender + edu, data = chol_adj_complete)
summary(m_adj)
```

```
Call:
lm(formula = chol ~ sed + age + gender + edu, data = chol_adj_complete)
```

```
Residuals:
    Min      1Q  Median      3Q     Max 
-3.4095 -0.7426 -0.0814  0.6424 15.9864 

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)    
(Intercept)              4.6614324  0.0230949 201.838   <2e-16 ***
sed                     -0.0011304  0.0005678  -1.991   0.0465 *  
age                      0.0047488  0.0003467  13.696   <2e-16 ***
genderFemale             0.1605092  0.0121807  13.177   <2e-16 ***
eduHigh school graduate -0.0130321  0.0177073  -0.736   0.4618    
eduAbove high school     0.0045031  0.0149561   0.301   0.7633    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.075 on 31222 degrees of freedom
Multiple R-squared:  0.01147,	Adjusted R-squared:  0.01131 
F-statistic: 72.43 on 5 and 31222 DF,  p-value: < 2.2e-16
```

```r
confint(m_adj)
```

```
                               2.5 %        97.5 %
(Intercept)              4.616165428  4.706699e+00
sed                     -0.002243383 -1.741808e-05
age                      0.004069176  5.428422e-03
genderFemale             0.136634488  1.843839e-01
eduHigh school graduate -0.047739058  2.167485e-02
eduAbove high school    -0.024811500  3.381773e-02
```

The coefficient on `sed` is now what this model calls the *adjusted* association between sedentary time and cholesterol: the association after holding age, gender, and education constant. "Holding constant" doesn't mean the model repeats the analysis separately for every combination of those variables. It means the model assumes the effect of an extra hour of sedentary time is the same regardless of a participant's age, gender, or education, and estimates that one shared slope from everybody at once.

7.5 Categorical predictors and reference levels

`gender` and `edu` entered that model as factors, and each one contributes a row per non-reference level rather than one row for the whole variable. `levels()`, from base R, shows a factor's categories in order; the first one listed is the reference level, the category every other row is compared against:

```r
levels(nhanes$edu)
```

```
[1] "Less than high school" "High school graduate"  "Above high school"
```

`"Less than high school"` is first, so it's already the reference we want, the same way IES Week 7 set it up when it built this factor for the first time. `relevel()`, met in IES Week 7 on `eth` and again in IES Week 9 on `strep_tb$arm`, confirms this explicitly rather than leaving it implicit:

```r
edu_ref_check <- relevel(nhanes$edu, ref = "Less than high school")
levels(edu_ref_check)
```

```
[1] "Less than high school" "High school graduate"  "Above high school"
```

Nothing changed, because `"Less than high school"` was already first. That's worth doing anyway: confirming a reference level is correct is cheap, and assuming it without checking is how a wrong reference level ends up in a report unnoticed.

Reading the coefficients table above: `eduHigh school graduate` compares that group to `"Less than high school"`, holding sed, age, and gender fixed, and `eduAbove high school` does the same for the highest education group. Neither is statistically significant here, but the reading is the same whether or not it reaches significance: each row is a difference from the one category everything else in that variable is measured against.

The same check applies to `gender`:

```r
levels(nhanes$gender)
```

```
[1] "Male"   "Female"
```

`"Male"` is first, so `"Male"` is the reference level and the model's `genderFemale` row is the difference for females relative to males, holding sed, age, and education fixed. This isn't alphabetical order, and it isn't R choosing for us: it's a direct consequence of how the setup chunk built the factor, `factor(gender, levels = c(1, 2), labels = c("Male", "Female"))`, where the numeric code `1` was listed first and labelled `"Male"`. Whichever label is attached to the first level named in `levels =` becomes the reference, so the reference level is a choice made back in the recoding step, not something to discover afterwards.

7.6 Crude versus adjusted: quantifying the change

IES Week 10's crude result and this chapter's adjusted one are worth setting side by side directly. Note that we refit the crude model here rather than reusing IES Week 10's, so that both models run on `chol_adj_complete`, the same set of participants. That matters: if the crude model used everyone with `sed` and `chol` recorded while the adjusted one dropped anybody also missing `edu`, the two estimates would differ for two reasons at once, adjustment and a different sample, and we couldn't tell which was doing the work. This is also why the crude figure below isn't identical to the one IES Week 10 reported.

```
m_crude <- lm(chol ~ sed, data = chol_adj_complete)
tidy(m_crude, conf.int = TRUE) |> filter(term == "sed")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>   <dbl>    <dbl>     <dbl>
1 sed   -0.000738  0.000570     -1.29   0.195 -0.00186  0.000379
```

```
tidy(m_adj, conf.int = TRUE) |> filter(term == "sed")
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic p.value conf.low  conf.high
  <chr>    <dbl>     <dbl>     <dbl>   <dbl>    <dbl>      <dbl>
1 sed   -0.00113  0.000568     -1.99  0.0465 -0.00224 -0.0000174
```

Crude, the slope on `sed` is about -0.00074 mmol/L per hour/day (95% CI -0.00186 to 0.00038, p = 0.195). Adjusted for age, gender, and education, it moves to about -0.00113 mmol/L per hour/day (95% CI -0.00224 to -0.00002, p = 0.047).

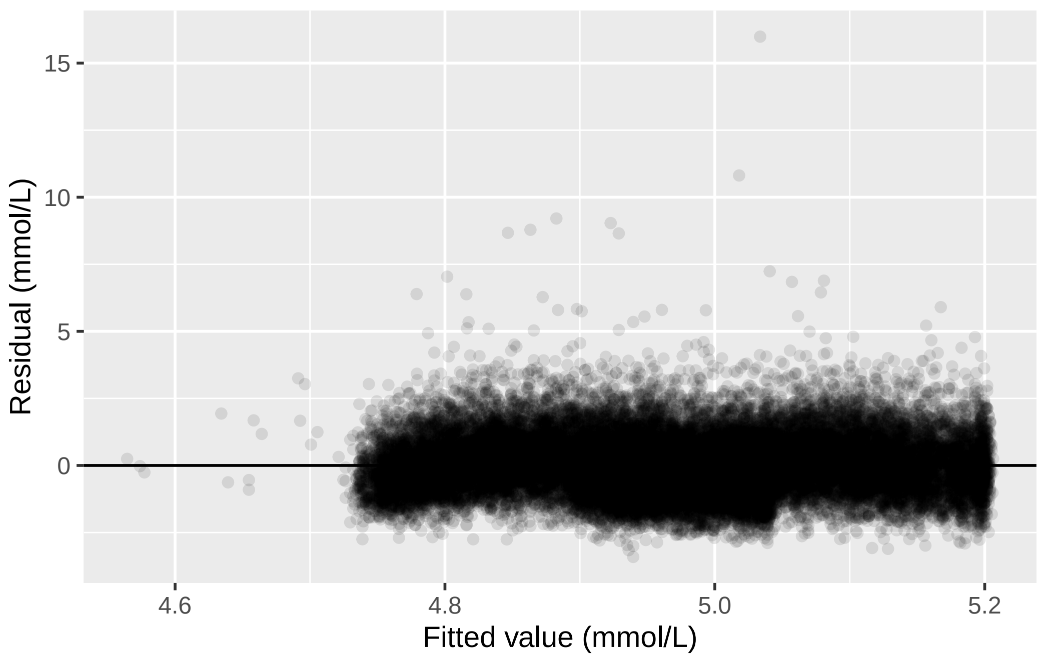
Look at the estimate first, not the p-value. The adjusted slope is about 1.5 times the size of the crude one: holding age, gender, and education constant pulls the association between sedentary time and cholesterol substantially further from zero. That movement in the estimate is what confounding looks like. Age and gender were masking part of a real association, and the model that ignores them understates it.

The p-value crosses 0.05 on the way, and it's worth being careful about how much weight that carries. It's a consequence of the estimate moving away from zero, not a separate finding, and nothing magic happened between 0.195 and 0.047. Had the adjusted interval still included zero, the shift in the estimate would have been just as real and just as worth reporting.

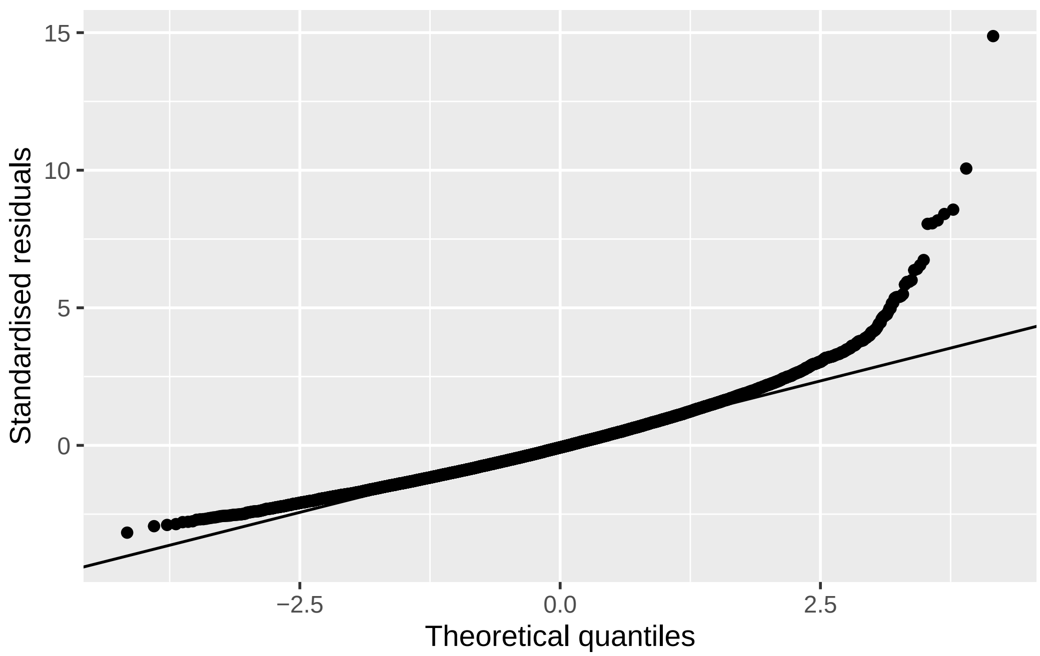
7.7 Diagnostics for the multivariable model

The same diagnostic plots IES Week 10 used still apply here, checked against the adjusted model rather than the simple one:

```r
adj_aug <- augment(m_adj)

ggplot(adj_aug, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.1) +
  geom_hline(yintercept = 0) +
  labs(x = "Fitted value (mmol/L)", y = "Residual (mmol/L)")
```




```r
ggplot(adj_aug, aes(sample = .std.resid)) +
  stat_qq() +
  stat_qq_line() +
  labs(x = "Theoretical quantiles", y = "Standardised residuals")
```



The same pattern IES Week 10 found reappears: a fairly even scatter around zero for most of the range, with a small group of participants with unusually high cholesterol pulling the upper tail away from the reference line in the QQ plot. Adding more predictors hasn't changed that; the issue lives in a handful of extreme observations, not in the number of variables in the model.

7.8 When the outcome is binary: logistic regression

`chol` is continuous, so `lm()` was the right tool. `dead_10yr` is 0 or 1, a genuinely different kind of outcome, and `lm()` is the wrong tool for it. A straight line fitted to a 0/1 outcome can, and eventually will, predict values below 0 or above 1, which aren't valid probabilities. `glm()`, base R's function for generalised linear models, fixes this by modelling the log odds of the outcome instead of the outcome directly, using maximum likelihood rather than least squares to fit the line. We don't need the machinery behind maximum likelihood to use it, only to know that the software is doing something different from `lm()` under the hood.

Fitting one looks almost identical to fitting an `lm()`. The formula is the same shape, and the only addition is `family = binomial`, which tells `glm()` the outcome is binary:

We build the complete-case sample first and fit both models on it, for the same reason the linear models above shared one sample: the crude and adjusted estimates should differ because of the adjustment, not because they were calculated on different people.

```r
dead_complete <- nhanes |>
  filter(!is.na(dead_10yr), !is.na(sed), !is.na(age), !is.na(gender), !is.na(edu))
```

```r
m_dead_crude <- glm(dead_10yr ~ sed, data = dead_complete, family = binomial)
summary(m_dead_crude)
```

```

Call:
glm(formula = dead_10yr ~ sed, family = binomial, data = dead_complete)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.162367   0.039802 -29.204  < 2e-16 ***
sed          0.039638   0.005826   6.803 1.02e-11 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12747  on 10695  degrees of freedom
Residual deviance: 12632  on 10694  degrees of freedom
AIC: 12636

Number of Fisher Scoring iterations: 5
```

```r
m_dead_adj <- glm(dead_10yr ~ sed + age + gender + edu,
  data = dead_complete, family = binomial)
summary(m_dead_adj)
```

```

Call:
glm(formula = dead_10yr ~ sed + age + gender + edu, family = binomial,
    data = dead_complete)

Coefficients:
                         Estimate Std. Error z value Pr(>|z|)
(Intercept)             -6.143091   0.138353 -44.402  < 2e-16 ***
sed                      0.019971   0.003697   5.402 6.59e-08 ***
age                      0.092216   0.001993  46.273  < 2e-16 ***
genderFemale            -0.514661   0.052973  -9.716  < 2e-16 ***
```

```
eduHigh school graduate -0.089589   0.069576  -1.288    0.198
eduAbove high school    -0.308002   0.061239  -5.030 4.92e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12746.5  on 10695  degrees of freedom
Residual deviance:  8905.5  on 10690  degrees of freedom
AIC: 8917.5

Number of Fisher Scoring iterations: 5
```

We keep this adjustment set deliberately small. `dead_complete` holds 10,696 participants, of whom 3,028 died within ten years. That's fewer events than the 3,748 we counted at the start of the chapter, because requiring a recorded value for every predictor drops participants of its own. Those 3,028 events are what the model actually has to work with, and there isn't room to add many more predictors before it runs short of events per predictor.

7.9 Odds ratios: exponentiating the coefficients

A logistic regression's coefficients are on the log-odds scale, which isn't how the epidemiology half reported odds ratios. `exp()`, base R's exponential function, undoes the log and converts a coefficient back to an odds ratio. `coef()` and `confint()` pull the coefficients and their confidence intervals the same way they did for `lm()`:

```
exp(coef(m_dead_crude))
```

```
(Intercept)         sed 
   0.312745    1.040434 
```

```
exp(confint(m_dead_crude))
```

```
Waiting for profiling to be done...
```

```
                2.5 %    97.5 %
(Intercept) 0.2888699 0.3373731
sed         1.0291891 1.0526653
```

`confint()` printed `Waiting for profiling to be done...` before the answer. That's a progress note, not a warning, and it's worth knowing what it's telling us because it marks a real difference between `glm()` and `lm()`.

For a linear model, `confint()` builds each interval the way IES Week 8 built one by hand: an estimate, plus or minus a multiplier times a standard error. For a logistic model that shortcut is only an approximation, so `confint()` does something better instead. It works along each coefficient, refitting the model repeatedly to find the range of values the data are consistent with. That search is the "profiling", and the message is `glm()` saying it's running. On this dataset it takes a second or two. On a much larger model it can take noticeably longer, which is the only reason the message exists at all.

```
exp(coef(m_dead_adj))
```

```
             (Intercept)                      sed                      age 
             0.002148272              1.020171639              1.096602080 
            genderFemale eduHigh school graduate    eduAbove high school 
             0.597703435              0.914306766              0.734914077 
```

```
exp(confint(m_dead_adj))
```

```
Waiting for profiling to be done...
```

```
                              2.5 %      97.5 %
(Intercept)             0.001633024 0.002809059
sed                     1.013277999 1.028199075
age                     1.092371089 1.100939444
genderFemale            0.538655006 0.662981437
eduHigh school graduate 0.797687946 1.047837119
eduAbove high school    0.651773505 0.828632927
```

The same message appears again above, for the same reason.

`broom::tidy()` does the same conversion in one step with `exponentiate = TRUE`:

```
tidy(m_dead_adj, exponentiate = TRUE, conf.int = TRUE) |> filter(term == "sed")
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic      p.value conf.low conf.high
  <chr>    <dbl>     <dbl>     <dbl>        <dbl>    <dbl>     <dbl>
1 sed       1.02   0.00370      5.40 0.0000000659     1.01      1.03
```

Table 7.1

Characteristic	**Beta**	**95% CI**	**p-value**
sed	0.00	0.00, 0.00	0.047
age	0.00	0.00, 0.01	<0.001
gender			
Male	—	—	
Female	0.16	0.14, 0.18	<0.001
edu			
Less than high school	—	—	
High school graduate	-0.01	-0.05, 0.02	0.5
Above high school	0.00	-0.02, 0.03	0.8

Abbreviation: CI = Confidence Interval

Crude, each additional hour/day of sedentary time is associated with an odds ratio of about 1.040 for ten-year mortality (95% CI: 1.029 to 1.053, $p < 0.001$). Adjusted for age, gender, and education, the odds ratio attenuates to about 1.020 (95% CI: 1.013 to 1.028, $p < 0.001$). Both intervals sit entirely above 1, the null value for an odds ratio, so both are statistically significant. But the adjusted estimate is noticeably smaller than the crude one: part of the crude association was really age, since older participants are both more sedentary and more likely to die within ten years for reasons that have nothing to do with sedentary behaviour itself.

Both odds ratios sit close to 1, so this is a quiet demonstration of confounding rather than a dramatic one. Exercise 2 below runs the identical comparison on vigorous activity outside work instead, where the same mechanism, age, moves the estimate by a factor of about six rather than a couple of percent. Do that exercise before deciding how seriously to take adjustment. It is the loudest example this dataset has.

7.10 Regression tables with `gtsummary`

`tbl_regression()`, from `gtsummary`, builds a publication-style table directly from a fitted model, the regression equivalent of `tbl_summary()`. For the linear model, no extra argument is needed:

```
tbl_regression(m_adj)
```

For the logistic model, `exponentiate = TRUE` reports odds ratios instead of log-odds coefficients:

Table 7.2

Characteristic	**OR**	**95% CI**	**p-value**
sed	1.02	1.01, 1.03	<0.001
age	1.10	1.09, 1.10	<0.001
gender			
Male	—	—	
Female	0.60	0.54, 0.66	<0.001
edu			
Less than high school	—	—	
High school graduate	0.91	0.80, 1.05	0.2
Above high school	0.73	0.65, 0.83	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

Table 7.3

	Crude			Adjusted		
Characteristic	**OR**	**95% CI**	**p-value**	**OR**	**95% CI**	**p-value**
sed	1.04	1.03, 1.05	<0.001	1.02	1.01, 1.03	<0.001
age				1.10	1.09, 1.10	<0.001
gender						
Male				—	—	
Female				0.60	0.54, 0.66	<0.001
edu						
Less than high school				—	—	
High school graduate				0.91	0.80, 1.05	0.2
Above high school				0.73	0.65, 0.83	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

```
tbl_regression(m_dead_adj, exponentiate = TRUE)
```

`tbl_merge()`, also from `gtsummary`, places two tables side by side under named headers, which is exactly what a crude-vs-adjusted comparison needs:

```
tbl_crude <- tbl_regression(m_dead_crude, exponentiate = TRUE)
tbl_adj <- tbl_regression(m_dead_adj, exponentiate = TRUE)

tbl_merge(list(tbl_crude, tbl_adj),
  tab_spanner = c("Crude", "Adjusted"), quiet = TRUE)
```

`quiet = TRUE` switches off a message `tbl_merge()` prints when the two tables have different numbers of rows, warning that rows might end up out of order. Different numbers of rows is precisely what we want here, since the adjusted model has three predictors the crude one doesn't, and the rows do come out in the right order. Check that yourself when you use it, though, rather than silencing the message out of habit.

7.11 The sensitivity analysis: is `blm.cvd` a confounder or a mediator?

The seminar raised a genuine difficulty with `blm.cvd`, baseline cardiovascular disease. It's plausible as a confounder: existing heart disease could make someone both less physically active and more likely to die, in which case adjusting for it removes bias. It's equally plausible as a mediator: sedentary behaviour could contribute to cardiovascular disease, which then contributes to death, in which case adjusting for it would remove part of the very effect this model is trying to estimate. Baseline CVD and sedentary time were both measured at the same study visit, so nothing in the data itself can settle which story is correct. That's not a gap in this analysis; it's a limit on what an observational snapshot can ever tell you, and the honest response is to report the estimate both ways rather than to pick one.

```r
dead_cvd_complete <- dead_complete |> filter(!is.na(blm.cvd))

m_dead_without <- glm(dead_10yr ~ sed + age + gender + edu,
  data = dead_cvd_complete, family = binomial)
m_dead_with <- glm(dead_10yr ~ sed + age + gender + edu + blm.cvd,
  data = dead_cvd_complete, family = binomial)

tbl_without <- tbl_regression(m_dead_without, exponentiate = TRUE)
tbl_with <- tbl_regression(m_dead_with, exponentiate = TRUE)

tbl_merge(list(tbl_without, tbl_with),
  tab_spanner = c("Without baseline CVD", "With baseline CVD"), quiet = TRUE)
```

	Without baseline CVD			With baseline CVD		
Characteristic	**OR**	**95% CI**	**p-value**	**OR**	**95% CI**	**p-value**
sed	1.02	1.01, 1.03	<0.001	1.02	1.01, 1.03	<0.001
age	1.10	1.09, 1.10	<0.001	1.09	1.09, 1.10	<0.001
gender						
Male	—	—		—	—	
Female	0.60	0.54, 0.66	<0.001	0.63	0.56, 0.70	<0.001
edu						
Less than high school	—	—		—	—	
High school graduate	0.91	0.80, 1.05	0.2	0.92	0.80, 1.06	0.2
Above high school	0.73	0.65, 0.83	<0.001	0.77	0.68, 0.87	<0.001
blm.cvd						
No				—	—	
Yes				2.36	2.06, 2.70	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

The odds ratio for `sed` moves only a little between the two models: 1.020 (95% CI 1.013 to 1.028, $p < 0.001$) without `blm.cvd`, and 1.019 (95% CI 1.012 to 1.026, $p < 0.001$) with it. `blm.cvd` itself is a strong predictor of ten-year mortality, but adjusting for it changes almost nothing about the sedentary time estimate. That's worth reporting honestly too: this isn't a case where the two models tell dramatically different stories. The point of running both isn't to hunt for a big swing, it's that *which* number is the right one to report, even when they're this close, depends on an assumption about `blm.cvd` that this study's design cannot verify. A results section reporting only one of these numbers would be making that assumption silently.

7.12 Exercises

Exercise 1 (ILOs 2, 3). Fit `chol ~ met_t + age + gender + edu`, where `met_t` is total physical activity in MET-hours/day, and compare it with the crude model `chol ~ met_t`. Report both slopes for `met_t` with their 95% confidence intervals and p-values, and write one sentence describing how the adjusted estimate differs from the crude one, including its direction.

Before reading the adjusted model's output, use `levels()` to state which category of `edu` the model is using as its reference. Then refit the adjusted model with `"Above high school"` as the reference instead, using `relevel()`, and write one sentence saying what changed in the `edu` coefficients and one saying what did *not* change about the `met_t` slope.

Exercise 2 (ILOs 4, 6). Fit `dead_10yr ~ vpa_r + age + gender + edu`, where `vpa_r` is vigorous physical activity outside work in hours/day, and compare it with the crude model `dead_10yr ~ vpa_r`. Exponentiate both models' coefficients to obtain odds ratios with confidence intervals, and build a `tbl_regression()` table for the adjusted model. Write one sentence interpreting the adjusted odds ratio for `vpa_r` in context.

Exercise 3 (ILO 5). Using the adjusted model from Exercise 2, fit a second version that also adjusts for `blm.cvd`. Combine both models into one table with `tbl_merge()`, and write one sentence comparing the two odds ratios for `vpa_r` and one sentence stating what each model assumes about `blm.cvd`.

7.13 Writing exercise

Write two short paragraphs, as they would appear in a results section.

In the first, report the adjusted association between sedentary time and cholesterol from this chapter's linear model, and the adjusted odds ratio for sedentary time and ten-year mortality from the logistic model, each with its 95% confidence interval and p-value. Add one sentence on what this analysis cannot establish: an observational adjustment reduces confounding by the variables actually included, but it cannot rule out confounding by variables that were never measured.

In the second, separate paragraph, report both odds ratios for `sed` from the sensitivity analysis, with and without `blm.cvd`, and state plainly what each model assumes about baseline CVD's role and why the data available here cannot decide between those assumptions.

7.14 Where this leaves you

That's the end of the course, so it's worth stopping to say what you now have and, just as importantly, what you don't.

7.14.1 What the five weeks built

The five practicals look like five topics. They were one thing, taken apart and put back together. IES Week 7 described a variable. IES Week 8 attached a confidence interval to that description, so we could say how precisely the sample pinned the population down. IES Week 9 compared two groups, and reactivated the risk ratios and odds ratios the epidemiology half had already taught you to calculate by hand, this time with uncertainty attached to them. IES Week 10 modelled one variable as a function of another. This week put several variables into that model at once, so an association could be adjusted for the others, and then swapped a continuous outcome for a binary one and got back an adjusted odds ratio, which is exactly the quantity IES Week 9 produced by hand from a 2×2 table. The pieces from IES Weeks 7 to 9 are now sitting inside one regression model.

That arc is the module's **ILO 3**, critically appraising and applying key statistical methods, and it is what this book exists for. Two of the other module outcomes ran quietly alongside it. Every writing exercise asked what a result *means* for a public health question rather than what the number is, which is **ILO 1**. And this chapter is where **ILO 4**, causality, stops being an abstraction: `blm.cvd` is equally arguable as a confounder and as a mediator, the data cannot settle it, and we reported both models rather than picking the answer we preferred. If one habit survives this course, make it that one. Deciding what to adjust for is a judgement you make from subject knowledge and study design, before the analysis. No software will make it for you, and no p-value will tell you that you got it wrong.

7.14.2 What this course has not covered

These are real boundaries, not gaps you're expected to quietly fill in yourself. Each has a home, and for most of them that home is Advanced Statistics.

- **Prognostic modelling.** Every model in this book has been *aetiological*: we asked whether one exposure is associated with one outcome, one coefficient at a time, adjusted for confounders, and everything else in the model was there to make that one coefficient trustworthy. A prognostic model asks a different question, how well can we predict who gets the outcome, and it is built differently, judged differently, and reported differently. Fitting the same `glm()` does not make it the same analysis.
- **Survival analysis.** Signposted already, where this chapter threw away `fu_month` and excluded everyone whose follow-up ended early. Kaplan-Meier curves and Cox regression use exactly the information we discarded.
- **Missing data handling.** Worth being blunt about, because it affected every result you have produced. Every model in this book is a *complete-case* analysis: anyone missing a value on any variable in the model was dropped. This chapter's logistic model ran on 10,696 of the 41,765 participants in the dataset. We said so each time, which is the minimum, but saying so is not the same as handling it. Methods that use the incomplete records rather than deleting them, multiple imputation chief among them, are not covered here.
- **Sample size calculation based on regression.** IES Week 8 sized a two-group comparison. Sizing a study around an adjusted regression coefficient is a different calculation.
- **Meta-analysis**, which combines estimates across studies rather than producing one from a single dataset.
- **Mediation analysis.** This chapter raised the confounder-versus-mediator ambiguity and stopped at reporting both models. Formally decomposing an effect into a direct and an indirect part is a separate method, and stopping where we did is the right place for an introductory course to stop.
- **Time-varying confounding**, where a confounder changes over the course of follow-up and standard adjustment stops working.

Two boundaries named earlier belong on the same list. **Survey weighting**, from IES Week 7: we have treated a complex survey as a simple cohort throughout, so nothing here is a national estimate. And the logistic-specific material this week's seminar set aside, **model fit statistics, pseudo-R², logistic diagnostics, interaction, and prediction**.

None of that makes what you have learned provisional. An adjusted estimate with a confidence interval, reported honestly, with its assumptions stated and its limits named, is the working core of applied epidemiology. It is most of what you will read in the literature and most of what you will be asked to produce. Knowing where it stops is part of knowing how to use it.

Solutions

Exercise 1

```
met_t_complete <- nhanes |>
  filter(!is.na(met_t), !is.na(chol), !is.na(age), !is.na(gender), !is.na(edu))

m_met_t_crude <- lm(chol ~ met_t, data = met_t_complete)
m_met_t_adj <- lm(chol ~ met_t + age + gender + edu, data = met_t_complete)

tidy(m_met_t_crude, conf.int = TRUE) |> filter(term == "met_t")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>   <dbl>    <dbl>     <dbl>
1 met_t -0.000772  0.000426     -1.81  0.0700 -0.00161 0.0000632
```

```
tidy(m_met_t_adj, conf.int = TRUE) |> filter(term == "met_t")
```

```
# A tibble: 1 x 7
  term  estimate std.error statistic  p.value conf.low conf.high
  <chr>    <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 met_t  0.00166  0.000443      3.74 0.000182 0.000790   0.00253
```

Crude, the slope on `met_t` is about -0.00077 mmol/L per MET-hour/day (95% CI: -0.00161 to 0.00006, $p = 0.07$). The interval includes zero, so on its own this says very little: these data are consistent with no association between total activity and cholesterol, and they don't rule one out either.
Adjusted for age, gender, and education, the slope changes sign and becomes clearly non-zero, at about 0.00166 mmol/L per MET-hour/day (95% CI: 0.00079 to 0.00253, $p < 0.001$). Once those three are held constant, more total activity is associated with slightly *higher* cholesterol in this sample. That's a reminder that adjustment can change

a result's direction, not just its size, and that a crude estimate near zero can hide an association rather than rule one out. Both models are fitted on `met_t_complete`, the same participants, so the difference between them is the adjustment and nothing else.
Now the reference level:

```
levels(met_t_complete$edu)
```

```
[1] "Less than high school" "High school graduate"  "Above high school"
```

```
met_t_relevelled <- met_t_complete |>
  mutate(edu = relevel(edu, ref = "Above high school"))

m_met_t_adj2 <- lm(chol ~ met_t + age + gender + edu, data = met_t_relevelled)
tidy(m_met_t_adj2, conf.int = TRUE)
```

```
# A tibble: 6 x 7
  term                estimate std.error statistic  p.value conf.low conf.high
  <chr>                  <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)          4.63     0.0216     215.     0          4.58e+0   4.67
2 met_t                0.00166  0.000443     3.74   1.82e- 4   7.90e-
4   0.00253
3 age                  0.00504  0.000356    14.2    1.71e-45   4.34e-
3   0.00574
4 genderFemale         0.169    0.0124      13.6    4.37e-42   1.45e-
1   0.193
5 eduLess than hig~ -0.00489  0.0150      -0.327 7.44e- 1 -3.42e-
2   0.0244
6 eduHigh school g~ -0.0188   0.0153      -1.23   2.20e- 1 -4.89e-
2   0.0113
```

`"Less than high school"` is listed first, so that's the reference the original model used, and both `edu` rows in its output were differences from that group. After `relevel()`, `"Above high school"` is the reference, and the two `edu` rows now compare the other groups against *it* instead. The numbers change, and so do their signs, because they're answering a different question: not "how does this group compare with the least-educated group?" but "how does this group compare with the most-educated group?"
The `met_t` slope doesn't move at all. It's identical in both models, to every decimal place. Changing which category a factor is measured against changes how that factor's own coefficients read, and nothing else: the model is fitting the same thing either way, just describing one part of it from a different starting point. That's worth seeing once, because it's easy to assume a reference level is a modelling decision that affects everything. It

isn't. It's a reporting decision, and it affects only the rows belonging to the variable you changed.
Note that here we *did* write the releveled factor back into a data frame, rather than working on a copy as IES Week 7 did. That's because this time we need to fit a model with it, and `lm()` has to find the changed factor inside the data it's given. We put it in a new object, `met_t_relevelled`, rather than overwriting `met_t_complete`, so the original model above stays reproducible.
Exercise 2

```
vpa_r_dead_complete <- nhanes |>
  filter(!is.na(vpa_r), !is.na(dead_10yr), !is.na(age), !is.na(gender), !is.na(edu))

m_vpa_r_crude <- glm(dead_10yr ~ vpa_r, data = vpa_r_dead_complete, family = binomial)
m_vpa_r_adj <- glm(dead_10yr ~ vpa_r + age + gender + edu,
  data = vpa_r_dead_complete, family = binomial)

tidy(m_vpa_r_crude, exponentiate = TRUE, conf.int = TRUE) |> filter(term == "vpa_r")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic  p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 vpa_r    0.0650     0.196     -14.0 2.49e-44   0.0437    0.0941
```

```
tidy(m_vpa_r_adj, exponentiate = TRUE, conf.int = TRUE) |> filter(term == "vpa_r")
```

```
# A tibble: 1 x 7
  term   estimate std.error statistic      p.value conf.low conf.high
  <chr>     <dbl>     <dbl>     <dbl>        <dbl>    <dbl>     <dbl>
1 vpa_r     0.376     0.172     -5.68 0.0000000137    0.264     0.520
```

```
tbl_regression(m_vpa_r_adj, exponentiate = TRUE)
```

Characteristic	OR	95% CI	p-value
vpa_r	0.38	0.26, 0.52	<0.001
age	1.10	1.09, 1.10	<0.001
gender			
Male	—	—	
Female	0.58	0.52, 0.65	<0.001
edu			
Less than high school	—	—	
High school graduate	0.94	0.82, 1.07	0.4
Above high school	0.79	0.70, 0.89	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

Adjusted for age, gender, and education, each additional hour/day of vigorous activity outside work is associated with an odds ratio of about 0.376 for ten-year mortality (95% CI: 0.264 to 0.520, $p < 0.001$): participants reporting more vigorous activity had substantially lower odds of dying within ten years, holding age, gender, and education constant.

Now compare that with the crude model, because the gap is the biggest one in this chapter. Crude, the odds ratio is 0.065, which would say that an extra hour a day of vigorous activity cuts the odds of dying by something like 94%. Adjusted, it is 0.376: still a substantial association, but a fraction of the size. Almost all of that difference is age. Older participants do far less vigorous activity and are far more likely to die within ten years, for reasons that have nothing to do with the activity itself, so a crude comparison of the active against the inactive is largely a comparison of the young against the old. This is the same confounding story the chapter told with `sed`, in a much louder version: there the adjusted estimate moved from 1.040 to 1.020, here it moves by a factor of about 6. Reporting the crude figure on its own would badly overstate what vigorous activity does.

Exercise 3

```
vpa_r_cvd_complete <- vpa_r_dead_complete |> filter(!is.na(blm.cvd))

m_vpa_r_without <- glm(dead_10yr ~ vpa_r + age + gender + edu,
  data = vpa_r_cvd_complete, family = binomial)
m_vpa_r_with <- glm(dead_10yr ~ vpa_r + age + gender + edu + blm.cvd,
  data = vpa_r_cvd_complete, family = binomial)

tbl_vpa_without <- tbl_regression(m_vpa_r_without, exponentiate = TRUE)
tbl_vpa_with <- tbl_regression(m_vpa_r_with, exponentiate = TRUE)

tbl_merge(list(tbl_vpa_without, tbl_vpa_with),
  tab_spanner = c("Without baseline CVD", "With baseline CVD"), quiet = TRUE)
```

Characteristic	Without baseline CVD OR	95% CI	p-value	With baseline CVD OR	95% CI	p-value
vpa_r	0.38	0.26, 0.52	<0.001	0.41	0.29, 0.56	<0.001
age	1.10	1.09, 1.10	<0.001	1.09	1.09, 1.09	<0.001
gender						
Male	—	—		—	—	
Female	0.58	0.52, 0.65	<0.001	0.61	0.55, 0.68	<0.001
edu						
Less than high school	—	—		—	—	
High school graduate	0.94	0.82, 1.07	0.4	0.94	0.82, 1.08	0.4
Above high school	0.79	0.70, 0.89	<0.001	0.82	0.73, 0.93	0.002
blm.cvd						
No				—	—	
Yes				2.34	2.05, 2.68	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

The odds ratio for `vpa_r` moves a little between the two models: 0.376 (95% CI 0.264 to 0.520, $p < 0.001$) without `blm.cvd`, and 0.409 (95% CI 0.289 to 0.564, $p < 0.001$) with it. That's a smaller shift than the main adjustment step but still a real one. Without `blm.cvd`, the model assumes that any relationship between baseline CVD and mortality operates entirely through the predictors already in the model; with it, the model assumes `blm.cvd` is a separate cause of death, worth adjusting for directly. Both are defensible readings of a single measurement taken at one study visit, and nothing in this dataset can say which one is correct.

Writing exercise (sample answer)

Sedentary time was associated with total cholesterol after adjustment for age, gender, and education (slope -0.00113 mmol/L per hour/day, 95% CI -0.00224 to -0.00002, $p = 0.047$), and with higher odds of ten-year all-cause mortality (adjusted odds ratio 1.020, 95% CI 1.013 to 1.028, $p < 0.001$). Both estimates adjust only for the variables included in this model; they cannot rule out confounding by factors that were never measured, such as overall frailty or comorbidity not captured here.

In a sensitivity analysis, the adjusted odds ratio for sedentary time and ten-year mortality was 1.020 (95% CI 1.013 to 1.028) without baseline cardiovascular disease in the model, and 1.019 (95% CI 1.012 to 1.026) with it included. The first model assumes baseline CVD's influence on mortality runs entirely through the other adjustment variables; the second treats it as an independent contributor to mortality risk worth adjusting for directly. Because sedentary behaviour and baseline CVD were both measured at the same study visit, this dataset cannot establish which assumption is correct, and both estimates are reported for that reason.